Chair of Information Systems and Business Process Management (i17)
Department of Informatics
TUM School of Computation, Information and Technology
Technical University of Munich

Structure-Aware, Cost-Controlled Petri Net Analysis in Rust

Struktur- und Kostenbewusste Analyse von Petri-Netzen in Rust

Michael Dyer

Thesis
for the Attainment of the Degree
Master of Science

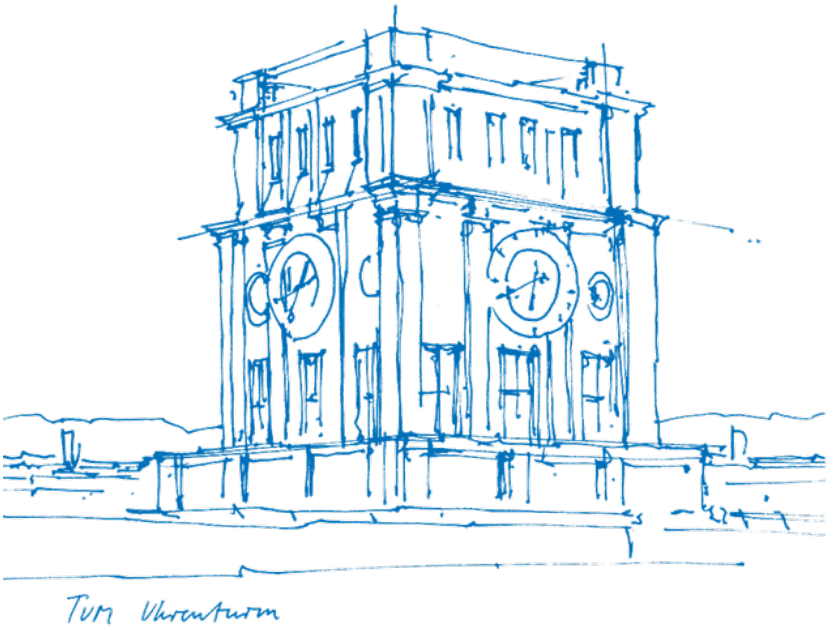
at the TUM School of Computation, Information and Technology,
Department of Informatics,
Chair of Information Systems and Business Process Management (i17)

Examiner
Prof. Dr. Stefanie Rinderle-Ma

Supervised by
Dr. Jürgen Mangler

Submitted by
Michael Dyer

Submitted on
02.11.2026



Acknowledgements

I would like to thank my parents for their unwavering support and encouragement throughout my academic journey.

And my girlfriend, for always being there for me, and providing emotional support and laughter during stressful times.

My advisor Dr. Jürgen Mangler, for his guidance, mentorship, and fortune-cookie wisdom.

And Prof. Dr. Javier Esparza, for his invaluable teaching and mastery of the subject matter, which has greatly influenced my understanding and appreciation of Petri nets.

Abstract

Abstract re-check the “Dispatch by net class” entry of every row against the tool’s documentation and source. The column was filled in as a side note when the survey was done, but dispatching by net structure has since become a central claim of the thesis (RQ2), so each entry must be verified before the finding below is relied on. Related Work Protocol Appendix Eval tool adapters combining queries like LoLA liveness appendix Evaluation figures avoid median for performance Evaluation memory figure avoid symbol overlap Double check start-up time and memory for each tool and whether the measurement floor was subtracted Bibliography entries

contribution: analyzer built into a process engine judged by cost control

- structured survey
- open source implementation
- design rests on
  1. recognize before deciding (build time classification)
  2. decide properties by a ladder of procedures
  3. caller controls the cost
  4. return unknown instead of guessing
  5. answers come with evidence - witnesses, counterexamples
  6. internal representation decoupled from api

linear time algorithms with proofs of correctness evaluation of correctness, dispatch, cost control, and comparison with established tools design principles for sound cost-aware analysis

- ordering procedures by worst-case complexity is not the same as ordering by expected cost
- interface layer should contain no logic
- scoped to only ordinary petri nets without temporal logic, non goal to be fastest analyzer

should reference chapters where the contribution

Contents

Abstract ... 3
1. Introduction ... 10
   1.1. Motivation ... 11
      1.1.1. Requirements for a Petri net Analyzer ... 12
   1.2. Research Questions ... 13
   1.3. Contribution ... 13
   1.4. Methodology ... 14
   1.5. Background ... 15
      1.5.1. Structure ... 15
      1.5.2. Tokens and Markings ... 17
      1.5.3. Transition Firing Semantics ... 17
         1.5.3.1. Incidence Matrix ... 18
         1.5.3.2. Marking Equation ... 19
      1.5.4. Behavioral Properties ... 20
         1.5.4.1. (K-)Boundedness ... 20
         1.5.4.2. Reachability ... 20
         1.5.4.3. Coverability ... 21
         1.5.4.4. Liveness ... 22
         1.5.4.5. Deadlock-freedom ... 23
      1.5.5. Structural Properties ... 23
         1.5.5.1. Traps and Siphons ... 23
         1.5.5.2. P-invariants and T-invariants ... 25
         1.5.5.3. Structural Subclasses ... 26
   1.6. Evaluation ... 31
   1.7. Structure ... 31
2. Related Work ... 32
   2.1. Methodology ... 32
   2.2. Survey of Tools ... 34
   2.3. Findings ... 36
   2.4. Dispatching by Net Structure ... 41
   2.5. The Gap ... 41
   2.6. Threats to Validity ... 42
3. Solution Design ... 44
   3.1. Design Goals ... 44
      3.1.1. API Design ... 44
   3.2. Functional Requirements ... 45
   3.3. Non-Functional Requirements and Pseudo-Requirements ... 46
4. Implementation ... 47
   4.1. Why Rust ... 47
   4.2. NetBuilder ... 47
   4.3. PNML Import and Export ... 51
   4.4. Petri Net Construction and Simulation ... 52
      4.4.1. Tokens and Markings ... 52
      4.4.2. Basic Simulation ... 54
   4.5. State Space Exploration ... 56
   4.6. Structural Properties ... 59
      4.6.1. Traps and Siphons ... 59

4.6.2. P-Invariants ... 61
4.6.3. Commoner/Hack Criterion ... 62
4.7. Behavioral Properties ... 64
4.7.1. (K-)Boundedness ... 65
4.7.1.1. Scope and Granularity ... 65
4.7.1.2. API ... 65
4.7.1.3. Implementation ... 65
4.7.2. Liveness ... 67
4.7.2.1. Scope and Granularity ... 67
4.7.2.2. API ... 67
4.7.2.3. Implementation ... 67
4.7.3. Deadlock-Freedom ... 70
4.7.3.1. Scope and Granularity ... 70
4.7.3.2. API ... 70
4.7.3.3. Implementation ... 71
4.7.4. Reachability and Coverability ... 74
4.7.4.1. Scope and Granularity ... 74
4.7.4.2. API ... 74
4.7.4.3. Implementation ... 75
4.7.5. CEGAR Engine ... 77
4.8. Closing Remarks ... 83
5. Evaluation ... 85
5.1. Method ... 85
5.1.1. Properties ... 85
5.1.2. Net Collections ... 86
5.1.3. Tools under Comparison ... 95
5.1.4. Reference Verdicts ... 96
5.1.5. Measurement Setup ... 97
5.1.6. Analysis ... 98
5.2. Correctness ... 98
5.3. Benefit of Procedure Dispatch ... 101
5.4. Cost-Control ... 110
5.5. Comparison with Established Tools ... 112
5.6. Assessment against the Remaining Requirements ... 124
5.7. Threats to Validity ... 127
6. Discussion ... 128
6.1. Research Questions Revisited ... 128
6.2. Strengths and Weaknesses ... 129
6.3. Implementation Discussion ... 130
7. Conclusion ... 133
7.1. Future Work ... 133
Bibliography ... 135
8. Appendix ... 143
8.1. Related Work Protocol Amendments ... 143
8.2. Proof of Correctness of Algorithm 5 ... 143
8.3. Proof of Correctness of Algorithm 6 ... 146
8.4. Proof of Correctness of Algorithm 7 ... 148
8.5. Proof of Correctness of Algorithm 8 ... 149

List of Tables

Table 1 Table of the 37 tools surveyed, their properties, and the scores for each requirement. Properties: B = boundedness, D = deadlock-freedom, L = liveness, R = reachability, C = coverability. 37
Table 2 The four collections of nets used in the evaluation. 86
Table 3 The fifteen families of parametrized nets used in the evaluation, with their properties and a small example. sm = state machine, mg = marked graph, fc = free-choice, ac = asym. choice. DF = deadlock-free, B = bounded, L = live (L_4), QL = quasi-live (L_1). 88
Table 4 Verdicts of the three modes of `petrivet` that could be compared with a reference verdict, per collection, the disagreements, the one-sided 95 percent upper bound on the disagreement rate of the automatic mode, and the share of runs without a verdict. 99
Table 5 Reachability and coverability on the randomized nets: verdicts of `petrivet` dispatch against the reference. "No verdict" counts the runs with a reference verdict for which `petrivet` returned no verdict. 100
Table 6 Performance summary of automatic dispatch against brute-force on the parametrized families, by structural class. 102
Table 7 The largest size parameters that each mode answers every query for without timing out, and the performance ratio of the two modes at the highest size where neither mode times out. A dash means that there is no such size. 106
Table 8 The number of runs where the efficient methods return a verdict, where they do not, where they return an incorrect verdict, and where they return a verdict while brute-force times out, for each collection of nets. 110
Table 9 Statistics of the overshoot of `petrivet`'s return time beyond the requested deadline, for deadlines of 10, 100 and 1,000 milliseconds. 111
Table 10 The ten largest deadline overshoots. 112
Table 11 Queries answered by each tool on the parametrized families (144 per property), and the penalized average time in seconds under each count. 113
Table 12 Queries answered by each tool on the 90 small MCC models, and the penalized average time in seconds under each count. 113
Table 13 Queries answered by each tool on the 20 large MCC models, and the penalized average time in seconds under each count. 114
Table 14 The largest size of each family at which each tool decides all four properties within the limit (a dash: no such size). The sizes tried are the powers of two from 2 to 1,024. 120
Table 15 Start-up cost of each tool: wall time and peak memory on deadlock-freedom of a trivial net. 124

List of Figures

Figure 1 A net with one place P_1, one transition T_1, and one arc from P_1 to T_1. 16
Figure 2 A place with one token. 17
Figure 3 T_1 is enabled because both of its input places are marked. Firing it consumes one token from each of P_1 and P_2 and produces one token in P_3. 18
Figure 4 A net and its incidence matrix. T_2 both consumes from and produces into P_3, so $C(P_3, T_2) = 0$: the matrix does not show that T_2 needs a token in P_3 to be enabled. 19
Figure 5 The marking equation of the net in Figure 4. For example, firing T_1 twice and T_2 once ($X = (2, 1)$) gives $M = (0, 1, 1)$, which is in fact reachable by the firing sequence $T_1 T_2 T_1$. 19

Figure 6 An unbounded Petri net: T_1 has no input places, so it is always enabled, and every firing adds a token to P_1. 20
Figure 7 The net of Figure 3 with its initial marking (filled tokens) and a target marking (outlined tokens).
The target in (a) is reached by firing T_1 once. The target in (b) is unreachable: it requires firing T_1 twice, but P_2 only has one token. 20
Figure 8 A Petri net in which each transition T_k is strictly L_k-live. T_0 is dead because P_0 is never marked. T_1 can fire at most once, since it consumes the only token in P_1. T_3 can fire arbitrarily many times, adding a token to P_2 each time, but never again after T_1 fires. T_2 can then fire once for each token in P_2, so any finite number of times, but not infinitely often. T_4 can always eventually fire in every reachable marking, so it is live. 22
Figure 9 The places $\{P_1, P_2\}$ (shaded) form a trap: every transition that consumes from them (T_1, T_2) also produces into them. It is unmarked initially, but once T_3 fires it stays marked forever. 23
Figure 10 The places $\{P_1, P_2\}$ (shaded) form a siphon: every transition that produces into them (T_1, T_2) also consumes from them. It is marked initially, but once T_3 fires it stays unmarked forever. 24
Figure 11 The siphon $\{P_1, P_2\}$ (light) contains the trap $\{P_2\}$ (dark), and no other trap. In (a) the trap is marked, so the siphon can never become empty. In (b) the trap is unmarked, and firing T_3 empties the siphon forever. 25
Figure 12 A net with places ordered $P_1, ..., P_4$. $y = (1, 1, 0, 0)$ is a P-invariant: $1 * M(P_1) + 1 * M(P_2) = 1$ in every reachable marking.
$y = (0, 0, 0, 1)$ is a P-subinvariant, because T_3 removes tokens from P_4 and no transition adds any, and $y = (0, 0, 1, 0)$ is a P-surinvariant, because T_3 adds tokens to P_3 and no transition removes any. 26
Figure 13 A net with the T-invariants $x = (1, 0, 1)$ and $x = (0, 1, 1)$ over T_1, T_2, T_3: firing T_1 and then T_3, or T_2 and then T_3, returns the net to the marking it started from. 26
Figure 14 A circuit with two places and two transitions. 27
Figure 15 The building block of state machines: a transition with one input and one output place. 27
Figure 16 A state machine which is not a circuit: the left place has two output transitions and the right place has two input transitions. 28
Figure 17 The building block of marked graphs: a place with one input and one output transition. 28
Figure 18 A marked graph which is not a circuit: the left transition has two output places and the right transition has two input places. 28
Figure 19 A free-choice net which is neither a state machine nor a marked graph: the left place has two output transitions, and the upper transition has two output places. 29
Figure 20 An extended free-choice net which is not an ordinary free-choice net: the two transitions share both of their input places. 29
Figure 21 An asymmetric-choice net which is not an extended free-choice net: the input places of the upper transition are a proper subset of those of the lower transition. 30
Figure 22 A general net which is not an asymmetric-choice net: the two places share the middle output transition, but neither postset contains the other. 30
Figure 23 The structural subclasses of nets within the scope of this thesis. 31
Figure 24 Number of the 37 tools that meet each requirement fully (dark), partly (medium), or not/not determined (light). The ratings for R6 follow the evidence rule given above. 39

Figure 25 The state space API of `petrivet`. An explorer is started from a `PetriNet` and advanced incrementally, or run to completion with `build_graph()`. Shortcut methods on `PetriNet` do both in one step.
Dashed arrows indicate operations which do not terminate for unbounded Petri nets. The coverability side converts into a reachability graph only if no ω-marking occurs; `try_build_reachability_graph()` stops at the first ω-marking and returns the explorer, so exploration can resume. 57
Figure 26 Counterexample-guided abstraction refinement (CEGAR) paradigm. 76
Figure 27 The calibration net `cal-22-ac-mutex`: two processes, each with an idle place (p_0, p_2) and a critical section (p_1, p_3), share the lock p_4. It has three reachable markings, and both processes are never in their critical section at the same time. 87
Figure 28 A randomized general net of medium size
(6 places, 6 transitions, 18 arcs, 4 tokens). 88
Figure 29 The small MCC model `HouseConstruction-PT-00002`
(26 places, 18 transitions, 1,501 reachable markings). 93
Figure 30 The large MCC model `SquareGrid-PT-020102`
(68 places, 72 transitions, about $2.1 \cdot 10^{12}$ reachable markings). 94
Figure 31 Analysis time of automatic dispatch (blue) against brute-force (orange) per parametrized family. Net size is on the x-axis, and time in milliseconds is on the y-axis (log scale, lower is better). The shape of the symbol indicates the property being checked, an outlined symbol means that the run timed out at that time value, and a blackened symbol means that the run ran out of memory. Symbols of different properties are offset slightly on the x-axis for visibility. 104
Figure 32 Every dot is a pair of runs: the time of the automatic dispatch (x-axis) against the time of brute-force (y-axis), colored by family. 105
Figure 33 Peak memory of the automatic dispatch (blue) and brute-force exploration (orange) per family. The net size is on the x-axis, and the peak memory in MiB is on the y-axis (lower is better). The shape of the symbol indicates the property being checked, an outlined symbol means that the run timed out at that memory value, and a blackened symbol means that the run ran out of memory. Symbols of different properties are offset slightly on the x-axis for visibility. 107
Figure 34 The automatic dispatch against brute-force on the 90 small MCC models, one panel per query, before and after the optimization of ω-acceleration described above. 109
Figure 35 Distribution of the overshoot time between the deadline and return, for deadlines of 10, 100 and 1,000 ms. Bars further left mean that `petrivet` stopped closer to its deadline. . 111
Figure 36 Cactus plots for the 90 small MCC models, one panel per property: the answered queries of each tool sorted ascending by time. The further a curve extends to the right, the more queries the tool answers, and the lower it runs, the faster. 115
Figure 37 Cactus plots for the parametrized families, one panel per property, as in Figure 36. . . . 115
Figure 38 Cactus plots for the 20 large MCC models, one panel per property, as in Figure 36. 116
Figure 39 Time of `petrivet` (dispatch, x-axis) against the time of LoLA (top row) and TINA (bottom row), one dot per small MCC model and property.
Dots above the diagonal are models on which `petrivet` was faster. 117
Figure 40 The same as Figure 39 for the 20 large MCC models. 117
Figure 41 Time taken to decide four properties against the size of the net family, per tool. A hollow marker means that at least one run timed out, so the time is a lower bound. 119
Figure 42 Peak memory of `petrivet` (dispatch, x-axis) against that of LoLA (top row) and TINA (bottom row) for each small MCC model and property. Dots above the diagonal are models on which `petrivet` used less memory. A dot with a dark outline includes a run

that was ended by the memory limit, and a hollow dot includes a run which timed out (its value is the memory when it was stopped, a lower bound). 121
Figure 43 The same as Figure 42 for the 20 large MCC models. 121
Figure 44 Peak memory against the size of the family, per tool: the largest peak of the four questions at each size (logarithmic axes, MiB; the dashed line is the memory limit of 8 GiB). A hollow marker means that some question was stopped by the time limit, so its value is the memory when it was stopped (a lower bound; it can be lower for the largest sizes, because the run got less far in 30 s), and a dark outline that some question was ended by the memory limit. 122
Figure 45 Memory footprint per reachable marking against the number of places, for each tool and each MCC model on which the tool reports a verdict on the number of markings. The start-up memory of the tool is subtracted. 123

1. Introduction

Petri nets are a well-established mathematical and graphical formalism for modeling concurrent systems, first introduced by Carl Adam Petri in his 1962 dissertation [1]. Various properties of Petri nets can offer insights into the behavior of the systems they model, which can be a powerful tool in the uncovering of design flaws and the verification of domain safety invariants before a system is deployed [2], [3], [4]. While the algorithmic complexity of most Petri net analysis questions is intractable in the general case, over six decades of Petri net research has produced a wealth of theorems that can be leveraged to derive efficient decision and semi-decision procedures[1] in broad contexts [4], [5]. Mature Petri net analysis tools apply these shortcuts instead of the general case procedures where they can [6], and higher-level tools are built on top of Petri net analyzers to fit more specific use cases, such as business process model editors [7], verification services, and teaching platforms.

The Chair of Information Systems at the TU Munich develops infrastructure for business process management, including the Cloud Process Execution Engine (CPEE) [8], [9], in which processes orchestrate heterogeneous software services and human actors. A Petri net analyzer is a natural fit for this domain: process models such as Business Process Model Notation (BPMN) diagrams [10] can be translated into Petri nets, and the properties of those nets (such as boundedness, liveness, deadlock-freedom, reachability, and coverability) can be used to verify the correctness of the process model [3], [7].

It is paramount that a Petri net analysis tool is correct: it must give the right answer, and where it cannot decide, it must say so instead of guessing. An analyzer that reports a wrong result is actively harmful because it can lead to the deployment of unsafe processes. Beyond correctness, this context of use yields a set of further requirements for a Petri net analysis tool: It must be callable programmatically, ideally in-process (or at least via a command-line interface with structured input and output), for flexible integration into a larger software system. It must be portable and memory-safe, so that it can be embedded without introducing security vulnerabilities or memory leaks. The cost of analysis must be controllable by a caller that cannot wait indefinitely for a result, and its time and memory use must be adequate for the models that occur in practice. It should read and write standard model formats, return evidence with its answers, be extensible as new procedures and model extensions are needed, and be approachable for the users and contributors who will build on it, including for students and the AI agents which increasingly assist in software development.

[1] A semi-decision procedure does not always return a definitive answer.

To my knowledge no existing Petri net analysis tool meets all of these requirements together (see Section 2). This thesis presents `petrivet`[2] , a Rust library designed to fill this gap. Its central design idea is that each question is answered by the cheapest procedure that is applicable to the net at hand. `petrivet` implements a variety of decision and semi-decision procedures for Petri net analysis, including some that are novel to this work. I evaluate whether its answers are correct, whether dispatching to the cheapest procedure pays off, how well the cost of an analysis can be bounded, and how `petrivet` compares with established tools in speed and memory. I also assess it against the requirements above and discuss the design principles that emerged during development.

1.1. Motivation

Process models are used to communicate and reason about the behavior of a system. A faulty model can lead to a system that fails in practice, and the cost of such failures can be high.

Some common questions that arise when reasoning about a process model are:

- Can the process ever get stuck in a state from which it cannot proceed?

  Example: a service that waits for a human to fill out a form, but the human never receives the form.

- Can the process ever reach a state in which some activity is permanently disabled?

  Example: a machine cannot be started because a required part has been removed, and the part cannot be replaced because the machine is not running.

- Can the process accumulate an unbounded amount of resources?

  Example: a printer that receives more print jobs than it can handle, and eventually runs out of paper.

- Can the process ever reach a dangerous configuration?

  Example: two trains are routed onto the same track, or two perpendicular traffic lights both show green.

Process models can be translated into Petri nets, which provide a mathematical model for reasoning about these questions, and the analysis results can offer rich insights into the behavior of the system [3], [4]. The questions above correspond to the following properties of Petri nets:

[2] https://github.com/MichaelOwenDyer/petrivet

- **Deadlock-freedom** (Section 1.5.4.5) can prove that a process can always proceed, or identify problematic states in which it cannot.
- **Liveness** (Section 1.5.4.4) can provide granular insight into the long-term availability of activities in a process.
- **Boundedness** (Section 1.5.4.1) can verify upper limits on the amount of resources a process can accumulate, or identify problematic resource consumption patterns.
- **Reachability** (Section 1.5.4.2) and **Coverability** (Section 1.5.4.3) can prove the impossibility of undesirable configurations, or provide a concrete example of how they can be reached.

These properties are the reason Petri nets are used in business process management, manufacturing, and other domains where the correctness of a process is critical.

1.1.1. Requirements for a Petri net Analyzer

Over time the Chair of Information Systems has assembled a collection of software tools for various use cases in process orchestration. A Petri net analyzer is a natural addition, to check the correctness of process models before they are deployed.

We would like a Petri net analyzer to meet the following requirements:

R1. be callable programmatically, either in-process or through a command-line interface with structured input and output;

R2. be portable and memory-safe, so that it can be embedded into larger software systems without introducing security vulnerabilities or memory leaks;

R3. let the caller anticipate and bound the cost of a query, since a process step cannot wait indefinitely;

R4. read and write standard model formats;

R5. perform adequately in time and memory;

R6. return evidence with its answers, such as a witness or counterexample where the question has one;

R7. be extensible, so that new procedures and model extensions can be added as the need arises; and

R8. be approachable for the users and contributors who will build on it, including students and AI agents, so that it can be used correctly and extended with little effort.

Every requirement above presupposes that the analyzer is correct. Where the analyzer cannot decide, which is unavoidable for some questions, it must say “unknown” instead of guessing.

The survey in Section 2 rates existing tools against the requirements R1-R8, and finds that none of them meet all of them together.

The project was initiated by the chair’s need rather than by an observed failure of existing tools to produce correct answers. In the terminology of Peffers et al. [11], it is an objective-centered design science project: the requirements of the problem owner define the objective, and the contribution is an artifact that meets them.

1.2. Research Questions

The objective of this thesis is to design, implement and evaluate an analyzer that meets the requirements of the chair. It is made precise by five research questions. The first concerns the problem, and the others concern the artifact, with one question for each of its central claims.

- RQ1: To what extent do existing Petri net analysis tools meet the requirements R1 to R8?
- RQ2: Does the implementation give correct answers, and does it return "unknown" where it cannot decide?
- RQ3: How much does answering each question with the cheapest applicable procedure, instead of one general procedure, reduce the cost of analysis?
- RQ4: Can a caller effectively bound the cost of an analysis, either by restricting the effort or by setting a deadline?
- RQ5: How does `petrivet` compare with established tools in speed and memory?

RQ1 is answered by the survey in Section 2. RQ2 is answered in Section 5.2. The design behind RQ3 and RQ4 is described in Section 3 and Section 4, and the effect of that design is measured in Section 5.3 and Section 5.4. RQ5 is answered in Section 5.5. The requirements that cannot be reduced to a measurement are assessed in Section 5.6. The design principles that emerged during the implementation are discussed in Section 6.

1.3. Contribution

The main contribution of this thesis is `petrivet`[2], a Rust library that answers each question with the cheapest procedure applicable to the net at hand, while letting the caller bound the cost. Its evaluation addresses the correctness of the answers, the effect of dispatching, the control of cost, and the standing of the library relative to established tools. I also discuss the design principles that emerged during implementation, and the trade-offs they involved.

1.4. Methodology

Hevner [12] visualizes design science as an engine sitting between the problem space and the academic foundations, forming three interrelated cycles: the Relevance Cycle, the Design Cycle, and the Rigor Cycle.

The Relevance Cycle represents the influence of the real-world domain: the people, organizations, and technologies exhibiting a concrete problem that the research aims to address. Consistent engagement with the problem domain ensures that the research is relevant, and that the resulting artifact alleviates the problem in a meaningful way.

The Rigor Cycle grounds the system design in the theoretical knowledge base, which includes the existing literature, methodologies, and formalisms that inform the design of the artifact. The experience gained from the design and evaluation of the artifact feeds back into the knowledge base, contributing to the academic foundations of the field.

At the center of the model is the Design Cycle, which represents the iterative process of building and evaluating the artifact. The Design Cycle is informed by the problem domain and the knowledge base, and in turn informs both of them.

Hevner, March, Park, and Ram [13] establish seven critical guidelines for conducting design science research.

1. Design as an Artifact: The research must produce a viable artifact.

Hevner et al. define artifacts as constructs (vocabulary/symbols), models (abstractions/representations), methods (algorithms/practices), or instantiations (implemented systems).

2. Problem Relevance: The research must address a problem that is relevant to the real world.
3. Design Evaluation: The mere construction of an artifact does not constitute research.

The utility, quality, and efficacy of the artifact must be demonstrated by well-executed evaluation methods.

4. Research Contributions: The research must provide clear and verifiable contributions to the existing knowledge base.
5. Research Rigor: The research must apply strict, academically sound methods in both the construction and evaluation phases.
6. Design as a Search Process: The research must acknowledge the iterative nature of the design process within

a search space of possible solutions, while constrained by available resources, computational limits, and environmental factors.

7. Communication of Research: The research must be presented effectively on both a technical level for other researchers

and a higher level for a broader audience of stakeholders and decision makers.

Relevance

The guidelines of Hevner et al. [13] are addressed in this research as follows: The artifact (*Design as an Artifact*) is a software instantiation `petrivet`, together with the methods and design principles that underpin it (Section 3, Section 4). Its *Problem Relevance* is established by the chair's need for an embeddable analyzer (Section 1.1), while the survey in Section 2 shows that no existing tool completely satisfies this need (RQ1).

The *Design Evaluation* performed in Section 5 assesses correctness, the benefit of procedure dispatch, cost control, and the artifact's standing in the context of existing tools (RQ2 to RQ5). It also assesses the remaining requirements of Section 1.1 with the same rubric as the survey (Section 5.6). The library and the evaluation harness are publicly available, so the results can be checked. *Research Rigor* is maintained throughout. The survey protocol was fixed before the survey was conducted, the verdicts of `petrivet` were checked against independent reference verdicts, and threats to validity are discussed for both the survey and the evaluation.

The *Design as a Search Process* is reflected in the iterative development of `petrivet` and its procedures, in a search space of possible API designs and algorithms for the backing implementation. The available resources also limited the search: the scope was restricted to ordinary Petri nets and the five behavioral properties discussed in Section 1.5.4, while temporal logic and broader Petri net schemas were left for future work. Finally, the *Communication of Research* is aimed at two audiences: this thesis addresses researchers and practitioners in the field of Petri nets, while the artifact itself is aimed at software engineers.

1.5. Background

This section provides a brief overview of the fundamental theory of Petri nets. For further details, I recommend consulting [4], [5], [14].

1.5.1. Structure

A **net** $N = (P, T, F)$ is a connected[3] bipartite directed graph consisting of:

- a finite set of **places** P,[4]

[3] Not all authors require nets to be connected.

- a finite set of **transitions** T disjoint from P, and
- a **flow relation** $F \subseteq (P \times T) \cup (T \times P)$.

The places (often drawn as circles) and transitions (rectangles or squares) are the nodes of the graph, and the flow relation defines directed edges (often called arcs) between them (Figure 1).

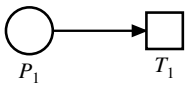


Figure 1: A net with one place P_1, one transition T_1, and one arc from P_1 to T_1.

The **preset of a place** $p \in P$, denoted $\bullet p$, is defined as $\{t \in T : (t, p) \in F\}$ (the set of transitions t such that there is a directed edge from t to p).

The **postset of a place** $p \in P$, denoted $p \bullet$, is defined as $\{t \in T : (p, t) \in F\}$ (the set of transitions t such that there is a directed edge from p to t).

The **preset of a transition** $t \in T$, denoted $\bullet t$, is defined as $\{p \in P : (p, t) \in F\}$ (the set of places p such that there is a directed edge from p to t).

The **postset of a transition** $t \in T$, denoted $t \bullet$, is defined as $\{p \in P : (t, p) \in F\}$ (the set of places p such that there is a directed edge from t to p).

For a set of nodes X, the preset and postset are the union of $\bullet x$ and $x \bullet$ over $x \in X$:

$$\bullet X := \bigcup_{x \in X} \bullet x \qquad X \bullet := \bigcup_{x \in X} x \bullet$$

In this thesis, the terms **input** and **output** are also used to refer to the preset and postset, respectively.

For nodes x and y of N, we write $x \rightsquigarrow y$ if N has a directed path of length zero or more from x to y.

For a node x,
$\text{Post}^*(x) := \{y : x \rightsquigarrow y\}$ is the set of nodes reachable from x, and
$\text{Pre}^*(x) := \{y : y \rightsquigarrow x\}$ is the set of nodes from which x is reachable.

For a set of nodes X, $\text{Pre}^*(X)$ and $\text{Post}^*(X)$ are the unions of $\text{Pre}^*(x)$ and $\text{Post}^*(x)$ over $x \in X$.

[4]The set of places is often denoted S in classical literature, from the German word "Stellen". In this thesis, I choose to use P for consistency with the English word "Place".

1.5.2. Tokens and Markings

A **marking** is a mapping $M : P \to \mathbb{N}$ that assigns a non-negative integer number of tokens to each place. Tokens are often drawn as black dots inside the places (Figure 2). Tokens are anonymous and indistinguishable from one another.[5]



Figure 2: A place with one token.

A marking M **covers** another marking M' if $M(p) \geq M'(p)$ for all $p \in P$, that is, M has at least as many tokens in every place as M'.

For a set of places $X \subseteq P$, $M(X) \coloneqq \sum_{p \in X} M(p)$ denotes the total number of tokens in X.

A place $p \in P$ is called **marked** if it contains at least one token, and **unmarked** otherwise.

A set of places $X \subseteq P$ is called marked if at least one place in the set is marked, and unmarked otherwise.

The **support** of a marking M, denoted $\langle M \rangle$, is the set of marked places in the marking: $\{p \in P : M(p) > 0\}$.

A **Petri net** (or **system**) (N, M_0) is a net N paired with an initial marking M_0.

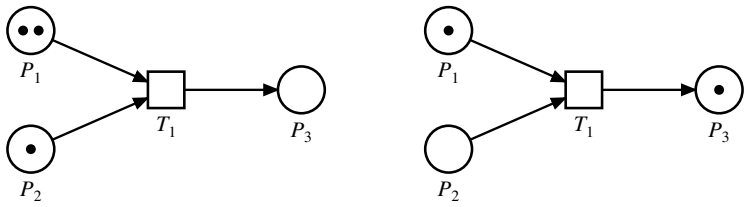
1.5.3. Transition Firing Semantics

A transition $t \in T$ is **enabled** at a marking M if all of its input places are marked, i.e. $\bullet t \subseteq \langle M \rangle$.

An enabled transition t may **fire**, consuming one token from each input place and producing one token in each output place, resulting in a new marking M' defined as follows:

$$M'(p) = \begin{cases} M(p) - 1 \text{ if } p \in \bullet t \wedge p \notin t \bullet \\ M(p) + 1 \text{ if } p \in t \bullet \wedge p \notin \bullet t \\ M(p) \text{ otherwise} \end{cases}$$

[5] See Colored Petri nets (CPNs) for tokens which carry data values. Colored Petri nets will not be discussed in this thesis, but they are a well-established extension of the classical Petri net model.



(a) Before firing T_1 (b) After firing T_1

Figure 3: T_1 is enabled because both of its input places are marked. Firing it consumes one token from each of P_1 and P_2 and produces one token in P_3.

A different set of transitions may then be enabled at the new marking M'.

A marking is called a **deadlock** if it enables no transitions.

A **firing sequence** is a sequence of transitions $\sigma = t_1 t_2 \ldots t_n$. A marking M enables a firing sequence if t_1 is enabled in M, and for each $i = 2, \ldots, n$, t_i is enabled at the marking resulting from firing $t_1, \ldots, t_{i-1}$ in order.

The **reachability set** (or reachable markings) of a Petri net (N, M_0) is the set of all markings reachable from the initial marking M_0 by firing some sequence of transitions.

1.5.3.1. Incidence Matrix

The **incidence matrix** of a net, denoted C, is a $|P| \times |T|$ integer matrix defined as follows:

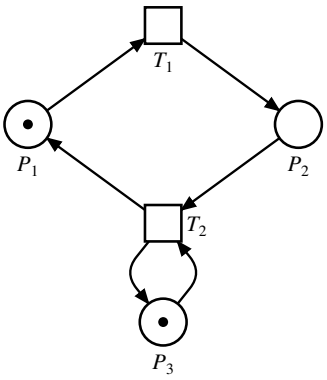
$$C(p,t) = |t\bullet \cap \{p\}| - |\bullet t \cap \{p\}|$$

for each $p \in P$ and $t \in T$.

It contains the net effect of firing each transition on the token count of each place:

- If $C(p,t) > 0$, then firing t adds $C(p,t)$ tokens to p.
- If $C(p,t) < 0$, then firing t removes $|C(p,t)|$ tokens from p.
- If $C(p,t) = 0$, then firing t does not change the token count of p.

The incidence matrix is a fundamental tool for analyzing the behavior of Petri nets, because it allows us to reason about the net's behavior algebraically. However, it is not sufficient to fully characterize the behavior of a Petri net, because it does not capture the enabling conditions of transitions.



$$C = \begin{array}{c|cc} & T_1 & T_2 \\ \hline P_1 & -1 & 1 \\ P_2 & 1 & -1 \\ P_3 & 0 & 0 \end{array}$$

Figure 4: A net and its incidence matrix. T_2 both consumes from and produces into P_3, so $C(P_3, T_2) = 0$: the matrix does not show that T_2 needs a token in P_3 to be enabled.

One can also separately define the **pre-incidence matrix** C^- and **post-incidence matrix** C^+, which contain only the negative and positive entries of C, respectively. This allows us to express the incidence matrix as $C = C^+ - C^-$, and provides a view that does not obscure read arcs (transitions that consume and produce into the same place).

1.5.3.2. Marking Equation

The **marking equation** is a linear algebraic overapproximation of the reachable markings of a Petri net [15] which relates the initial marking M_0, the incidence matrix C, and a vector of transition firing counts X to a reachable marking M: any reachable marking M can be expressed as $M = M_0 + CX$ for some non-negative integer vector X (the **Parikh vector** of the firing sequence leading to M). In other words, any reachable marking is a linear combination of transition effects added to the initial marking. The marking equation is a necessary condition for reachability, but not sufficient in general. Figure 5 shows the marking equation of the net in Figure 4.

$$\begin{pmatrix} M(P_1) \\ M(P_2) \\ M(P_3) \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \begin{pmatrix} -1 & 1 \\ 1 & -1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} X(T_1) \\ X(T_2) \end{pmatrix} \quad \Longleftrightarrow \quad \begin{cases} M(P_1) = 1 - X(T_1) + X(T_2) \\ M(P_2) = X(T_1) - X(T_2) \\ M(P_3) = 1 \end{cases}$$

Figure 5: The marking equation of the net in Figure 4. For example, firing T_1 twice and T_2 once ($X = (2, 1)$) gives $M = (0, 1, 1)$, which is in fact reachable by the firing sequence $T_1 T_2 T_1$.

1.5.4. Behavioral Properties

Behavioral properties of Petri nets are properties that emerge through the firing of transitions and the movement of tokens through the net.

1.5.4.1. (K-)Boundedness

The boundedness problem asks whether the reachability set of a Petri net is finite. This is the case if and only if there exists a finite integer k such that no reachable marking contains more than k tokens in any place. A place $p \in P$ is said to be k-bounded iff (if and only if) $M(p) \leq k$ for all reachable markings M, and a Petri net is k-bounded iff all of its places are k-bounded. If any place is not k-bounded for any finite k, it is said to be unbounded, and the Petri net as a whole is unbounded (Figure 6).

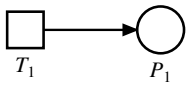
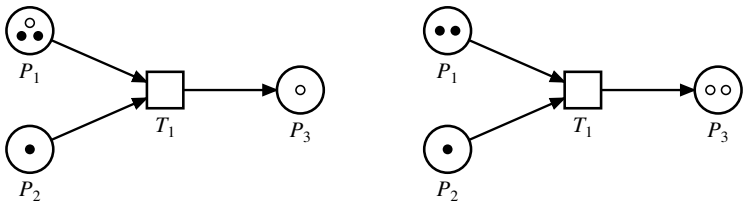


Figure 6: An unbounded Petri net: T_1 has no input places, so it is always enabled, and every firing adds a token to P_1.

1-Boundedness is also sometimes referred to as **safety**, or 1-safety. Less frequently, boundedness and safety are used synonymously.

1.5.4.2. Reachability

The reachability problem asks whether there exists an enabled firing sequence σ that leads from the initial marking M_0 to a given target marking M (Figure 7).



(a) Reachable target (b) Unreachable target

Figure 7: The net of Figure 3 with its initial marking (filled tokens) and a target marking (outlined tokens).
The target in (a) is reached by firing T_1 once. The target in (b) is unreachable: it requires firing T_1 twice, but P_2 only has one token.

Reachability was shown to be decidable independently by Ernst W. Mayr (1981) [16] and S. R. Kosaraju (1982) [17], but an its complexity was settled only recently: in 2020, Czerwiński, Lasota, Lazić, Leroux, and Mazowiecki [18]

showed that the problem is non-elementary[6] and Czerwiński and Orlikowski [19] later showed that it is Ackermann-complete[7]. It is one of the most difficult decidable problems in computer science, sitting at the frontier of what is known to be computable.

The naive approach to the reachability problem is to construct the **reachability graph** of the Petri net: fire all enabled transitions at the initial marking, then fire all enabled transitions at the resulting markings, and so on, until the target marking is reached, or until all reachable markings have been explored. Unfortunately, this approach has a fatal flaw: an unbounded Petri net has infinitely many reachable markings, so the procedure may not terminate. And even for a bounded Petri net the size of the graph may be astronomically large, making the procedure intractable for all but the smallest systems. However, if one does manage to construct the reachability graph, it provides full information about the dynamic behavior of the Petri net and can be used to decide any property of interest. The reachability graph is also called the **state space** of the Petri net.

1.5.4.3. Coverability

The coverability problem is a relaxation of the reachability problem which asks whether there exists an enabled firing sequence σ that leads from the initial marking M_0 to some marking M' which *covers* a given target marking M.

This was shown to be decidable by Karp and Miller (1969) [20] and EXPSPACE-complete by Lipton (1976) [21] (hardness) and Rackoff (1978) [22] (upper bound).

The analog of the reachability graph for the coverability problem is the **coverability graph** (or Karp-Miller graph) [20]. The coverability graph is always finite, even for unbounded Petri nets, because it uses a special symbol ω to represent an unbounded number of tokens in a place, rather than hopelessly enumerating infinitely many markings. By doing so, however, it loses some information about the exact number of tokens in unbounded places. The coverability graph can be used to decide coverability and boundedness, but not reachability, liveness, or deadlock-freedom.

In the bounded case, the coverability graph contains no ω and is identical to the reachability graph. In the unbounded case, despite the omission of infinitely

---

[6] Computational effort is not bounded by any tower of exponentials $n^{n^{n^{\cdots}}}$ of fixed height.

[7] https://en.wikipedia.org/wiki/Ackermann_function

[8] Technically, for unbounded systems, there are infinitely many possible coverability graphs, depending on the method of construction. “The” coverability graph is therefore not a well-defined concept, but rather a family of graphs with the same properties.

many markings, the coverability graph[8] can still be astronomically large, and its construction may be prohibitively expensive[9] for all but the smallest systems.

1.5.4.4. Liveness

A transition is said to be **live** iff every reachable marking enables a firing sequence containing that transition, that is, there is always a way to fire the transition again, no matter what has happened in the past. A Petri net is said to be live iff all of its transitions are live.

Liveness can be further classified into five levels, L_0 through L_4 [4], which I summarize here:

- L_0 ("dead"): No firing sequence contains the transition. The transition is not enabled in any reachable marking.
- L_1 ("not dead"): There exists a firing sequence containing the transition. The transition is enabled in at least one reachable marking.
- $L_2 \subseteq L_1$: For any finite integer k, there exists a firing sequence containing the transition k times.
- $L_3 \subseteq L_2$: There exists an infinite firing sequence containing the transition infinitely many times (there may also be reachable markings where this is no longer possible).
- $L_4 \subseteq L_3$ ("live"): From every reachable marking, there exists a firing sequence containing the transition. The transition can always be fired again.

A transition is called **strictly** L_k-live if it is L_k-live but not L_{k+1}-live.
A Petri net is L_k-live if all of its transitions are L_k-live, and strictly L_k-live if it is L_k-live but not L_{k+1}-live. The "default" notion of liveness is L_4-liveness, which is often simply called "liveness". Figure 8 gives an example of each level.

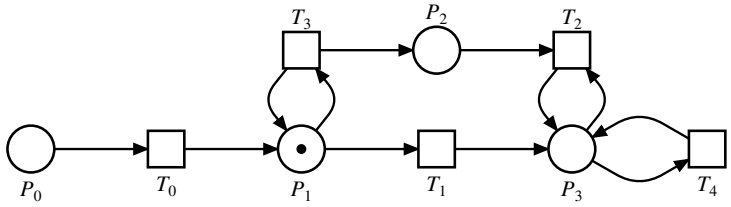


Figure 8: A Petri net in which each transition T_k is strictly L_k-live. T_0 is dead because P_0 is never marked. T_1 can fire at most once, since it consumes the only token in P_1. T_3 can fire arbitrarily many times, adding a token to P_2 each time, but never again after T_1 fires. T_2 can then fire once for each token in P_2, so any finite number of times, but not infinitely often. T_4 can always eventually fire in every reachable marking, so it is live.

[9] In fact, the extra overhead of checking for ω-markings makes constructing the coverability graph even more expensive than constructing the reachability graph in the bounded case.

Note that strict L_2-liveness is exceptionally rare due to the combination of conditions which are required to produce it: an L_3-live transition producing an unbounded number of tokens on some place(s) and an L_1-live transition which stops the production of tokens and enables the L_2-live transition.

1.5.4.5. Deadlock-freedom

A Petri net is said to be **deadlock-free** if every reachable marking enables at least one transition.

Note that deadlock-freedom is a weaker property than liveness: a live Petri net is necessarily deadlock-free, but a deadlock-free Petri net may not be live (there may be transitions which are never enabled again after some reachable marking).

Despite this, reachability, liveness, and deadlock-freedom are all reducible to one another (recursively equivalent) [23], [24]: For any pair of these properties, a Petri net can be transformed in polynomial time into another Petri net such that the first net has the first property if and only if the second net has the second property. For this reason, they all share the same Ackermann-complete complexity class.

1.5.5. Structural Properties

Structural properties relate to the structure of the net itself, irrespective of the marking. Despite this, they can hold strong implications for the behavior of the net, and are often used to prove behavioral properties.

1.5.5.1. Traps and Siphons

Traps. A trap is a set of places $Q \subseteq P$ with $Q \bullet \subseteq \bullet Q$: every transition which consumes a token from some place in the trap also produces a token in some place in the trap. Once a trap becomes marked, it is impossible to completely unmark it (Figure 9). A trap is called **proper** if it is non-empty (the empty set is trivially a trap). The union of two traps is also a trap, so every set of places $S \subseteq P$ contains a single maximal trap.

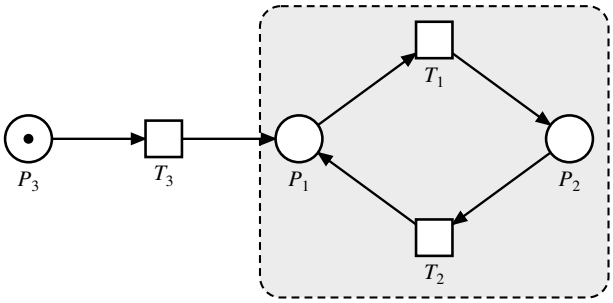


Figure 9: The places $\{P_1, P_2\}$ (shaded) form a trap: every transition that consumes from them (T_1, T_2) also produces into them. It is unmarked initially, but once T_3 fires it stays marked forever.

Siphons. A siphon is a set of places $D \subseteq P$ with $\bullet D \subseteq D \bullet$: every transition which produces a token in a place in the siphon also consumes a token from some place in the siphon. Once a siphon becomes unmarked, it is impossible to mark it again (Figure 10). A siphon is called **proper** if it is non-empty (the empty set is trivially a siphon). The union of two siphons is also a siphon, so every set of places $S \subseteq P$ contains a single maximal siphon.

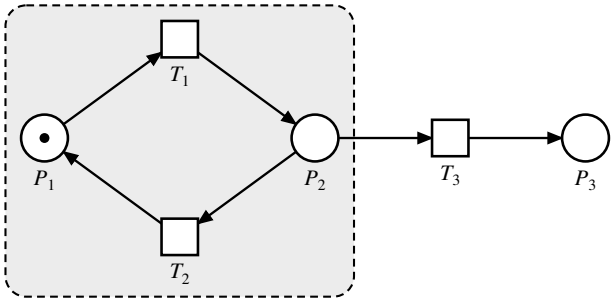
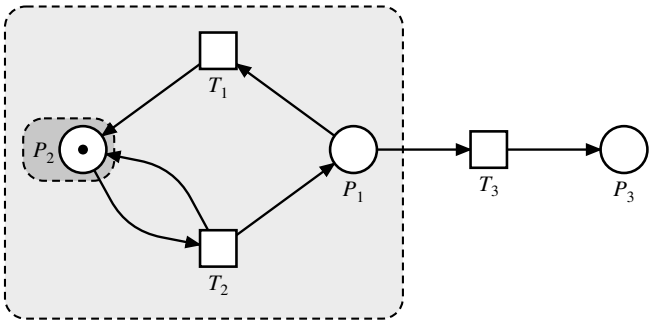


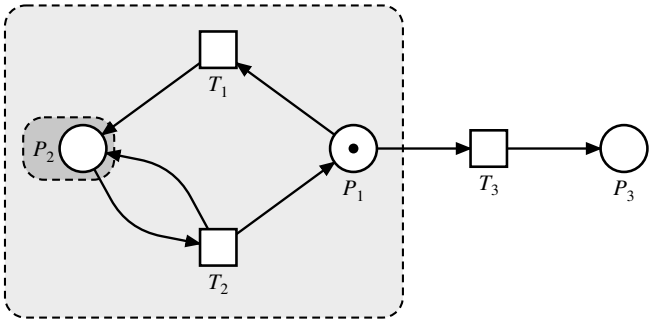
Figure 10: The places $\{P_1, P_2\}$ (shaded) form a siphon: every transition that produces into them (T_1, T_2) also consumes from them. It is marked initially, but once T_3 fires it stays unmarked forever.

Siphons have a strong conceptual connection to deadlocks. Indeed, the set of unmarked places in any deadlock marking (the complement of its support) forms a siphon: every transition is dead because at least one of its input places is unmarked, and none of those unmarked places can be marked again because no transition is enabled to produce a token into them.

The Commoner/Hack Criterion. A Petri net satisfies the Commoner/Hack criterion (CHC, also known as the **siphon-trap property**) if every proper siphon contains a trap marked in the initial marking. The overlap of these properties implies that no siphon can ever become unmarked, and therefore no deadlock can ever occur (Figure 11). This characteristic yields even more powerful insights into the behavior of more restricted subclasses of Petri nets, as discussed in Section 1.5.5.3.



(a) The trap is marked



(b) The trap is unmarked

Figure 11: The siphon $\{P_1, P_2\}$ (light) contains the trap $\{P_2\}$ (dark), and no other trap. In (a) the trap is marked, so the siphon can never become empty. In (b) the trap is unmarked, and firing T_3 empties the siphon forever.

However, deciding whether the CHC holds is a co-NP-complete problem [25]. The number of minimal siphons in a net can grow exponentially with the number of places, making the naive approach of enumerating all siphons and checking for a marked trap intractable for large nets. But the problem of finding a counterexample (a siphon without a marked trap) maps directly to the Boolean satisfiability problem (SAT), which allows us to leverage the power of modern constraint solvers.

1.5.5.2. P-invariants and T-invariants

P-invariants. A P-invariant is a weighted sum of token counts of places which is constant under transition firing. Formally, a P-invariant is a vector $y \in \mathbb{N}^{|P|}$ such that $y^T C = 0$ (the weighted sum is constant). Less restrictively,
a **P-subinvariant** is a vector $y \in \mathbb{N}^{|P|}$ such that $y^T C \leq 0$ (non-increasing), and
a **P-surinvariant**[10] is a vector $y \in \mathbb{N}^{|P|}$ such that $y^T C \geq 0$ (non-decreasing).
P-invariants allow us to reason about the number of tokens in a set of places across all reachable markings, and can be used to prove (un)boundedness and

[10] Also called a **superinvariant**.

conservation properties of the net, or regions of it. Figure 12 shows an example of each kind.

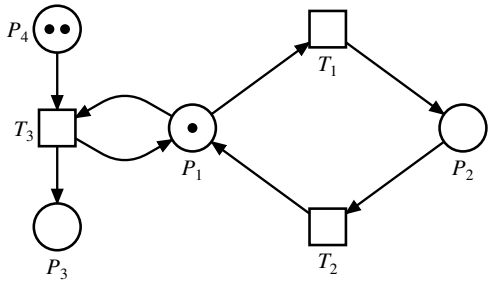


Figure 12: A net with places ordered $P_1, \ldots, P_4$. $y = (1, 1, 0, 0)$ is a P-invariant: $1 * M(P_1) + 1 * M(P_2) = 1$ in every reachable marking.
$y = (0, 0, 0, 1)$ is a P-subinvariant, because T_3 removes tokens from P_4 and no transition adds any, and $y = (0, 0, 1, 0)$ is a P-surinvariant, because T_3 adds tokens to P_3 and no transition removes any.

T-invariants. A T-invariant is a weighted sum of transition firings which has zero net effect on the marking of the net.
Formally, it is a vector $x \in \mathbb{N}^{|T|}$ such that $Cx = 0$ (the net token change is zero).
T-invariants allow us to reason about liveness properties of nets. Figure 13 shows an example.

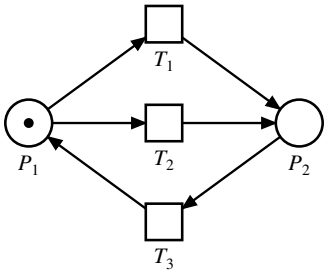


Figure 13: A net with the T-invariants $x = (1, 0, 1)$ and $x = (0, 1, 1)$ over T_1, T_2, T_3: firing T_1 and then T_3, or T_2 and then T_3, returns the net to the marking it started from.

1.5.5.3. Structural Subclasses

The structure of a net can be classified into various subclasses, which are defined by restrictions on the flow relation. These subclasses are important because they allow for more efficient analysis of many behavioral properties.

Some notable net subclasses include, in order of increasing generality (Figure 23):

Circuits. A net $N = (P, T, F)$ is called a **circuit** if $\forall p \in P : |\bullet p| = |p \bullet| = 1$ and $\forall t \in T : |\bullet t| = |t \bullet| = 1$.
Intuitively: the entire net forms a single directed cycle (Figure 14). The behavioral properties of circuits are trivial. They inherit the properties of state machines and marked graphs, discussed next.

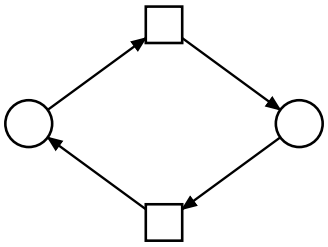


Figure 14: A circuit with two places and two transitions.

State Machines. A net $N = (P, T, F)$ is called a **state machine** if $\forall t \in T : |\bullet t| = |t \bullet| = 1$.
Intuitively: every transition has exactly one input and one output place (Figure 15).

The behavioral properties of state machines are also trivial. Most notably, the number of tokens in a state machine is invariant under transition firing: $M(P) = M_0(P)$ for every reachable marking M. This is a special case of boundedness called **conservation**. Additionally, reachability, coverability, liveness, and deadlock-freedom all have linear-time algorithms in state machines [4]. Figure 16 shows a state machine which is not a circuit.

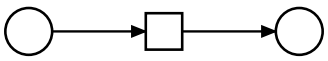


Figure 15: The building block of state machines: a transition with one input and one output place.

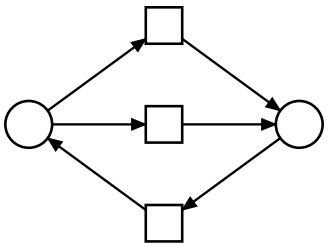


Figure 16: A state machine which is not a circuit: the left place has two output transitions and the right place has two input transitions.

Marked Graphs. A net $N = (P, T, F)$ is called a **marked graph** if
$\forall p \in P : |\bullet p| = |p \bullet| = 1$.
Intuitively: every place has exactly one input and one output transition (Figure 17).

Marked graphs are the structural dual of state machines; however, they are not quite as trivial to analyze. The fundamental property of marked graphs is that the sum of tokens in a contained *circuit* is invariant under transition firings, where a circuit is a directed cycle in the net. An unmarked circuit can never become marked, making the transitions in that circuit dead (L_0-live). A place in a live marked graph is bounded if and only if it is contained in a circuit. Boundedness, liveness and deadlock-freedom can be decided in linear time in marked graphs. *k*-boundedness (finding the exact bound), reachability and coverability are also decidable in polynomial time, though the algorithms are more complex [4].
Figure 18 shows a marked graph which is not a circuit.

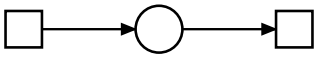


Figure 17: The building block of marked graphs: a place with one input and one output transition.

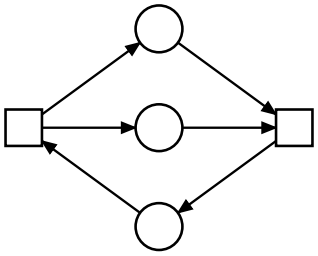


Figure 18: A marked graph which is not a circuit: the left transition has two output places and the right transition has two input places.

(Extended) Free-Choice Nets. A net $N = (P, T, F)$ is called **free-choice** if
$\forall t_1, t_2 \in T : t_1 \neq t_2 \wedge \bullet t_1 \cap \bullet t_2 \neq \emptyset \Rightarrow |\bullet t_1| = |\bullet t_2| = 1$.
Intuitively: if two transitions share an input place, then that is their only input place (Figure 19).

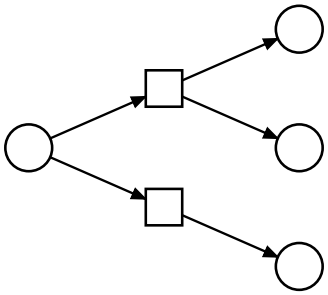


Figure 19: A free-choice net which is neither a state machine nor a marked graph: the left place has two output transitions, and the upper transition has two output places.

A net $N = (P, T, F)$ is called **extended free-choice** if
$\forall t_1, t_2 \in T : \bullet t_1 \cap \bullet t_2 \neq \emptyset \Rightarrow \bullet t_1 = \bullet t_2$.
Intuitively: if two transitions share an input place, then they share all input places (Figure 20).

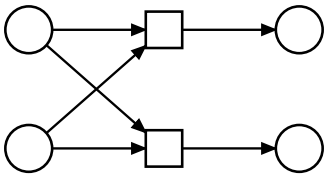


Figure 20: An extended free-choice net which is not an ordinary free-choice net: the two transitions share both of their input places.

A fundamental consequence of these properties is that if a set of transitions shares any input places (a so-called “conflict”), then no marking can enable one transition without also enabling the others.

This has powerful analytical implications. For instance, the Commoner/Hack criterion is both necessary and sufficient for liveness in free-choice nets [26], and deciding whether a free-choice net is simultaneously live and bounded (properties which are often desirable together) can be done in polynomial time [27], [28].

For purposes of analysis, free-choice nets and extended free-choice nets are equivalent. Any extended free-choice net can be transformed into a behaviorally

equivalent free-choice net through a polynomial-time transformation [5]. They are listed separately for historical reasons, as the free-choice property was first defined by Michel Hack (1972) [26], and the extended free-choice generalization was only introduced later [29], [30], [31].

In modern Petri net literature, the term “free-choice” now almost universally refers to the extended free-choice property, and the original free-choice property is now often called “ordinary” or “simple” free-choice.

Asymmetric-Choice Nets. A net $N = (P, T, F)$ is an asymmetric-choice net if $\forall p_1, p_2 \in P : p_1 \bullet \cap p_2 \bullet \neq \emptyset \Rightarrow p_1 \bullet \subseteq p_2 \bullet \vee p_2 \bullet \subseteq p_1 \bullet$.
Intuitively: if two places share an output transition, then the output transitions of one place are a subset of the output transitions of the other place.

The asymmetric-choice property is only a marginal restriction on general nets, and is not as analytically useful as the free-choice property. However, the Commoner/Hack criterion is still sufficient (but not necessary) to show liveness in asymmetric-choice nets [4]. Figure 21 shows an asymmetric-choice net which is not an extended free-choice net.

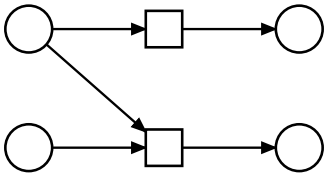


Figure 21: An asymmetric-choice net which is not an extended free-choice net: the input places of the upper transition are a proper subset of those of the lower transition.

Nets without any of these restrictions are called general nets, or just nets (Figure 22).

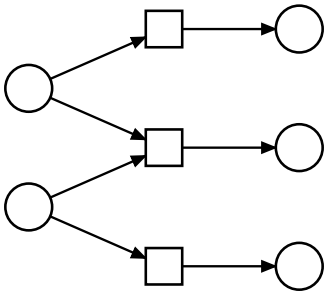


Figure 22: A general net which is not an asymmetric-choice net: the two places share the middle output transition, but neither postset contains the other.

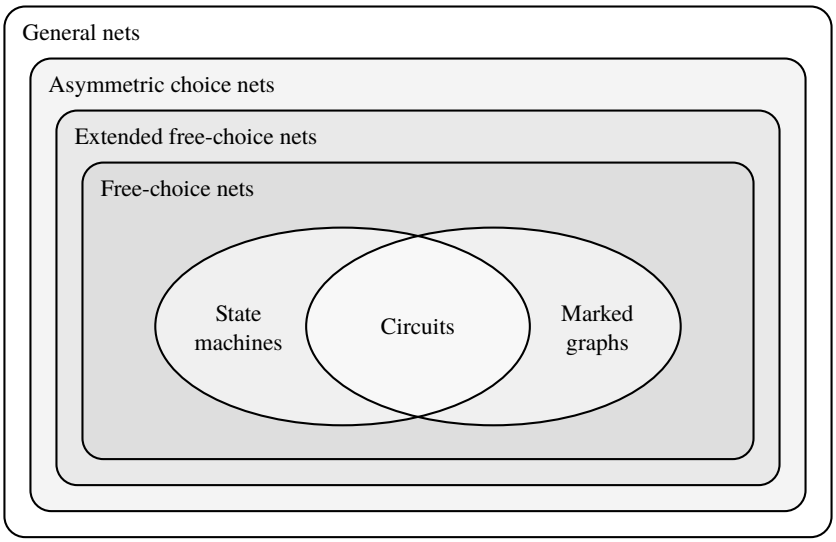


Figure 23: The structural subclasses of nets within the scope of this thesis.

1.6. Evaluation

The evaluation consists of a measured component and an assessed component. The measured component examines the correctness of the answers (RQ2), the benefit of dispatching to the cheapest procedure (RQ3), the ability to control the cost by restricting effort or setting a deadline (RQ4), and the speed and memory use of `petrivet` in comparison with established tools (RQ5). The assessed part rates `petrivet` against the requirements of the chair that cannot be reduced to a measurement, using the same rubric as the survey of existing tools. The goal is *not* to best existing model checkers, which have had sometimes decades to mature; this project is a starting point for a new library in the Rust ecosystem, and in its current state it has no chance of outperforming existing model checkers in several areas. The comparison is meant to place `petrivet` relative to the state of the art. I will continue to improve the library over time, adding new procedures and optimizations. The evaluation is described in detail in Section 5.

1.7. Structure

Section 2 surveys existing Petri net analyzers and identifies the gap that this thesis aims to address. Section 3 describes the design goals and the functional and non-functional requirements for `petrivet`, and Section 4 describes the details of the Rust implementation. Section 5 evaluates the correctness of the library, the effect of its central design decisions, and its standing relative to established tools, and assesses it against the requirements. Section 6 discusses the design principles that emerged and the limitations of the work, and Section 7 concludes.

2. Related Work

This chapter answers RQ1: to what extent do existing Petri net analysis tools meet the requirements R1 to R8 derived in Section 1.1?

These are:
- programmatic access (R1),
- memory-safe portability (R2),
- cost control (R3),
- model interchange (R4),
- performance and memory efficiency (R5),
- evidence with answers (R6),
- extensibility (R7), and
- approachability (R8).

Correctness is a precondition for any tool, but it is not evaluated here. See Section 5.2 for a correctness evaluation of `petrivet` compared to a small set of established tools.

I first describe the methodology used to select the tools to survey, and then present the 37 selected tools with regard to their support for the requirements.

Beyond the ratings, I look at one more feature that matters for RQ3: whether a tool documents that it chooses its procedure by the structure of the net.

Finally, I state the conclusions drawn from the survey and state the gap that `petrivet` aims to fill.

2.1. Methodology

I followed a structured literature review protocol [32] to identify relevant Petri net analysis tools. The protocol was fixed before the survey was conducted.

I searched four sources for academic papers:
- Google Scholar[11],
- DBLP[12],
- the ACM Digital Library[13], and
- SpringerLink[14]

three sources for software packages:
- crates.io[15],
- PyPI[16], and

[11] https://scholar.google.com, last accessed: 2026-10-27
[12] https://dblp.org, last accessed: 2026-10-14
[13] https://dl.acm.org, last accessed: 2026-10-14
[14] https://link.springer.com, last accessed: 2026-10-14
[15] https://crates.io, last accessed: 2026-10-14

- GitHub[17]

and three sources for curated lists of Petri net tools:
- the Hamburg Petri Nets Tool Database[18], and
- the Model Checking Contest (MCC) participant lists for 2025[19] and 2026[20] [21]

In addition, the tools already known through prior experience were included in the survey. A set of search queries was defined for each source. For relevance-ranked sources (Google Scholar, ACM DL, SpringerLink, PyPI), the first 50 results per query were screened; all other sources were screened in their entirety.

A record entered the review in two stages.

At stage 1, search results were roughly filtered for relevance based on title, description/abstract, and metadata. At stage 2, the remaining records were filtered for relevance by reading the documentation and/or source code of the tool.

A tool was included if:
- its documentation mentions a decision procedure for place/transition nets (or nets which can be interpreted as such) for at least one of boundedness, deadlock-freedom, liveness, reachability, or coverability (Inclusion Criterion 1, IC1);
- it is at least partially documented and available for download (IC2); and
- its last release or commit is dated within ten years of the survey, 2016-10-14 or later (IC3).

Pure editors and simulators without analysis capabilities were excluded (Exclusion Criterion 1, EC1), as well as tools which only support colored, timed, stochastic, or other non-ordinary Petri nets (EC2).

IC3 was widened from 5 to 10 years after the survey had begun, triggered by LoLA [6], a prominent tool which was last released in 2020. Previous exclusions based on IC3 were re-evaluated, and some tools were added back to the survey. This and other protocol amendments are documented in Section 8.1.

The 785 records collected from the ten sources were filtered in stage 1 with the help of Claude Sonnet 5.5, and a random sample of 20% (157) of the records was

[16] https://pypi.org, last accessed: 2026-10-14

[17] https://github.com, last accessed: 2026-10-14

[18] https://www2.informatik.uni-hamburg.de/tgi/PetriNets/tools/db.html, last accessed: 2026-10-14

[19] https://mcc.lip6.fr/2025/results.php, last accessed: 2026-10-14

[20] https://mcc.lip6.fr/2026/results.php, last accessed: 2026-10-14

[21] An early version of `petrivet` participated in the 2026 MCC (revision `c5af7627d5e016c4c7294a62db7591379e875e0a`, committed 25.05.2026). It placed last in every examination it entered, but the version predated the majority of `petrivet`'s current procedures and is not representative of the tool evaluated in this thesis.

filtered by hand to verify the quality of the filtering. Agreement between the two filtering methods was 94.3%, and Cohen's kappa was a moderate 0.50, which is explained by the low pass rate (most results did not contain a relevant tool). No disagreement was responsible for adding or removing a tool from stage 2. Forty-six records passed stage 1 and were filtered by hand in stage 2, resulting in 37 tools which finally entered the survey.

2.2. Survey of Tools

The 37 tools fall into five categories based on their primary purpose:

Performance-oriented model checkers (9) are built with linear temporal logic (LTL) and computation tree logic (CTL) properties in mind:

- TAPAAL/VerifyPN[22] [33],
- TINA[23] [34], [35],
- ITS-Tools[24] [36],
- LoLA[25] [6],
- Marcie[26],
- GreatSPN[27],
- LTSmin[28],
- pnmc[29], and
- mcc4mcc[30], which is a meta-tool that dispatches queries to other tools.

They scale through reductions [37], decision diagrams [38] and partial-order methods [39]. Most are written in C or C++ and accept queries in the form of temporal-logic formulae.

Constraint-based checkers (6) formulate Petri net properties in a constraint language and dispatch to a SAT or satisfiability modulo theories (SMT) solver:

- SMPT[31],
- Petrinizer[32] [40],
- QCover[33],

[22] https://www.tapaal.net/, last accessed: 2026-10-27
[23] https://projects.laas.fr/tina/, last accessed: 2026-10-25
[24] https://lip6.github.io/ITSTools-web/, last accessed: 2026-10-27
[25] https://theo.informatik.uni-rostock.de/theo-forschung/tools/lola/, last accessed: 2026-10-27
[26] https://www-dssz.informatik.tu-cottbus.de/DSSZ/Software/Marcie, last accessed: 2026-10-14
[27] https://github.com/greatspn/SOURCES, last accessed: 2026-10-14
[28] https://github.com/utwente-fmt/ltsmin, last accessed: 2026-10-14
[29] https://github.com/ahamez/pnmc, last accessed: 2026-10-14
[30] https://github.com/cui-unige/mcc4mcc, last accessed: 2026-10-14
[31] https://github.com/nicolasAmat/SMPT, last accessed: 2026-10-14
[32] https://github.com/cryptica/pnerf, last accessed: 2026-10-14
[33] https://github.com/michaelblondin/qcover, last accessed: 2026-10-14

- Inequalizer[34],
- mist[35], and
- bounded-model-checker-for-petri-nets[36].

Their results tend to include independently verifiable certificates as a by-product of the constraint solving process.

Workflow and process-mining tools (8) are designed to analyze soundness of workflow nets[37]:
- WoPeD[38] [42],
- ProM with Woflan[39],
- PM4Py[40],
- Wolfgang[41],
- jBPT[42],
- YAPNE[43],
- wasm4pm[44], and
- bpmn2pnml-ltl-analyzer[45].

They answer a single composite query: “Is this workflow net sound[46]?” and can point out particular transitions or places which violate soundness.

Libraries and small tools in language ecosystems (7) offer analysis as a function call from Python, Rust or Go:
- SNAKES[47],
- petritub[48],
- go-pflow[49],

---

[34] https://github.com/florianfurbach/Inequalizer, last accessed: 2026-10-14

[35] https://github.com/pierreganty/mist, last accessed: 2026-10-14

[36] https://github.com/tatsuhirotsuchiya/bounded-model-checker-for-petri-nets, last accessed: 2026-10-14

[37] A workflow net is a Petri net exhibiting a single source place (with an empty preset) which can reach every other node, and a single sink place (with an empty postset) which every other node can reach [41]. They are commonly used to model and analyze business processes.

[38] https://github.com/woped/WoPeD, last accessed: 2026-10-14

[39] https://github.com/promworkbench, last accessed: 2026-10-14

[40] https://github.com/process-intelligence-solutions/pm4py, last accessed: 2026-10-14

[41] https://github.com/iig-uni-freiburg/WOLFGANG, last accessed: 2026-10-27

[42] https://github.com/jbpt/codebase, last accessed: 2026-10-14

[43] https://github.com/chimenkamp/YAPNE-Yet-Another-Petri-Net-Editor, last accessed: 2026-10-14

[44] https://github.com/seanchatmangpt/wasm4pm, last accessed: 2026-10-14

[45] https://github.com/Research-MCI/BPMN2PNML-LTL-Analyzer, last accessed: 2026-10-14

[46] Soundness is a composite property of workflow nets requiring that (1) there is exactly one reachable deadlock (final marking) which puts one token in the end place and no tokens elsewhere, (2) the final marking is reachable from every reachable marking, and (3) every transition is at least L_1-live in the initial marking.

[47] https://github.com/fpom/snakes, last accessed: 2026-10-14

- isene/petri[50],
- visualpetri-mcp[51],
- libpetri[52], and
- petricheck[53].

They are generally limited in scope (capped state space, only one property, etc.) and their documentation is overall sparse.

Teaching, desktop and web analyzers (7) place a graphical user interface in front of classical algorithms (mostly Karp-Miller coverability tree [20]):
- APT[54],
- Charlie[55] [43],
- the MATLAB Petri Net Toolbox[56],
- GPenSIM[57],
- Hippo-CPS[58],
- PnAT[59], and
- petri-nets-tool-reactjs[60].

2.3. Findings

I assigned each tool a score on a four-point scale for each requirement based on my reading of the documentation and source code: "✓" = fully meets the requirement, "(✓)" = partly meets the requirement, "✕" = does not meet the requirement, and "?" = not determined. Table 1 lists the scores of all 37 tools. These scores are a subjective, qualitative assessment of the tools' capabilities and can be disputed. R6 is rated by what a tool returns as evidence for an answer: "✓" if it can return a witness (for example a firing sequence or a counterexample) or an independently verifiable certificate, "(✓)" if it can only name the net elements responsible for a negative result, and "✕" if it offers neither (naming the technique is recorded in the table but does not count). In the column "Evidence

[48] https://control.gitlab-pages.tu-berlin.de/discrete-event-systems/bsc-thesis-code/petri-tub-max-plus-automata, last accessed: 2026-10-14
[49] https://github.com/pflow-xyz/go-pflow, last accessed: 2026-10-14
[50] https://github.com/isene/petri, last accessed: 2026-10-14
[51] https://github.com/everlnggg/visualpetri-mcp, last accessed: 2026-10-14
[52] https://github.com/debe/libpetri, last accessed: 2026-10-14
[53] https://github.com/erwanM974/petricheck, last accessed: 2026-10-14
[54] https://github.com/CvO-Theory/apt, last accessed: 2026-10-14
[55] https://www-dssz.informatik.tu-cottbus.de/DSSZ/Software/Charlie, last accessed: 2026-10-14
[56] https://www.pntool.ac.tuiasi.ro/, last accessed: 2026-10-14
[57] https://www.davidrajuh.net/gpensim, last accessed: 2026-10-14
[58] https://hippo.uz.zgora.pl, last accessed: 2026-10-14
[59] https://github.com/dbera/Petri-net-Analysis-Tools, last accessed: 2026-10-14
[60] https://github.com/albertovalerio/petri-nets-tool-reactjs, last accessed: 2026-10-14

offered”, “method” means that the tool names the technique it used, “witness” and “certificate” are as above, “culprits” means that the tool names the net elements responsible for a negative result, and “theorem” means that the tool can name the theoretical foundation of its answer (for example a theorem from the literature). The column “Dispatch by net class” records whether the documentation says that the tool selects its procedure by the structure of the net; it is not a rated requirement and is discussed separately below.

#	Tool	Properties decided	Dispatch by net class	R1	R2	R3	R4	R5	R6	Evidence offered	R7	R8	Last release or commit
Performance-oriented model checkers													
1	TAPAAL/ VerifyPN	R, CTL, LTL (query language)	by query type	(✓)	×	(✓)	(✓)	✓	✓	method, witness	(✓)	✓	2026-06
2	TINA	B, D, L, R, C (graphs, model checker)	by net features (time, priorities)	✓	✓	✓	(✓)	✓	✓	method, witness, culprits	×	×	2026-04
3	ITS-Tools	R, D, B (MCC examinations)	not documented	(✓)	×	✓	✓	✓	✓	method, witness	(✓)	(✓)	2026-09
5	LoLA	B, D, L, R, C (graph)	per property only	✓	×	✓	×	✓	✓	method, witness, theorem	(✓)	×	2020-07
6	Marcie	D, L, R, CTL; B only as degree	not documented	(✓)	×	(✓)	(✓)	✓	✓	method, witness, culprits	×	×	2017-06
7	GreatSPN	B, D, L, CTL (P/T part of a stochastic suite)	not documented	(✓)	×	×	(✓)	✓	(✓)	method, culprits	(✓)	✓	2026-02
23	mcc4mcc	delegates to MCC tools (all examinations)	learned from net characteristics	(✓)	(✓)	×	×	✓	×	not documented	(✓)	(✓)	2018-10
24	LTSmin	R, D (L only as formula)	not documented	(✓)	×	(✓)	(✓)	✓	✓	witness, method	✓	✓	2024-10
25	pnmc	D, dead transitions	not documented	✓	×	✓	(✓)	✓	✓	witness, method	(✓)	(✓)	2026-04 (bot)
Constraint-based checkers (SMT, SAT)													
4	SMPT	R, D, quasi-L, C	not documented	(✓)	(✓)	✓	(✓)	✓	✓	method, witness, certificate	(✓)	(✓)	2025-06
10	Petrinizer	B, D, R (unreachability proof)	no	(✓)	(✓)	(✓)	(✓)	(✓)	✓	method, certificate, culprits	(✓)	(✓)	2020-11
11	QCover	C	not documented	(✓)	(✓)	×	×	(✓)	×	method	(✓)	(✓)	2019-08
12	Inequalizer	R, C (disproof only)	not documented	(✓)	(✓)	(✓)	×	(✓)	✓	method, certificate, theorem	(✓)	(✓)	2020-05
31	bounded-model-checker-for-petri-nets	D on 1-safe nets only	not documented	(✓)	×	(✓)	×	×	×	method	(✓)	×	2018-12
36	mist	C	user picks the algorithm	(✓)	×	✓	×	(✓)	✓	method, witness	(✓)	(✓)	2026-09 (bot)
Workflow and process-mining tools (soundness)													

#	Tool	Properties decided	Dispatch by net class	R1	R2	R3	R4	R5	R6	Evidence offered	R7	R8	Last release or commit
15	PM4Py	workflow soundness (B, D, L)	in code, not documented	✓	(✓)	(✓)	✓	×	✓	method, witness, culprits	✓	✓	2026-10
16	ProM (Woflan)	B, L, C and R graphs, soundness	no	(✓)	✓	×	(✓)	×	✓	method, theorem, witness, culprits	×	×	2025-09
17	WoPeD	soundness (B, D, L), C and R graphs	no	×	✓	(✓)	✓	×	(✓)	method, culprits	(✓)	✓	2026-05
18	Wolfgang	B (in source)	not documented	×	✓	×	✓	×	×	not documented	(✓)	×	2017-12
21	YAPNE	data-aware soundness (D, dead transitions)	not documented	(✓)	✓	×	✓	×	✓	method, witness, culprits	(✓)	✓	2026-10
22	bpmn2pnml-ltl-analyzer	D, livelock via LTL	not documented	✓	✓	×	(✓)	×	×	not documented	✓	(✓)	2026-07
26	jBPT	B (own); R, L, soundness via bundled LoLA	structure detected only	✓	(✓)	✓	✓	×	(✓)	culprits, method	✓	✓	2026-08
35	wasm4pm	workflow soundness, safeness	detection only	✓	✓	(✓)	✓	×	(✓)	theorem, culprits	(✓)	(✓)	2026-10
Libraries and small tools in language ecosystems													
13	SNAKES	R (finite state graph)	not documented	✓	✓	×	(✓)	×	×	method	✓	(✓)	2024-06
14	petritub	B, C, dead transitions	not documented	✓	✓	×	×	×	×	method	×	×	2026-04
27	go-pflow	D, B, quasi-L, R	structural first, then exhaustive	✓	✓	(✓)	(✓)	×	✓	method, witness, theorem, certificate, culprits	✓	(✓)	2026-10
30	isene/petri	B, D (capped search)	not documented	(✓)	✓	(✓)	×	×	(✓)	culprits	(✓)	(✓)	2026-10
32	visualpetri-mcp	R, D (bounded graph)	not documented	✓	✓	✓	×	×	✓	witness, culprits	(✓)	(✓)	2026-10
33	libpetri	D, place bound, unreachability, mutual exclusion	by net features	✓	(✓)	✓	×	×	✓	method, witness, certificate, theorem	✓	(✓)	2026-10
34	petricheck	k-safeness; R, D, L via user CTL	no	✓	✓	(✓)	(✓)	×	(✓)	culprits	✓	(✓)	2026-02
Teaching, desktop and web analyzers													
8	APT	B, C, weak L	not documented	✓	✓	✓	(✓)	×	✓	method, witness, culprits	✓	(✓)	2026-02
9	Charlie	B, L, C, dead states, R via CTL/LTL	class detected, not a switch	(✓)	✓	(✓)	×	×	✓	method, theorem, witness, culprits	×	×	2017-06
19	Petri Net Toolbox	B, L, D, C (omega tree)	not documented	(✓)	(✓)	×	×	×	(✓)	method, culprits	×	×	2025-09

#	Tool	Properties decided	Dispatch by net class	R1	R2	R3	R4	R5	R6	Evidence offered	R7	R8	Last release or commit
20	Hippo-CPS	B, L	user chooses method	×	?	(✓)	?	(✓)	(✓)	method, culprits	×	×	2023-06
28	petri-nets-tool-reactjs	R, B, L, invariants, siphons, traps	not documented	×	✓	(✓)	×	×	×	none	(✓)	(✓)	2023-02
29	PnAT	B, weak termination, k-bound	not documented	×	✓	(✓)	(✓)	×	✓	witness	(✓)	×	2023-12
37	GPenSIM	B, D, C, R (capped)	user picks cotree vs cotreei	✓	(✓)	(✓)	✓	×	(✓)	method, culprits	×	×	2025-08

Table 1: Table of the 37 tools surveyed, their properties, and the scores for each requirement. Properties: B = boundedness, D = deadlock-freedom, L = liveness, R = reachability, C = coverability.

Figure 24 shows, for each requirement, how many tools meet it fully, partly or not.

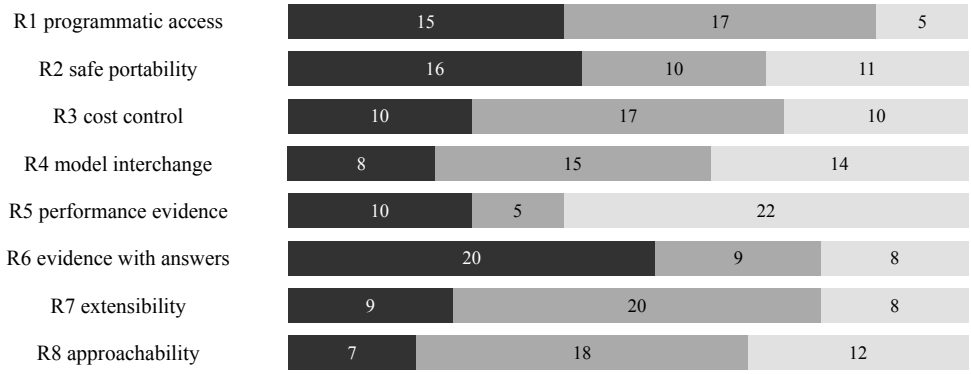


Figure 24: Number of the 37 tools that meet each requirement fully (dark), partly (medium), or not/not determined (light). The ratings for R6 follow the evidence rule given above.

R1, programmatic access. Fifteen tools return structured data from a function call or command-line interface, and seventeen tools can only be scripted by parsing textual output or reading exit codes. Five tools provide no interface other than a GUI. Among the performance-oriented model checkers, only TINA, LoLA and pnmc document a JSON output format; the others print text lines (ITS-Tools) or XML traces and exit codes (VerifyPN).

R2, safe portability. Seventeen tools are written in a memory-safe language and do not depend on native code. Seven of the nine performance-oriented checkers have a native C or C++ core and so sacrifice memory safety for performance (this is not a judgment of the quality of their implementation). TINA is implemented in

Standard ML, which is memory-safe, and mcc4mcc containerizes other (not memory-safe) tools within Docker. Four of the six constraint-based checkers depend on the Z3 SMT solver, which is written in C++ and has a history of memory-safety issues [44], [45]. Of the libraries scored for R2, the Rust, Go, and Python libraries are at least memory-safe in principle, but I did not audit their use of `unsafe` or of native dependencies.

R3, cost control. Ten tools allow the user to bound a query with a timeout, memory limit, state-space cap, or choice of procedure. Seventeen offer partial control (a fixed cap, timeout, or depth bound). Ten tools offer no cost-control mechanism at all.

R4, model interchange. Eight tools read and write Petri Net Markup Language[61] (PNML), the standardized XML interchange format for Petri net models. Fifteen tools use only proprietary formats, and partially meet the requirement. LoLA documents that it deliberately uses a proprietary format instead of PNML. Thirteen tools only allow manual construction of nets or do not document any interchange format, and so fail the requirement.

R5, performance and memory efficiency. Ten tools have independent evidence from the Model Checking Contest; five more have a published evaluation, mostly by their authors. The remaining twenty-two tools have no performance evidence.

R6, evidence with answers. Twenty tools can return a witness or counterexample (eighteen tools) or an independently verifiable certificate (five tools, three of which also return a witness). Nine more can only name the net elements responsible for a negative result, and eight offer no evidence beyond the answer.

R7, extensibility. Nine tools (APT, SNAKES, PM4Py, bpmn2pnml-ltl-analyzer, LTSmin, jBPT, go-pflow, libpetri and petricheck) publish their source under an OSI-approved license and are either usable as a library or document an extension mechanism. Twenty tools publish source but document neither a library interface nor an extension point, or lack a clear open license. Among them, Petrinizer, Inequalizer, the bounded model checker and visualpetri-mcp state no license at all, which gives no right to reuse or modify the code, and wasm4pm combines an Apache, an MIT and a commercial license. Eight tools, including TINA, Charlie, Marcie and the Petri Net Toolbox, offer no source that I could find.

R8, approachability. This rating covers only the contributor side of the requirement, through observable repository indicators; it does not measure how easy a tool is to use, for people or for AI agents, and it is the softest rating of the survey. I counted five indicators for each tool: a public repository with an issue

[61] https://www.pnml.org/

tracker, a documented build from a clean checkout with a standard toolchain, automated tests or continuous integration, contribution guidelines or developer documentation, and commits by at least two distinct human authors between 2024-10-14 and 2026-10-14 (bots and AI accounts excluded). Seven tools (TAPAAL/VerifyPN, GreatSPN, PM4Py, WoPeD, YAPNE, LTSmin and jBPT) meet at least four, including the first and the last. Eighteen meet two or three, or lack one of the first and the last. Twelve meet at most one indicator or have no public repository. Only PM4Py, LTSmin and jBPT are rated as met on both R7 and R8, and each of them fails another requirement (R5 for PM4Py and jBPT, R2 for LTSmin).

Across the requirements, no tool fully meets all of R1 to R8. The most requirements met in full is five of the eight (TINA, PM4Py, jBPT and APT), followed by four (ITS-Tools, LoLA, LTSmin, pnmc, YAPNE, go-pflow, visualpetri-mcp and pnmclibpetri). Only SMPT and Petrinizer fail none of the eight requirements outright, and both meet only some of them partly.

2.4. Dispatching by Net Structure

The answer to RQ3 depends on whether tools choose a decision procedure by the structure of the net, so I looked at what the documentation says about this. Most tools do not say anything: for twenty-one tools I found no documentation of any selection, and for four more the documentation states explicitly that there is none. Some tools select by something other than the net structure: TAPAAL chooses by query type, LoLA applies a procedure per property, and the contest meta-tool mcc4mcc chooses among other tools using a learned model of net characteristics. In a few tools the user makes the choice (mist, Hippo-CPS, GPenSIM). Only a handful of tools document anything like a selection by structure: TINA by net features (time and priorities), go-pflow by structural checks before exhaustive search, and libpetri by net features; jBPT, Charlie and wasm4pm detect the structure of the net but do not use it to switch procedures. No tool documents a dispatch that spans the net classes (for example free-choice, marked graph and state machine) and cost-bounded fallbacks. These counts depend on the verification noted with the table. It is a gap in documentation as much as in implementation, but a user can only rely on what is documented.

2.5. The Gap

The requirements pull in different directions in the surveyed tools. The tools with performance evidence (R5) mostly give up safe portability or extensibility: of the ten tools with independent contest results, only TINA is rated fully on R2, and TINA publishes no source and offers no contribution path (R7 and R8 not met), while LoLA fails R2 and R4, and LTSmin fails R2. In the other direction, the libraries that provide programmatic access, extensibility and at least partly safe

portability (SNAKES, go-pflow, libpetri and petricheck) have no performance evidence (R5 is not met by any of the seven libraries), and their cost control is full only for libpetri, partial for go-pflow and petricheck, and absent for SNAKES. Workflow tools and teaching tools return the most helpful evidence, but they answer a narrow set of questions on small nets and, except for Hippo-CPS, show no performance evidence either. Finally, almost no tool documents choosing its procedure by the structure of the net, so a user who wants the cheapest applicable procedure has to know the theory and pick the tool, or the option, themselves. This is the gap that `petrivet` aims to fill: a library that is programmatically accessible, memory-safe[62], and extensible, that decides each question with the cheapest procedure applicable to the net, that lets the caller bound the cost, and that is measured against the established tools in Section 5.

2.6. Threats to Validity

- The sample is selected by search and not exhaustive; tools without English or German documentation or without a public page are not included.
- R7 and R8 were added after the survey had begun, as an amendment to the protocol. The rating rules were fixed before any data for them was collected. They are coarse, observable proxies: the license, the presence of a library interface or documented extension point, and five repository indicators. The facts were collected with the help of Claude Sonnet 5.5 from the tools' public pages and repositories, at most a few minutes per tool, and not checked by a second rater. For five tools (Marcie, Charlie, petritub, ProM/Woflan and Wolfgang) the first pass could not reach or identify the source; their ratings rest on a later manual look or default to "not met". Commits by AI accounts are not counted as a second author, which lowers the R8 rating of tools maintained by one human with an AI assistant.
- Correctness of the surveyed tools is not rated and is assumed. For the contest participants it is supported by their participation in the Model Checking Contest, whose results are cross-checked [46] but for the other tools no evidence was collected.
- R6 and R8 were defined more narrowly than in the first version of the protocol: R6 started as "traceability to theory" and was reduced to evidence for answers, and R8 is rated only through contributor indicators. The ratings in the table follow the narrower definitions.
- Tools I already knew from prior experience were added by hand, so the selection is biased towards tools familiar to me and, through them, to the model checking community.

[62] When compiled with a pure-Rust SMT backend like oxiz (https://github.com/cool-japan/oxiz), `petrivet` is memory-safe. By default it links to the Z3 SMT solver, which is written in C++.

- I later apply the same rubric to `petrivet` myself (Section 5.6), so that rating is not independent of the one given to the surveyed tools, and no second rater checked either.
- The scoring is subjective and unverified; I did not ask the authors of the tools to confirm my assessment.
- “Not documented” means that I could not find documentation, not that the tool does not have it.
- Maintenance is judged by the date of the last release or commit, which may not reflect the actual state of the project.

3. Solution Design

This chapter discusses the high-level design of a Petri net analysis tool which aims to meet the requirements derived in Section 1.1.1.

3.1. Design Goals

The following is a list of informal guiding principles for the design of `petrivet`:

Avoid unnecessary computation and reuse previously computed results. Petri net procedures are generally extremely heavy, and so the tool should not waste resources on unnecessary computations. If the user did not ask for it, the tool should not compute it. For the same reason, the tool should be able to reuse previously computed results whenever possible, or provide a mechanism for the user to do so. There should be no situation where the user's only option is to recompute a result that has already been computed because of the inflexibility of the tool.

Decoupled API and implementation. The API of the tool and the underlying implementation should be able to evolve independently of each other, allowing for future extensibility and maintainability.

Closeness to the theory. The tool should be strongly rooted in the theoretical foundations of Petri nets, and its design should be guided by the study of the theory. Idiomatic formalisms and notations should be used whenever possible, and the tool should not introduce unnecessary abstractions or concepts that are not present in the theory. The tool should serve not only as a practical implementation of the theory, but also as a didactic tool for learning and understanding it.

Usable for both experts and non-experts. The user of the library should not need to have a deep understanding of the underlying theory in order to use the tool effectively. The tool should pick sensible defaults and provide a simple interface for common use cases, while still allowing experts to access the full range of functionality. At the same time, it should do so in a way which is transparent to the user, so that they can understand what is happening and why.

3.1.1. API Design

The API a Petri net analysis tool should aim to mirror Petri net theory as closely as possible, and so its design should be grounded in the study of the theory. The five behavioral properties discussed in this thesis do not correspond to just one question each. I identified two dimensions along which the behavioral properties can vary.

Scope. All properties can be checked for the entire Petri net. Some (boundedness and liveness) can also be checked for individual nodes (places and transitions,

respectively). The user of the library may be interested in one or the other, and so the API should provide a way to specify the scope of the property check.

Granularity. All properties can be framed in the form of a yes/no question, e.g. “Is this net bounded?” or “Is this transition live?”. But the user may also be interested in the “why” of the answer. The API should provide a way to specify the desired granularity of the result.

Furthermore, as discussed in Section 1.1.1, the user may only have a limited amount of time to wait for an answer, and so the API should provide control mechanisms for the user to prevent the analysis from taking too long. This is especially important if the user is analyzing a net of unknown structure and size. Checking a property may be trivial for some nets, but take prohibitively long (effectively forever) for others. Not being able to predict or control this would be a major inconvenience.

I propose two mechanisms aimed to address this issue, which can be used independently or in combination.

Wall-clock timeout. The user can specify a maximum duration to wait for an answer. The analysis will be interrupted if it exceeds this time limit, and the analysis will return “unknown”. This mechanism is useful when the user has a hard limit on how long they can wait for an answer, and is willing to accept that the analysis may not complete in time.

Efficiency restriction. The user can specify that only polynomial-time procedures should be used to check the property. The analysis will return “unknown” early if no polynomial-time procedure is known for checking the property for the given Petri net. This mechanism is useful when the user does not have a hard limit on how long they can wait for an answer, but just wants to avoid the risk of waiting effectively indefinitely for an answer. An early “unknown” is preferable to waiting for a time limit, as the user can immediately re-evaluate whether they want to attempt the analysis again with a different cost-control mechanism.

Beyond these considerations, it is up to the implementation to determine how best to answer the user’s query, and the implementation should be free to use any combination of procedures with the aim of providing the answer as quickly and efficiently as possible.

3.2. Functional Requirements

- **FR1.** The tool must provide a native builder API for programmatically constructing Petri net models.

- **FR2.** The tool must be able to parse PNML files into the internal Petri net representation.
- **FR3.** The tool must be able to export Petri nets from the internal representation into PNML files.
- **FR4.** The tool must distinguish between the pure structure (“net”) and a net paired with an initial marking (“Petri net”).
- **FR5.** The tool must support manual simulation of Petri net models via transition firing.
- **FR6.** The tool must support a variety of significant Petri net structural subclasses: Circuit, State Machine, Marked Graph, (Extended) Free-Choice, Asymmetric-Choice, and General nets.
- **FR7.** The tool must provide an API to analyze the fundamental properties (k-)boundedness, deadlock-freedom, liveness, coverability, and reachability.
- **FR8.** The tool must support automatic state space exploration and generation of the reachability and Karp-Miller coverability graphs.
- **FR9.** The tool must return evidence with its answers, such as witnesses and counterexamples.
- **FR10.** The tool must return “unknown” when it cannot answer a query, rather than guessing or returning an incorrect answer.
- **FR11.** The tool must provide a mechanism for the user to control the cost of analysis, either by specifying a wall-clock timeout or by restricting the analysis to polynomial-time procedures.

3.3. Non-Functional Requirements and Pseudo-Requirements

- **NFR1.** The tool must be implemented in a modern programming language with a strong type system and memory safety guarantees.
- **NFR2.** The tool must be implemented as a library, not a standalone application.

4. Implementation

This chapter discusses the implementation of `petrivet`[2].

4.1. Why Rust

`petrivet` is implemented as a Rust library (NFR2) [47], a systems programming language whose design addresses several of the requirements in Section 1.1.1 directly. Rust compiles to native code without a garbage collector or runtime, which places its performance in the same class as C and C++ (R5). Unlike those languages, however, the Rust compiler's ownership and borrowing rules guarantee freedom from use-after-free errors, data races, and other classes of memory errors before the program ever runs (R2) (NFR1) [48]. `petrivet` forbids `unsafe` code entirely (`#![forbid(unsafe_code)]`), so these guarantees apply to the entire codebase. They do not, however, extend to the C++ code of the underlying Z3 solver, which is linked by default; the pure-Rust solver `OxiZ` can be selected instead via a compile-time feature flag, but it is not yet as mature as Z3 and is not used in the evaluation. Because Rust does not need a runtime, a Rust library can be embedded almost anywhere: it can be linked into native applications through a C-compatible interface, wrapped for languages such as Python or TypeScript, compiled to WebAssembly, or shipped as a single static command-line binary (R1, R2).

Beyond safety and speed, Rust's type system helps to make the library both correct and approachable (NFR1). Algebraic data types and exhaustive pattern matching let the API state the possible outcomes of a query precisely, and force the caller to handle each of them: an answer of `None` for "unknown" cannot be confused with `Some(false)`, and evidence (such as a witness firing sequence or a set of contradicting lemmas) is carried only in the variant of the result it belongs to (R6). Newtypes for place and transition IDs (Section 4.2) prevent mixing up indices of different kinds, and traits allow procedures to be written once over a generic interface, such as the state space explorer over marking types (Figure 25) or the CEGAR engine over SMT solver backends (R7).

Finally, Rust's tooling lowers the barrier for new contributors (R8): Cargo builds, tests, benchmarks and documents the library with a single command each, and the compiler's detailed error messages catch many mistakes that would otherwise only appear at runtime. This is valuable for students and AI agents alike, who receive immediate and precise feedback when they extend the library.

4.2. NetBuilder

The builder API serves as `petrivet`'s entry point for programmatically defining the structure of a net: places, transitions, and the arcs between them (FR1).

A builder can be created with `NetBuilder::new()`, and places and transitions can be added with `add_place()` and `add_transition()`. These methods return opaque handles to the created nodes, which are wrapper types (*newtypes*) around ascending integer IDs. Rust's type system allows distinct concepts to have distinct types at no additional runtime cost, preventing accidental misuse of the API at compile time, such as passing a place where a transition is expected.

Arcs are represented as an enum: either a place-to-transition or transition-to-place arc, guaranteeing the bipartite structure of the net by refusing to compile otherwise Listing 1.

```
enum Arc {
    PlaceToTransition(Place, Transition),
    TransitionToPlace(Transition, Place),
}
```

Listing 1: The `Arc` enum guarantees the bipartite structure of the net at compile time.

To improve API ergonomics, I provide implementations of Rust's `From` trait for the tuple types `(Place, Transition)` and `(Transition, Place)` to be converted into an `Arc`, which works well with Rust's `Into` trait to allow the user to specify an arc as a pair of nodes Listing 2.

```
let mut b = NetBuilder::new(); // Create a new builder
let p = b.add_place(); // Add a place
let t = b.add_transition(); // Add a transition
b.add_arc((p, t)); // Add an arc from place `p` to transition `t`
b.add_arc((t, p)); // Add an arc from transition `t` to place `p`
```

Listing 2: The builder API allows the user to add and remove places, transitions, and arcs, and returns opaque handles to the created nodes. The `add_arc()` method accepts any type that can be converted into an `Arc`, including tuples of opposite node types, which are semantically interpreted as arcs.

Removing a node or an arc is as simple as calling the corresponding `remove_*()` method with the handle of the element to be removed. All adjacent arcs are automatically removed as well, and the removed node's internal ID is not reused to ensure that the invalidated handles do not accidentally refer to a new node that has been added later.

`petrivet`'s API also provides bulk operations for adding multiple places, transitions, or arcs at once, with the goal of reducing boilerplate code. Rust's type system can be leveraged to provide ergonomic syntax here as well.

For bulk adding nodes, *const generics* and *array destructuring* allow the user to create an arbitrary number of places or transitions at once without having to specify the desired number explicitly Listing 3.

```rust
let mut b = NetBuilder::new();
// Add three places
let [p0, p1, p2] = b.add_places();
// Add four transitions
let [t0, t1, t2, t3] = b.add_transitions();
```

Listing 3: Bulk adding of places and transitions is supported via const generics, allowing the user to add an arbitrary number of nodes at once.

For adding multiple arcs at once, the `add_arcs()` method accepts a tuple of 3-12[63] alternating places and transitions, which is semantically interpreted as a path through the net, with an arc created between each neighboring pair of nodes in the path Listing 4.

```rust
let mut b = NetBuilder::new();
let [p0, p1, p2] = b.add_places();
let [t0, t1] = b.add_transitions();
// Add three arcs using path syntax
b.add_arcs((p0, t0, p1, t1));
// This will fail to compile, as it is not a valid path
b.add_arcs((t0, t1, p2));
```

Listing 4: `petrivet`'s `add_arcs()` builder method allows adding multiple arcs at once by specifying a path through the net as a tuple of alternating places and transitions. Rust's type system allows flexibility without sacrificing correctness.

The `NetBuilder` is optimized for mutability, allowing the user to add and remove nodes and arcs in any order. Once construction is complete, the user can call the `build()` method to finalize the net and return a read-only `Net` object.

Internally, `build()` transforms the internal mutation-oriented data structures into a densely packed representation optimized for analysis.

- Every place and transition is assigned a unique index in the internal arrays, removing any gaps in the ID space which might have arisen via removals of nodes during construction. This creates a canonicalized internal layout decoupled from the API construction order, which allows for optimizations such as sorting the nodes by their degree or other properties to improve cache locality. This also allows any data associated with places or transitions to be

[63] The maximum length of 12 was chosen arbitrarily, and could be increased if desired. Rust's lack of variadic generics prevents a more general solution.

stored in a CPU cache-friendly contiguous array in the canonical order (as opposed to a hash map or other associative data structure). Sets of places or transitions can be represented as bitsets, which is especially data-dense. A mapping is maintained between the user-facing semantic labels of the nodes and their internal canonical indices, which allows the user to continue to refer to nodes by their original handles without knowing the internal representation.
- Arcs are no longer represented as separate objects, but rather as adjacency lists of node indices. This allows for cache-friendly iteration over the preset and postset of a node, which is a common operation in many analysis algorithms.
- The structural class of the net is evaluated once and cached, along with the net's incidence matrix and whether the net is strongly connected. All of these properties will be useful for analysis later, and since the net is immutable, will not need to be recomputed for the lifetime of the net.

The `build()` method additionally performs two validation checks on the net to ensure that it is a valid net:
- The net must be connected. A non-connected net, in the opinion of the author, is not a net at all, but rather a collection of nets.
- The net must contain at least one place and one transition. A single place, a single transition, or an empty net is a degenerate case with no interesting properties.

If either of these checks fails, the `build()` method returns a `NetError` enum indicating the reason for failure. Otherwise, it returns a fully constructed, read-only `Net` object that is ready to be analyzed Listing 5.

```
let mut b = NetBuilder::new();
// ... construct the net ...
let net = match b.build() {
    Ok(net) => net, // The net is valid and can be analyzed
    Err(NetError::NotConnected) => panic!("The net is not connected"),
    Err(NetError::Degenerate) => panic!("The net must contain at least one place
and one transition"),
};
// Alternatively, use the `?` operator to propagate the error
let net = b.build()?;
```

Listing 5: The `build()` method returns a `Result<Net, NetError>`, which is either a fully constructed, read-only net model that can be analyzed, or a `NetError` if the net violates one of two rules: it must be connected and contain at least one place and one transition.

The user can check the net's structural class and whether it is strongly connected via the `class()` and `is_strongly_connected()` methods, respectively (Listing 6)

```rust
let mut b = NetBuilder::new();
let [p0, p1] = b.add_places();
let t0 = b.add_transition();
b.add_arcs((p0, t0, p1));
let net = b.build()?;
assert_eq!(net.class(), NetClass::StateMachine);
assert_eq!(net.is_strongly_connected(), false);
```

Listing 6: The `class()` and `is_strongly_connected()` methods allow the user to query the net's structural class and whether it is strongly connected, respectively.

If the user wishes to modify the net after construction, they can convert a `Net` back into a `NetBuilder` and continue to add or remove nodes and arcs. Building the net again will re-evaluate the canonical internal representation and the validation checks, ensuring that the unique story of how a net was constructed does not affect its analysis properties.

```rust
// ... construct the net ...
let net = b.build()?;
// Convert the net back into a builder to modify it
let mut b2 = NetBuilder::from(net);
let t1 = b2.add_transition();
b2.add_arc((p1, t1));
let net2 = b2.build()?;
```

Listing 7: A `Net` can be converted back into a `NetBuilder` to allow further modifications. The canonical internal representation and validation checks are re-evaluated when the net is built again.

4.3. PNML Import and Export

To facilitate interoperability with other Petri net tools, `petrivet` also provides functionality for importing and exporting Petri nets in the PNML format, following the specification defined in the PNML standard[64].

Because the PNML format is XML-based, `petrivet` uses the `quick-xml`[65] crate to parse and generate XML data, which integrates with `serde`[66] to serialize and deserialize the data structures. These are two well-established crates in the Rust ecosystem, and they significantly lower the boilerplate code required to implement PNML interchange.

[64] https://www.pnml.org/
[65] https://docs.rs/quick-xml/latest/quick_xml/
[66] https://docs.rs/serde/latest/serde/

`petrivet` defines a Rust data structure `PnmlDocument` corresponding to the PNML grammar and annotates it with `serde` attributes, then simply calls `quick-xml::de::from_str()` to deserialize a PNML XML string into a `PnmlDocument` structure (FR2, import), and `quick-xml::se::to_string()` to serialize a `PnmlDocument` structure into a PNML XML string (FR3, export). In the interest of low coupling, `petrivet` does not handle any kind of file or network I/O, leaving that responsibility to the user of the library[67].

A `PnmlDocument` can then be converted (via `NetBuilder`) into a `Net` structure with relatively little code, and vice versa. Since PNML is a versatile format that can represent a wide variety of Petri net types and `petrivet` currently only supports ordinary Petri nets, the conversion process is fallible and returns an error if the PNML document specifies an unsupported Petri net type or contains otherwise invalid data.

A high-level method `Net::from_pnml()` is provided as an abstraction over these steps and returns a `Net` structure from a PNML XML string, and `Net::to_pnml()` provides the reverse operation. Corresponding methods are also provided for the `PetriNet` structure, which additionally parses the initial marking specified in the PNML document. The `PetriNet` structure will be discussed in the next section.

4.4. Petri Net Construction and Simulation

As discussed in Section 4.2, a `NetBuilder` constructs the static `Net` structure. To simulate dynamic behavior, however, the net must be paired with a marking. This section introduces the implementation of markings and Petri net simulation in `petrivet`, which is the basis for behavioral analysis in the library.

4.4.1. Tokens and Markings

Tokens, having no intrinsic properties or identity, are represented as a simple integer count of the number of tokens in each place. Currently, this integer type is `u32`, which holds a maximum of 4,294,967,295 tokens in a single place, providing a reasonable upper bound for most practical applications.

Markings have an API representation and an internal representation, each optimized for their respective use cases.

From a user-facing perspective, it is convenient to only specify the (usually sparse) places that have non-zero token counts in a marking, leaving the unmentioned places to be implicitly zero. `petrivet`'s API therefore allows the user to construct a `Marking` from any array or iterator of `(Place, u32)` pairs, where the `Place` is a handle to a place in the net and the `u32` is the number of tokens in that place.

[67] https://sans-io.readthedocs.io/how-to-sans-io.html

Listing 8 shows an example of constructing a `Marking` from an array of `(Place, u32)` pairs, and then using that marking to construct a `PetriNet` from a `Net`.

```rust
let mut b = NetBuilder::new();
let [p0, p1, p2] = b.add_places();
// ... construct the net ...
let net = b.build().expect("valid net");
let marking = Marking::from([(p0, 1), (p2, 3)]);
let petri_net = PetriNet::new(net, marking);
// Alternatively, pass the marking array inline
let petri_net = PetriNet::new(net, [(p0, 1), (p2, 3)]);
```

Listing 8: A `PetriNet` can be constructed from a `Net` and a `Marking`, which can be created from an array of `(Place, u32)` pairs. The Petri net has no tokens in `p1`, as it is not mentioned in the marking.

The `Marking` constructor filters, sorts, and deduplicates the input pairs to produce a canonical representation of the marking, ensuring that equal markings are represented identically regardless of their construction. This is important for equality comparisons and hashing of markings.

For the core logic, however, a sparse representation is inefficient, as the token count for a given place is not located at a fixed position in memory. Instead, `petrivet`'s internal representation of a marking is a dense array of token counts indexed by the canonical place index, allowing for fast access and manipulation of the marking during simulation and analysis. This index-based representation is named `IdxMarking` to distinguish it from the user-facing `Marking` type. The `Marking` passed into the `PetriNet` constructor is converted into an `IdxMarking` for internal use, while all user-facing methods of `PetriNet` accept and return the more convenient `Marking` type.

The `PetriNet` structure also offers flexibility with respect to the *ownership* of the underlying `Net` structure, a unique feature of Rust's type system. If there is only one marking to be simulated, the `PetriNet` can fully encapsulate the `Net`, forming a single unified structure that moves as one. But if the user wishes to simulate multiple initial markings on the same static net structure, without having to clone the net for each marking, he or she can also construct a `PetriNet` with a `&Net` reference, creating a lightweight secondary structure that merely points to the structure of the net without carrying its data. This is achieved by making the `PetriNet` generic over any type that implements the `AsRef<Net>` trait, allowing for both owned and borrowed[68] nets to be used. Listing 9 shows an example of constructing two `PetriNet` instances that share the same underlying `Net` structure but have different initial markings.

[68] and reference-counted, if desired.

```rust
let mut b = NetBuilder::new();
let [p0, p1, p2] = b.add_places();
// ... construct the net ...
let net = b.build().expect("valid net");
// Two Petri nets can share the same net structure
// with different initial markings.
let petri_net_a = PetriNet::new(&net, [(p0, 1)]);
let petri_net_b = PetriNet::new(&net, [(p1, 2)]);
```

Listing 9: A `PetriNet` can either own the underlying `Net` structure or borrow it, allowing the user to simulate multiple initial markings on the same static net structure without cloning the structure.

Because it is just as valid to ask a structural question about a Petri net as about the static net structure, `PetriNet` dereferences `Net` via the `Deref` trait, allowing the user to seamlessly call any of the `Net` methods directly on a `PetriNet` (Listing 10).

```rust
// ... construct the net ...
let petri_net = PetriNet::new(net, [(p0, 1)]);
// The '.' operator dereferences to the underlying Net structure
assert_eq!(petri_net.class(), NetClass::StateMachine);
assert_eq!(petri_net.is_strongly_connected(), false);
```

Listing 10: A `PetriNet` dereferences to the underlying `Net` structure, allowing the user to seamlessly call any of the `Net` methods directly on a `PetriNet`.

A `PetriNet` can also be deconstructed into the underlying `Net` and the current `Marking` using the `into_parts()` method.

4.4.2. Basic Simulation

A `PetriNet` can be used to simulate the behavior of a Petri net by firing transitions and observing the resulting markings. `PetriNet` is a stateful structure; the marking contained within it is mutable and can be modified by firing transitions.

The `is_enabled()` method can be used to check if a transition is enabled, and the `enabled_transitions()` method returns an iterator over all currently enabled transitions.

The `fire()` method receives a transition and attempts to fire it, modifying the marking in place if successful and returning an error if the transition was not enabled. If the user is already sure that a transition is enabled, the `fire_unchecked()` method can be used to fire a transition without checking its enabledness, panicking if the assumption is violated. For arbitrary firing of any

enabled transition, the `fire_any()` method can be used, which returns the transition that was fired, or `None` if the Petri net is in a deadlock state with no enabled transitions.

`PetriNet` also provides a `is_deadlocked()` helper method to check if the Petri net is in a deadlock state.

Listing 11 shows an example of simulating a Petri net by firing transitions and observing the resulting markings.

```rust
let mut b = NetBuilder::new();
let [p0, p1, p2] = b.add_places();
let [t0, t1] = b.add_transitions();
b.add_arcs((p0, t0, p1, t1, p2));
let net = b.build().expect("valid net");
let mut petri_net = PetriNet::new(net, [(p0, 1)]);
// t0 is enabled, so it can be fired
assert!(petri_net.fire(t0).is_ok());
// t0 is no longer enabled, as it has consumed the only token in p0
assert!(!petri_net.is_enabled(t0));
// t1 is now the only enabled transition
assert_eq!(petri_net.fire_any(), Some(t1));
// No enabled transitions remain
assert!(petri_net.is_deadlocked());
// The final marking has one token in p2 and no others
assert_eq!(petri_net.marking(), Marking::from([(p2, 1)]));
```

Listing 11: A `PetriNet` can be used to simulate the behavior of a Petri net by firing transitions and observing the resulting markings.

`PetriNet` also holds an immutable snapshot of the marking it was created with, so that the user can always reset the Petri net to its initial state by calling the `reset()` method, as shown in Listing 12.

```rust
petri_net.reset();
assert_eq!(petri_net.marking(), Marking::from([(p0, 1)]));
```

Listing 12: Continuation of Listing 11: a `PetriNet` can be reset to its initial marking by calling the `reset()` method.

For all upcoming behavioral property checks (Section 4.5, Section 4.7), the `PetriNet`'s current marking is used as the context for the analysis. This is the "initial marking" of the analysis, even if it is not the marking that the `PetriNet` was constructed with.

4.5. State Space Exploration

The natural successor to firing transitions manually is the automatic exploration of a Petri net's state space. The basic idea is to fire all enabled transitions in a marking, record the resulting markings as children of the current marking (via the fired transitions), and repeat this process for each newly discovered marking until no new markings can be discovered. What results is a directed graph of markings connected by transitions.

This procedure is equally applicable to both the reachability graph and the Karp-Miller coverability graph [20], with the only difference being that the coverability graph modifies the markings before recording them, in a process called ω-*acceleration*. See Section 1.5.4.2 and Section 1.5.4.3 for background on the reachability and coverability graphs.

`petrivet` contains a implementation of state space exploration that is generic over the marking type, allowing it to be shared by the two types of state space graphs. The implementation allows the user to construct the state space incrementally: a `StateGraphExplorer` structure (with concrete instantiations `ReachabilityGraphExplorer` and `CoverabilityGraphExplorer`) represents an in-progress exploration, which the user can advance cumulatively at their own pace, and stop at any point. The explorer tracks the "frontier" of the state space and provides methods to advance the exploration by one step, or to run it to completion, following either a breadth-first (FIFO) or depth-first (LIFO) strategy. It also provides methods to search for a marking fulfilling a user-specified predicate, and accepts an optional deadline to halt exploration, which is particularly useful for infinite state spaces. The `build_graph()` method runs the exploration to completion[69] and returns a `StateGraph` structure (with concrete instantiations `ReachabilityGraph` and `CoverabilityGraph`) representing the fully-explored state space, which offers more advanced methods to query the graph for properties (coverability and k-boundedness for the coverability graph, and additionally reachability, liveness, and deadlock-freedom for the reachability graph). Figure 25 visualizes `petrivet`'s state space API.

Furthermore, since the coverability graph and the reachability graph coincide for bounded Petri nets, a fully-explored `CoverabilityGraph` containing no ω-markings can be converted into a `ReachabilityGraph`, to benefit from the larger set of properties that can be answered by the reachability graph. This pattern is used frequently as `petrivet`'s last-resort method for deciding behavioral properties on unknown Petri nets: explore the coverability graph, and if we fully construct it without encountering any ω-markings, then we can safely convert it

[69] or attempts to do so: the documentation heavily warns the user that an unbounded reachability graph is infinite, and so this method might not terminate.

into a reachability graph and use the reachability graph's more powerful analysis methods. However, this is at the cost of a nontrivial runtime overhead of ω-acceleration, the implementation of which is shown in Algorithm 2.

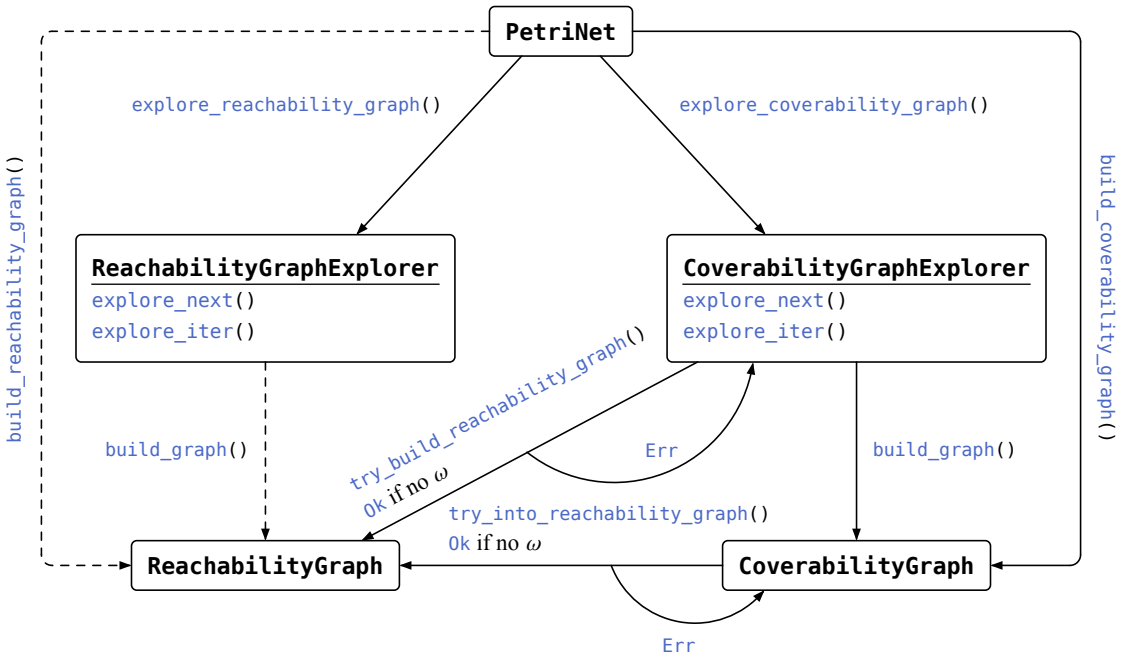


Figure 25: The state space API of `petrivet`. An explorer is started from a `PetriNet` and advanced incrementally, or run to completion with `build_graph()`. Shortcut methods on `PetriNet` do both in one step.
Dashed arrows indicate operations which do not terminate for unbounded Petri nets. The coverability side converts into a reachability graph only if no ω-marking occurs; `try_build_reachability_graph()` stops at the first ω-marking and returns the explorer, so exploration can resume.

Both explorers share one implementation, shown in Algorithm 1. The explorer stores the graph discovered so far, a map from each discovered marking to its node in the graph, and the frontier: a worklist of pairs of a node and a transition that *might be* enabled at its marking. Rather than testing every transition at every new marking, the algorithm adds the postset transitions of the support of the marking ($\langle M \rangle \bullet$) to the frontier for every newly discovered marking M, plus the ever-enabled source transitions. $\langle M \rangle \bullet$ is an over-approximation, since a transition may consume from further, unmarked places, so the enabling is checked later when the frontier item is popped (or dequeued, depending on depth-first or breadth-first) for exploration.

Algorithm 1: `StateGraphExplorer`(N, M_0)

```
1  G ← graph with the single node v_0 labelled M_0
2  T_source ← T ▷ filter(t ↦ • t = ∅)
3  F ← Candidates(M_0) ▷ map(t ↦ (v_0, t))
4  I ← {M_0 ↦ v_0}
5  procedure ExploreNext():
6  |  loop:
7  |  |  if F = ∅: return ⊥
8  |  |  (v, t) ← pop(F)
9  |  |  if • t ⊈ ⟨G[v]⟩: continue
10 |  |  M' ← fire(G[v], t)
11 |  |  Accelerate(v, M')     (coverability graph only)
12 |  |  (v', new) ← Register(v, t, M')
13 |  |  return (t, v', new)
14 procedure Candidates(M):
15 |  return ⟨M⟩ ▷ flat_map(p ↦ p •) ∪ T_source
16 procedure Register(v, t, M'):
17 |  if M' ∈ dom(I):
18 |  |  add the edge v -t-> I[M'] to G
19 |  |  return (I[M'], ⊥)
20 |  add a node v' labelled M' and the edge v -t-> v' to G;  I[M'] ← v'
21 |  for each t' ∈ Candidates(M'):  push(F, (v', t'))
22 |  return (v', ⊤)
```

Algorithm 1: State space exploration, shared by the reachability and the coverability graph explorers.

The only difference between reachability and coverability graph exploration is Line 11: before registering a new marking, the coverability graph explorer applies ω-acceleration to it (Algorithm 2). It searches backwards through the graph discovered so far for the ancestors of the new node. If the new marking covers any of its ancestors, unboundedness has been detected: the new marking enables all the firing sequences that the ancestor marking enabled, including the one that added the additional tokens, so that same firing sequence can be repeated indefinitely to produce arbitrarily many tokens in exactly those places which increased [20], [14, Thm. 3.2.8]. To avoid this infinite stream of new markings, those places are set to ω to mean arbitrarily large; unbounded. Then the new

marking is registered as usual, and the exploration continues. Algorithm 2 shows pseudocode for this process.

Algorithm 2: `Accelerate`(v, M')

```
1 S ← [v];  V ← ∅
2 while S ≠ ∅:
3 |  u ← pop(S)
4 |  if u ∈ V: continue
5 |  V ← V ∪ {u}
6 |  if G[u] < M′:
7 |  |  for each p ∈ P with M′(p) > G[u](p):  M′(p) ← ω
8 |  for each edge u′ → u in G:  push(S, u′)
```

Algorithm 2: ω-acceleration of a discovered marking M' during coverability graph exploration.

The performance of this algorithm, and an optimization which was applied to it, is discussed in the section preceding Figure 34.

Summary. The state space exploration algorithm is a generic implementation shared by both the reachability and coverability graph explorers. It separates the partially-explored graph from the full state space in the type system, providing a clear distinction between the two and allowing the user to explore the state space incrementally. This serves as a foundational, if expensive, procedure for the analysis of behavioral properties, which will be expanded upon in the following sections.

4.6. Structural Properties

This section discusses the implementation of advanced structural properties in `petrivet`. This will provide several useful tools for the behavioral analysis discussed in Section 4.7.

4.6.1. Traps and Siphons

Many procedures in `petrivet` rely on the siphons and traps of a net, which were introduced in Section 1.5.5.1. Specifically, they need the *maximal* siphon or trap in a given set of places $S \subseteq P$, which is unique because siphons and traps are closed under union. Typically S is the set of unmarked places in some marking M, i.e. $S = P \setminus \langle M \rangle$.

Both are computed by the same shrinking fixpoint algorithm [5, Alg. 6.19]. Starting from S, the algorithm repeatedly removes places that violate the siphon or trap condition until no more places can be removed.

- **Siphon:** p is removed if some transition $t \in {\bullet p}$ produces into p but consumes from no place in the current set, i.e. ${\bullet t} \cap S = \emptyset$. Such a t violates ${\bullet D} \subseteq D {\bullet}$.
- **Trap:** p is removed if some transition $t \in p {\bullet}$ consumes from p but produces into no place in the current set, i.e. $t {\bullet} \cap S = \emptyset$. Such a t violates $Q {\bullet} \subseteq {\bullet Q}$.

The set only ever shrinks, so the algorithm terminates after at most $|S|$ removals. The result is a siphon (or trap), since no place violating the property remains, and it is maximal, since a place belonging to some siphon (or trap) $D \subseteq S$ is never removed. If the result is empty, S contains no proper siphon (or trap).

Rather than rescanning all places after every removal, `petrivet` keeps a worklist of places to check. Removing p can only affect places that are connected to p through a transition in the relevant direction: for siphons, removing p can only invalidate the places $q \in p {\bullet} {\bullet}$, since only the transitions in $p {\bullet}$ lose an input place, so only those places are pushed onto the worklist again. Algorithm 3 shows the siphon version; the trap version is obtained by swapping every preset and postset.

Algorithm 3: `MaximalSiphonIn`(N, S)

```
1 D ← S;   W ← S
2 while W ≠ ∅:
3 |  p ← pop(W)
4 |  if p ∈ D and ∃t ∈ •p : •t ∩ D = ∅:
5 |  |  D ← D \ {p}
6 |  |  for each t ∈ p•, q ∈ t• with q ∈ D:  push(W, q)
7 return D
```

Algorithm 3: The maximal siphon contained in a set of places S.

Sets of places are represented as fixed-size bitsets over the place indices of the net, allowing membership tests and set operations to be performed in constant time. Complementing the support of a marking to obtain the initial set S is a single pass over the bitset.

These procedures are used throughout the behavioral analysis. A dead marking always has an unmarked proper siphon, so the maximal unmarked siphon is a cheap check for deadlock-freedom (see Section 4.7.3). A marked trap stays

marked and an unmarked siphon stays unmarked, so if the maximal trap that is unmarked in a target marking M is marked in M_0, then M is unreachable. This gives a quick unreachability check [14, Cor. 4.4.4] and several refinements of the CEGAR engine (see Section 4.7.4).

4.6.2. P-Invariants

P-invariants, P-subinvariants, and P-surinvariants (see Section 1.5.5.2) are used by several procedures in `petrivet`, each of which needs an invariant with some additional property. All of them share the same basic encoding as an SMT problem over integer variables, which consists of two parts: what **kind** of invariant is being sought, and **where** to look for it.

Kind of invariant. We introduce one integer variable $y_p \geq 0$ per place $p \in P$ to represent its weight in the invariant. For each transition $t \in T$, the weighted effect of firing t is $\sum_{p \in P} C(p,t) \cdot y_p$, the change in the weighted token sum when t fires. Depending on the kind of invariant, we constrain this effect with a relation $\{=, \leq, \geq\}$ for every transition:

$$\varphi_{\text{kind}} \coloneqq \bigwedge_{t \in T} \sum_{p \in P} C(p,t) \cdot y_p \bowtie 0$$

$$\text{where } \bowtie = \begin{cases} = \text{ for a P-invariant (constant weighted sum)} \\ \leq \text{ for a P-subinvariant (non-increasing)} \\ \geq \text{ for a P-surinvariant (non-decreasing)} \end{cases}$$

In the implementation, transitions which no effect on any place (self-loops only) give the trivial constraint $0 \bowtie 0$ and are omitted from the SMT problem.

Every solution y of φ_{kind} is an invariant of the requested kind, but so is the trivial solution $y = 0$, which is not useful. On its own, φ_{kind} is therefore not useful: we must add constraints which say where the invariant should be.

Covering a set of places. For a set of places $S \subseteq P$ that the invariant should cover, we require each of them to have positive weight in y:

$$\varphi_{\text{cover}(S)} \coloneqq \bigwedge_{p \in S} y_p \geq 1$$

For $S = P$ this asks for a positive invariant covering the whole net.

Separating two markings. It is also useful to seek an invariant which proves that a marking is not reachable from another marking, because they disagree on the constant or monotone weighted token sum of the invariant. For two markings M_A and M_B, we require the weighted token sums $y \cdot M = \sum_{p \in P} y_p \cdot M(p)$ to *violate* the invariant when going from M_A to M_B:

$$\varphi_{\text{separate}(M_A,M_B)} := \begin{cases} y \cdot M_B \neq y \cdot M_A \text{ for a P-invariant} \\ y \cdot M_B > y \cdot M_A \text{ for a P-subinvariant} \\ y \cdot M_B < y \cdot M_A \text{ for a P-surinvariant} \end{cases}$$

This constraint excludes the trivial solution on its own, since $y = 0$ gives $y \cdot M_A = y \cdot M_B = 0$. A solution of $\varphi_{\text{kind}} \wedge \varphi_{\text{separate}(M_A,M_B)}$ proves that M_B is not reachable from M_A: by the marking equation, every M reachable from M_A satisfies $M = M_A + Cx$ for some $x \in \mathbb{N}^{|T|}$, so

$$y \cdot M = y \cdot M_A + \left(y^T C\right)x.$$

For a P-invariant $y^T C = 0$, so $y \cdot M = y \cdot M_A$. For a P-subinvariant $y^T C \leq 0$ and $x \geq 0$, so $y \cdot M \leq y \cdot M_A$. For a P-surinvariant, symmetrically, $y \cdot M \geq y \cdot M_A$. In each case, M_B is excluded.

If the solver reports a model, then the assignment of the y_p is the invariant. Its support $\langle y \rangle = \{p \in P : y_p > 0\}$ gives the places it assigns positive weight to, and all other places are irrelevant to the invariant.

The covering constraint is used in Section 4.7.1 to decide structural boundedness, and the separating constraint is used in Section 4.7.4 to refine the CEGAR abstraction.

4.6.3. Commoner/Hack Criterion

The Commoner/Hack criterion (CHC), also called the Siphon-Trap Criterion, states that every proper siphon contains a marked trap. We encode the search for a CHC counterexample as a SAT problem following the approach of [49] et al.

The CHC fails iff there exists a proper siphon such that all traps contained in it are unmarked. Because this is an $\exists\forall$ statement, we cannot encode it directly. Instead we exploit that every set of places D contains a unique maximal trap, which the shrinking algorithm in Section 4.6.1 computes, and we run that algorithm symbolically in the SAT encoding. We then assert that this maximal trap is unmarked, which is equivalent to asserting that all traps in D are unmarked.

Siphons. We introduce one Boolean variable $x_p^{(0)}$ per place $p \in P$, with the intended meaning $x_p^{(0)} = \top$ iff $p \in D$.

The assignments satisfying

$$\varphi_{\text{siphon}} := \bigvee_{p \in P} x_p^{(0)} \wedge \bigwedge_{t \in T} \bigwedge_{p \in t\bullet} \left(x_p^{(0)} \rightarrow \bigvee_{q \in \bullet t} x_q^{(0)} \right)$$

correspond exactly to the siphons of N [49, Lemma 1]. The first conjunct enforces $D \neq \emptyset$ (at least one place is in the siphon). The second states $\bullet D \subseteq D \bullet$:

if a transition t produces into D, it must also consume from D. In the implementation the inner conjunction over $p \in t\bullet$ is folded into the premise $\bigvee_{p \in t\bullet} x_p^{(0)} \to \bigvee_{q \in \bullet t} x_q^{(0)}$, which is logically equivalent.

Maximal trap. The siphon D is not known when the formula is built, so we cannot run the worklist algorithm of Section 4.6.1 on it. Instead we use a round-based variant, in which each round removes *all* places that currently violate the trap property:

$$D_0 := D, \quad D_{i+1} := \{p \in D_i : \forall t \in p\bullet : t\bullet \cap D_i \neq \emptyset\}.$$

Lemma. $D_{|P|}$ is the maximal trap contained in D (empty if D contains no proper trap).

Proof. Each round removes the same places that the worklist algorithm could remove, so by the argument in Section 4.6.1 no place of a trap $Q \subseteq D$ is ever removed. The sequence is non-increasing and starts with at most $|P|$ places, so it reaches a fixpoint after at most $|P|$ rounds, and a fixpoint contains no violating place, so it is a trap. ■

We unroll this iteration inside the formula. For each stage $i \in \{1, \ldots, |P|\}$ and place $p \in P$ we define the term

$$x_p^{(i)} := x_p^{(i-1)} \wedge \bigwedge_{t \in p\bullet} \bigvee_{q \in t\bullet} x_q^{(i-1)}.$$

The term $x_p^{(i)}$ is true iff $p \in D_i$. This unrolling uses only the original $|P|$ free variables and no auxiliary variables. Stage i adds one term per place, and each term has size $O(|p\bullet| \cdot \max_t |t\bullet|)$, so the total size is $O(|P| \cdot (|P| + |T| + |F|))$.

Unmarked trap. The maximal trap must be unmarked or empty:

$$\varphi_{\text{unmarked}} := \bigwedge_{p \in P : M_0(p) > 0} \neg x_p^{(|P|)}.$$

Correctness. *Theorem.* The formula $\varphi := \varphi_{\text{siphon}} \wedge \varphi_{\text{unmarked}}$ is satisfiable iff N has a proper siphon that contains no marked trap. Equivalently, φ is unsatisfiable iff the CHC holds.

Proof. ($\Rightarrow$) Let β satisfy φ and set $D := \{p : \beta(x_p^{(0)}) = \top\}$. By φ_{siphon}, D is a proper siphon. By the Lemma, $D_{|P|}$ is the maximal trap in D, and $\varphi_{\text{unmarked}}$ gives $M_0(D_{|P|}) = 0$. Any trap $Q \subseteq D$ satisfies $Q \subseteq D_{|P|}$, so $M_0(Q) = 0$.
($\Leftarrow$) Let D be a proper siphon with no marked trap, and let β assign $x_p^{(0)}$ true iff $p \in D$. Then φ_{siphon} holds. The maximal trap $D_{|P|}$ of D is unmarked or empty, so $\varphi_{\text{unmarked}}$ holds. ■

When φ is satisfiable, the assignment gives a counterexample: the siphon D and its maximal trap $D_{|P|}$ (possibly empty).

The following pseudocode mirrors the implementation of the above encoding in `petrivet`, which returns a counterexample if one exists, or $\bot$ if the CHC holds.

Algorithm 4: `FindProperSiphonWithUnmarkedTrap`$(|P|, M_0)$

for each $p \in P$: introduce a Boolean variable x_p
$\boldsymbol{x}^{(0)} \leftarrow \left(x_p\right)_{p \in P}$
assert $\bigvee_{p \in P} x_p^{(0)}$
for each $t \in T$:
  assert $\left(\bigvee_{p \in t^\bullet} x_p^{(0)}\right) \rightarrow \left(\bigvee_{p \in {}^\bullet t} x_p^{(0)}\right)$
define $\mathrm{Step}(\boldsymbol{x}) \coloneqq \mathrm{zip}(P, \boldsymbol{x}) \rhd \mathrm{map}\Big(\left(p, x_p\right) \mapsto x_p \wedge \bigwedge_{t \in p^\bullet} \bigvee_{q \in t^\bullet} x_q\Big)$
$\boldsymbol{x}^{(|P|)} \leftarrow \mathrm{fold}\big(\boldsymbol{x}^{(0)}, |P| \text{ times}, \mathrm{Step}\big)$
for each $p \in P$ **with** $M_0(p) > 0$:
  assert $\neg x_p^{(|P|)}$
if the solver reports UNSAT: **return** $\bot$
$D \leftarrow \left\{p \in P : \beta\left(x_p^{(0)}\right) = \top\right\}$
$Q \leftarrow \left\{p \in P : \beta\left(x_p^{(|P|)}\right) = \top\right\}$
return (D, Q)

Algorithm 4: Counterexample search for the CHC as a Boolean formula.

4.7. Behavioral Properties

The following sections discuss the API design and implementation of `petrivet` for the behavioral properties discussed in Section 1.5.4, following the guiding principles outlined in Section 3.

A `PetriNet` provides methods for behavioral properties which attempt to answer the query as quickly as possible by dispatching to specialized procedures. This is done on a best-effort basis: which procedure will yield the answer fastest is generally not known for sure, and we must rely on heuristics. Some procedures are complete and will always return a definitive answer, while others are incomplete, and we might need to fall back on a more expensive procedure if the first one does not yield a definitive answer.

A `PetriNet` also caches the results of some property checks to save time on subsequent queries. This cache is invalidated if the Petri net's marking is modified, as the behavioral properties only hold in the context of a specific marking.

Each of the next sections describes the scope and granularity of a behavioral property, followed by the design of `petrivet`'s API for checking that property, and then the implementation backing the API, including pseudocode and correctness proofs of the algorithms used.

4.7.1. (K-)Boundedness

This section describes the implementation of the (k-)boundedness property of Petri nets within `petrivet`. See Section 1.5.4.1 for a discussion of the property itself.

4.7.1.1. Scope and Granularity

Scope. (K-)Boundedness can be checked for the entire Petri net, or for a specific place.

Granularity. Boundedness (with or without `k`) can be framed as a yes/no question ("is the Petri net (k-)bounded?"), but also as a search for the minimal `k` (the exact bound).

4.7.1.2. API

`petrivet` offers two entry points for checking (k-)boundedness of a `PetriNet`, covering all combinations:

1. `boundedness() -> Option<Boundedness>` returns either `Bounded(k)` with the minimal k, or `Unbounded`. The user can optionally specify the place to check, otherwise the entire Petri net is checked.
2. `is_bounded() -> Option<bool>` corresponds to the yes/no framing. In addition to optionally specifying the place, the user can also optionally specify a specific value of `k`, to check for k-boundedness instead of general boundedness.

Furthermore, both methods also allow specifying a restriction to only use efficient procedures, in which case they return `None` if no efficient procedure is known for the given net, and an optional timeout to limit the maximum duration of the analysis.

4.7.1.3. Implementation

`petrivet` descends an array of boundedness procedures ordered by increasing computational cost, checking each in turn until the query is answered or a cost-control mechanism is triggered. These procedures sometimes yield exact bounds, and sometimes only upper bounds (for example, a place is guaranteed 5-bounded, but might also be 4-bounded). The initial marking also gives a lower bound for the exact bound of a place (the exact bound of a place which starts with 3 tokens is at least 3). `petrivet` therefore tracks a lower and upper bound for each place, and can conclude the exact bound when the lower and upper bounds meet.

State Machines. State machines (including circuits) are conservative, meaning their token count is constant across all reachable markings. Therefore, they are always bounded, and k-bounded for any k greater than or equal to the initial token count.

Determining the exact bound of a place requires a case distinction, however: if the net is strongly connected, then all tokens can reach any place, and so every place is exactly k-bounded for k equal to the initial token count.

If the net is not strongly connected, then not all tokens can reach every place. We can determine the exact bound of a place p by computing $M_0(\mathrm{Pre}^*(p))$: the number of tokens in the initial marking that can reach p [4].

Marked Graphs. Live marked graphs, as discussed in Section 1.5.5.3, are bounded if every place lies on a circuit. Every place is bounded by the minimum number of tokens on the circuits it lies on, and unbounded if it does not lie on any circuit [14, Thm. 5.2.4]. If the marked graph is not live, then some circuits are unmarked, which can prevent tokens on other circuits from circulating, leading to some places being bounded by a lower number than the minimum of the circuits they lie on. `petrivet` therefore computes the minimum number of tokens on the circuits a place lies on, and records this as exact if the marked graph is live, and as an upper bound if it is not live. See Section 4.7.2.3 for the procedure used to determine liveness of marked graphs.

Structural Boundedness. If the query has not been answered by the above procedures, then `petrivet` searches for a P-subinvariant which covers the places of interest $S \subseteq P$, using the encoding $\varphi_{\text{kind}} \wedge \varphi_{\text{cover}(S)}$ from Section 4.6.2 with $\bowtie = \{\leq\}$. For queries about the entire Petri net, $S = P$. This is a linear programming problem, and can be solved in polynomial time by an SMT solver. If a solution y exists, then the weighted token sum $y \cdot M$ can never exceed its initial value $y \cdot M_0$, and since every place $p \in S$ has weight $y_p \geq 1$, it is bounded by $\left\lfloor \frac{y \cdot M_0}{y_p} \right\rfloor$ for any M_0 (*structural* boundedness) [4], [5]. This is only a semi-decision procedure: the lack of a P-subinvariant gives us no information.

Exploring the Coverability Graph. As a last resort, `petrivet` can explore the coverability graph of the Petri net, which is guaranteed to provide full information about the boundedness of every place, but is expensive to compute. The stopping condition for the exploration depends on the query: `is_bounded()` queries can stop with the verdict `false` as soon as a (or the) place is found to be unbounded (or not k-bounded); otherwise the whole graph must be explored. `boundedness()` queries must explore the entire coverability graph to find the exact bounds.

4.7.2. Liveness

This section describes the implementation of the liveness property of Petri nets within `petrivet`. See Section 1.5.4.4 for a discussion of the property itself.

4.7.2.1. Scope and Granularity

Scope. Liveness is simultaneously a property of the entire Petri net and of individual transitions.

Granularity. Liveness can be framed as a yes/no question ("is the Petri net L_k-live?"), but also as a search for the *strict* liveness level (the maximum k).

4.7.2.2. API

`petrivet` offers two entry points for checking liveness of a `PetriNet`, covering all combinations:

1. `liveness() -> Option<LivenessLevel>` returns the strict liveness level, L_0 through L_4. The user can optionally specify a transition to check, otherwise the entire Petri net is checked.
2. `is_live() -> Option<bool>` corresponds to the yes/no framing. In addition to optionally specifying the transition, the user can also specify a minimum liveness level to check for, e.g. "is this transition at least L_3-live?". If no level is specified, L_4 is assumed.

Furthermore, both methods also accept an optional restriction to only efficient procedures and/or a timeout to limit the maximum duration of the analysis.

4.7.2.3. Implementation

`petrivet` descends an array of liveness procedures ordered by increasing computational cost, checking each in turn until the query is answered or a cost-control mechanism is triggered. These procedures sometimes yield exact liveness levels, and sometimes only lower bounds (for example, we have seen a transition fire once, so it is at least L_1-live, but we cannot yet conclude it is not L_2-live). Therefore, `petrivet` tracks a lower and upper bound on the true liveness level of each transition, and can conclude the exact level when the lower and upper bounds meet.

Unmarked Siphon. The first procedure computes the maximal siphon contained in the places unmarked in M_0, using the algorithm of Section 4.6.1 with $S = P \setminus \langle M_0 \rangle$. If it is non-empty, it stays unmarked in every reachable marking, so every transition in its postset has an input place that never holds a token and can never fire. This cheaply proves those transitions to be L_0.

State Machines. The liveness of transitions in state machines is determined by the structure of the strongly connected components (SCCs) of the net, and the

placement of the tokens within them. Transitions within an SCC always have the same liveness level. If there are no tokens in the initial marking, then all transitions are L_0. If there is at least one token, then all transitions are L_4-live if the net is strongly connected [4, Thm. 4], [14, Thm. 5.1.3]. If the net is not strongly connected, then the answer is more nuanced.

The following algorithm computes the liveness levels of all transitions in a state machine in linear time $O(|P| + |T| + |F|)$. To my knowledge this is an original contribution. First, the algorithm computes the condensation graph of the state machine using a linear time SCC algorithm (implementation uses Kosaraju's algorithm via `petgraph` [50]). A pass through the SCCs in topological order finds all SCCs reachable by a token. A pass in reverse topological order computes for every SCC whether it reaches exactly one terminal SCC using the following reduction $\sqcup$:

$$\mathrm{One}(a) \sqcup \mathrm{One}(b) \coloneqq \begin{cases} \mathrm{One}(a) \text{ if } a = b \\ \mathrm{Multiple} \text{ otherwise} \end{cases}$$

$$\mathrm{Multiple} \sqcup x \coloneqq \mathrm{Multiple} =: x \sqcup \mathrm{Multiple}.$$

Reachable trivial SCCs are L_1, reachable nontrivial ones L_3, and a nontrivial SCC that is the unique terminal SCC reachable from some marked SCC is L_4.

Algorithm 5: `StateMachineLiveness`(N, M_0)

$\mathscr{C} \leftarrow \mathrm{condensation}(N)$; $\pi \leftarrow \mathrm{toposort}(\mathscr{C})$
$m[K] \leftarrow (M_0(K) > 0)$ for all $K \in \mathscr{C}$; $r \leftarrow m$
for each $K \in \pi$ **with** $r[K]$:
  for each successor K' of K: $r[K'] \leftarrow \top$
for each $K \in \mathrm{rev}(\pi)$:
  if K has no successor: $\sigma[K] \leftarrow \mathrm{One}(K)$
  else $\sigma[K] \leftarrow \mathrm{succ}(K) \rhd \mathrm{map}(K' \mapsto \sigma[K']) \rhd \mathrm{reduce}(\sqcup)$
$\Sigma \leftarrow \{K' : \exists K \in \mathscr{C} : m[K] \wedge \sigma[K] = \mathrm{One}(K')\}$
$\ell \leftarrow \mathscr{C} \rhd \mathrm{map}\left(K \mapsto \begin{cases} \mathrm{L}_0 \text{ if } \neg r[K] \\ \mathrm{L}_1 \text{ if } |K| = 1 \\ \mathrm{L}_4 \text{ if } K \in \Sigma \\ \mathrm{L}_3 \text{ otherwise} \end{cases}\right)$
return $\mathscr{C} \rhd \mathrm{flat_map}(K \mapsto K \rhd \mathrm{filter_map}(v \mapsto (v, \ell[K]) \text{ if } v \in T))$

Algorithm 5: Liveness levels of a state machine.

A formal proof of correctness is given in Section 8.2.

Marked Graphs. The liveness of transitions in marked graphs centers around the presence of unmarked circuits [4, Thm. 7], [14, Thm. 5.2.3]. The transitions within them are L_0-live (dead), and the transitions downstream of them are at most L_1-live. All transitions not downstream of an unmarked circuit are L_4-live.

I present a linear-time algorithm ($O(|P| + |T| + |F|)$) for computing the liveness levels of all transitions in a marked graph, based on the work of [4], [14], and [51]. To my knowledge this is an original contribution. The algorithm first sets the liveness level of all transitions to L_4. Then it identifies all unmarked circuits using Tarjan's SCC algorithm [51] on the subnet induced by all transitions and only the unmarked places. By [4], all transitions in unmarked circuits are L_0. Then, for each dead transition, the algorithm performs a depth-first search to mark downstream transitions as L_1 if all paths from the dead transitions to them pass through a marked place, and L_0 otherwise.

Algorithm 6: `MarkedGraphLiveness`(N, M_0)

$N_0 \leftarrow N[T \cup \{p \in P : M_0(p) = 0\}]$
$Z \leftarrow \mathrm{SCC}(N_0) \triangleright \mathrm{filter}(|C| > 1) \triangleright \mathrm{flat_map}(C \mapsto C) \triangleright \mathrm{filter_map}(v \mapsto v \text{ if } v \in T)$
$\ell \leftarrow (\mathrm{L}_4)_{t \in T}$
for each $z \in Z$:
  $\mathrm{Dfs}(z, \bot)$
return $\mathrm{zip}(T, \ell)$
procedure $\mathrm{Dfs}(t, b)$:
  match $\ell[t]$:
    L_0: **return**
    L_1 **if** b: **return**
    otherwise: $\ell[t] \leftarrow \begin{cases} \mathrm{L}_1 \text{ if } b \\ \mathrm{L}_0 \text{ otherwise} \end{cases}$
  for each $p \in t\,\bullet$:
    $\{t'\} \leftarrow p\,\bullet$
    $b' \leftarrow b \vee M_0(p) > 0$
    $\mathrm{Dfs}(t', b')$

Algorithm 6: Liveness levels of a marked graph.

A formal proof of correctness is given in Section 8.3.

Commoner/Hack Criterion. For free-choice and asymmetric-choice nets, the Commoner/Hack criterion (checked as described in Section 4.6.3) is sufficient to

show that all transitions are L_4. If it does not hold, we know that at least one transition in a free-choice net is not L_4, but not which one, and we know nothing additional for asymmetric-choice nets.

Coverability Graph Exploration. As a final fallback, `petrivet` explores the coverability graph, with the hope that the Petri net is bounded so that the coverability graph is also the reachability graph. If it is bounded and we do obtain the reachability graph, analyzing its SCCs provides a decision procedure for the liveness level of every transition:
- a transition which is never fired is L_0,
- a transition which appears in all terminal SCCs is L_4,
- a transition which appears in any nontrivial SCC is L_3, and
- L_1 otherwise.

If the Petri net is unbounded, then we halt exploration after the first ω-marking is reached, and can at least conclude that all transitions which have fired so far are L_1-live. There is currently no procedure which can confirm strict L_2-liveness implemented in `petrivet`: The Petri nets where strict L_2-liveness can arise are unbounded and of more general structural class, which is a particularly difficult case for liveness analysis.

`petrivet` therefore does not have a complete decision procedure for liveness in unbounded Petri nets, and will invariably return “unknown” for some queries. Future work may improve this situation, for example by reformulating a liveness query as a reachability query, and then applying the reachability procedures described in Section 4.7.4.

4.7.3. Deadlock-Freedom

This section describes the implementation of the deadlock-freedom property of Petri nets within `petrivet`. See Section 1.5.4.5 for a discussion of the property itself.

4.7.3.1. Scope and Granularity

Scope. Deadlock-freedom, unlike boundedness and liveness, is a property of the entire Petri net only, and not of any individual node.

Granularity. Deadlock-freedom can be framed as a yes/no question (“is the Petri net deadlock-free?”), or as a search for all reachable deadlocks.

4.7.3.2. API

`petrivet` offers two entry points for checking deadlock-freedom of a `PetriNet`, covering both combinations:

1. `is_deadlock_free() -> Option<bool>` returns Some(true) if the Petri net is deadlock-free, Some(false) if it is not deadlock-free, and None if the analysis was inconclusive.
2. `deadlocks() -> Deadlocks` returns an iterator over all reachable deadlock markings, if any exist.

The former supports both cost-control mechanisms, while the latter only supports a timeout. This is because currently no efficient procedures are implemented for enumerating all reachable deadlocks, and so the efficiency restriction would always return `None` for this method. Future work may implement efficient procedures for enumerating deadlocks, in which case the efficiency restriction could be supported.

4.7.3.3. Implementation

The following procedures are utilized for `PetriNet::is_deadlock_free()`, in order of increasing computational cost.

Initial Marking. Trivial: if the initial marking is a deadlock, then the Petri net is not deadlock-free, and the initial marking is the only reachable deadlock.

Circuits. A circuit is deadlock-free if and only if it has at least one token in its initial marking.

Source Transitions. The existence of a transition with an empty preset is a sufficient condition for deadlock-freedom, because such a transition is vacuously enabled at every reachable marking [37, Rule 6].

State Machines. A deadlock in a state machine puts all tokens on sink places (places with an empty postset), or has no tokens at all. A natural consequence of this is that a strongly connected state machine, where every place has a non-empty postset, will always be deadlock-free if it has at least one token. A state machine that is not strongly connected may have sink places, but a deadlock is only reachable if all tokens can reach a sink. Tokens move independently, so a deadlock is reachable iff every marked place can reach a sink. The algorithm therefore computes the set of places that can reach a sink by a backward search from the sink places, and reports deadlock-freedom iff some marked place is not among them.

Algorithm 7: `DeadlockFreeStateMachine`(N, M_0)

$P_0 \leftarrow P \rhd \text{filter}(p \mapsto M_0(p) > 0)$
$\Sigma \leftarrow P \rhd \text{filter}(p \mapsto p\bullet = \emptyset)$
$B \leftarrow \text{PreStar}(\Sigma)$
return $P_0 \not\subseteq B$
procedure $\text{PreStar}(X)$:
  $W \leftarrow \text{list}(X)$
  while $W \neq \emptyset$:
    $p \leftarrow \text{pop}(W)$
    for each $t \in \bullet p$ **and each** $q \in \bullet t$:
      if $q \notin X$: $X \leftarrow X \cup \{q\}$; $\text{push}(W, q)$
  return X

Algorithm 7: Deadlock-freedom of a state machine.

A proof of correctness is given in Section 8.4.

Marked Graphs. For strongly connected marked graphs, liveness, deadlock-freedom, and the existence of an infinite firing sequence perfectly coincide [14, Thm. 5.2.8]. All hold iff every circuit is marked in the initial marking [4, Thm. 7]. For marked graphs that are not strongly connected, deadlock-freedom can hold even if liveness does not, and it only requires that some transition is not downstream of an unmarked circuit. Because the transitions on an unmarked circuit are dead [4], any transition downstream of an unmarked circuit will eventually die too, limited by the number of tokens the initial marking places on the path from the unmarked circuit to that transition. But a transition that is not on or downstream of an unmarked circuit is necessarily live [4], and so no deadlock marking can ever occur.

The following algorithm implements this reasoning: we search for a transition which is not on or downstream of an unmarked circuit, and report deadlock-freedom iff one is found.

Algorithm 8: `DeadlockFreeMarkedGraph`(N, M_0)

$N_0 \leftarrow N[T \cup \{p \in P : M_0(p) = 0\}]$
$Z \leftarrow \mathrm{SCC}(N_0) \triangleright \mathrm{filter}(|C| > 1) \triangleright \mathrm{flat_map}(C \mapsto C) \triangleright \mathrm{filter_map}(v \mapsto v \text{ if } v \in T)$
$d \leftarrow (\bot)_{t \in T}$; $W \leftarrow Z$
while $W \neq \emptyset$:
    $t \leftarrow \mathrm{pop}(W)$
    if $\neg d[t]$:
        $d[t] \leftarrow \top$
        for each $p \in t\,\bullet$: $\mathrm{extend}(W, p\,\bullet)$
return $\exists t \in T : \neg d[t]$

Algorithm 8: Deadlock-freedom of a marked graph.

A proof of correctness is given in Section 8.5.

If the procedures so far have not provided an answer and the efficient-only cost-control mechanism has been enabled, then at this point `petrivet` will give up and return `None`. Otherwise, it will continue to the expensive fallback procedures.

Commoner/Hack Criterion. Every dead marking has a proper siphon which is unmarked [14, Prop. 4.4.6]. If the Commoner/Hack criterion holds, every proper siphon contains a trap which is marked initially, and a marked trap stays marked (see Section 1.5.5.1), so no siphon can ever become unmarked and no dead marking is reachable. The Commoner/Hack criterion is therefore a sufficient condition for deadlock-freedom of general Petri nets. However, if it does not hold, we know little to nothing more of use.

Reachability Graph Exploration. As a last resort, `petrivet` can explore the reachability graph of the Petri net, searching for deadlock markings. This is a risky procedure, because the current implementation does not ensure that the Petri net is bounded. As a safer alternative, `petrivet` could explore the coverability graph instead, halting on the detection of the first unbounded ω-marking, similar to the liveness procedure. Neither is a perfect solution, however, and further work is needed to implement a safe and complete procedure for deadlock-freedom of general Petri nets.

4.7.4. Reachability and Coverability

This section describes the implementation of reachability and coverability properties of Petri nets within `petrivet`. See Section 1.5.4.2 and Section 1.5.4.3 for a discussion of the properties themselves.

4.7.4.1. Scope and Granularity

Scope. Reachability and coverability are properties of the entire Petri net with respect to a specific target marking.

Granularity. Reachability and coverability can be framed as a yes/no question ("is the target marking reachable/coverable?"), or, more granularly, as a search either for a witness firing sequence fulfilling the property or for evidence that no such firing sequence can possibly exist.

4.7.4.2. API

`petrivet` offers two entry points for checking reachability and coverability of a `PetriNet`, covering both combinations:

1. `is_reachable(target: Marking<u32>) -> Option<bool>` returns Some(true) if the marking is reachable, Some(false) if it is not reachable, and None if the analysis was inconclusive.
2. `is_coverable(target: Marking<u32>) -> Option<bool>` returns Some(true) if the marking is coverable, Some(false) if it is not coverable, and None if the analysis was inconclusive.
3. `reachability(target: Marking<u32>) -> Option<Reachability>` returns Some(Reachability::Reachable { firing_sequence }) with a witness firing sequence if the target marking is reachable, Some(Reachability::Unreachable { contradiction }) with a set of contradictory lemmas if it is not reachable, and None if the analysis was inconclusive.
4. `coverability(target: Marking<u32>) -> Option<Coverability>` returns Some(Coverability::Coverable { marking, firing_sequence }) with a witness covering marking and firing sequence if the target marking is coverable, Some(Coverability::Uncoverable { contradiction }) with a set of contradictory lemmas if it is not coverable, and None if the analysis was inconclusive.

`is_reachable()` and `is_coverable()` support both cost-control mechanisms, while `reachability()` and `coverability()` only support a timeout. This is because currently no efficient procedures are implemented which can return a witness firing sequence or evidence of unreachability/uncoverability, and so the efficiency restriction would always return `None` for these methods. This could be added in the future.

4.7.4.3. Implementation

This section describes the implementation of reachability and coverability together, as their implementation shares a common engine in `petrivet`.

Strongly connected state machines. Since tokens circulate freely in strongly connected state machines, a target marking M is reachable from M_0 in a circuit iff $M(P) = M_0(P)$, and coverable from M_0 iff $M(P) \leq M_0(P)$. This does not yield a witness firing sequence or evidence of unreachability/uncoverability, so is only used in the `is_reachable()` and `is_coverable()` methods.

Maximal unmarked trap. If the maximal unmarked trap in the target marking is marked in the initial marking, then the target marking is unreachable. This is a direct result of the fact that traps remain marked once they are marked, and can be checked using the procedure in Section 4.6.1. Note that this only applies to reachability, and not to coverability: the target marking may not mark an initially marked trap, but the covering marking can mark it. This also does not yield a witness firing sequence or evidence of unreachability/uncoverability, so is only used in the `is_reachable()` method.

Fallback procedures. In early versions of `petrivet`, the only fallback approach was simply to explore the reachability graph breadth-first and check each marking for equality or coverability with the target marking. While this produces a witness firing sequence for positive results, and its evidence for the unreachability/uncoverability of the target marking in negative results is absolute, it is intractable for all but the smallest bounded Petri nets.

The current implementation is more sophisticated. It is based on the paradigm of counterexample-guided abstraction refinement (CEGAR) [52], a well-known technique in formal verification and model checking that has been applied to Petri nets before [37], [53], [54]. CEGAR provides an effective semi-decision procedure for various properties while largely avoiding the state space explosion problem.

The paradigm constructs and iteratively refines a behavioral abstraction (overapproximation) of the original system, maintaining the invariant that if the property of interest does not hold in the abstraction, then by construction it cannot not hold in the original system either. If the property of interest does hold in the abstraction, then it yields a potential solution which can be checked by simulating it in the original system. The simulation either confirms the solution or shows it to be spurious, yielding a counterexample which is used to refine the abstraction and prevent similarly spurious solutions from being generated in the future. Repeating this process eventually yields either a solution valid in the original system, or an abstracted proof that no solution exists: some combination

of insights from the original system eliminates all potential solutions from the abstraction. Figure 26 illustrates the CEGAR paradigm as a BPMN diagram.

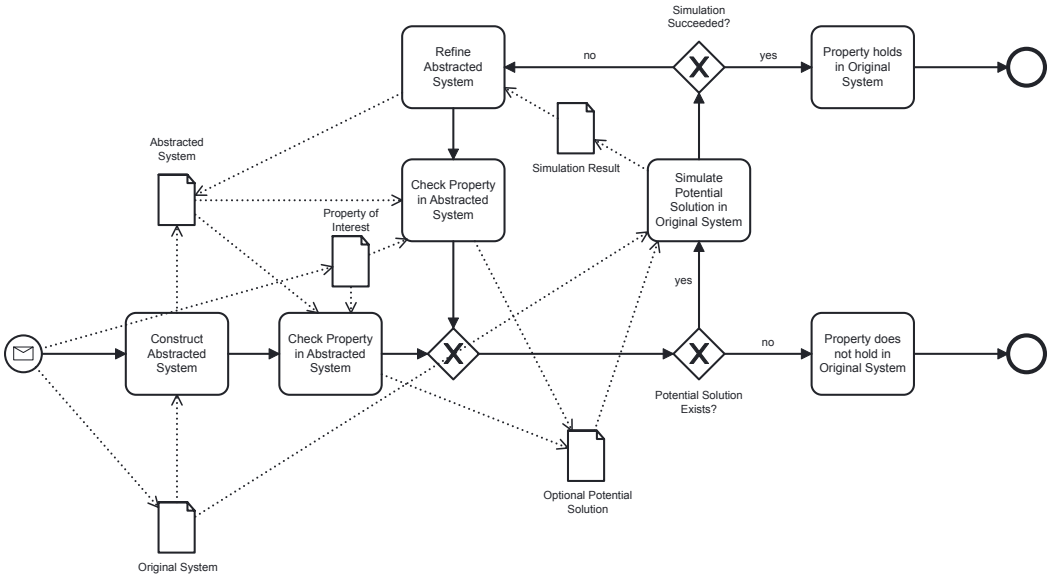


Figure 26: Counterexample-guided abstraction refinement (CEGAR) paradigm.

CEGAR can be implemented elegantly using a satisfiability modulo theories (SMT) solver such as Z3 [55]. The abstraction is encoded as an SMT problem definition, and checking the property becomes a satisfiability check using the SMT solver. Refinements are performed by adding additional constraints, and sometimes additional variables, to the SMT problem definition. Simulation, the source of counterexamples and the refinements derived from them, is specific to the type of system being analyzed.

In the case of Petri nets, simulation is not just a matter of firing transitions. While a witness firing sequence or lack thereof is the ultimate evidence for or counterexample to the property of interest, there are several (much cheaper) methods to find counterexamples which do not require simulating the Petri net, and instead only require analyzing the structure of the net with respect to the potential solutions produced by the SMT solver. These refinements delay confrontation with the state space explosion problem as long as possible. By arranging these refinement opportunities in order of increasing computational cost, we can often find cheap wins and avoid expensive simulation entirely, and if we do have to simulate, the likelihood of the simulation being successful is much higher, since the potential solution has been refined to be more likely to be valid.

4.7.5. CEGAR Engine

`petrivet` implements a CEGAR engine for the reachability and coverability properties of Petri nets, which backs the API methods described above. The abstraction is a set of constraints Φ, which an SMT solver checks for satisfiability. Initially, Φ contains one non-negative integer variable $M(p)$ per place $p \in P$, constrained by the target marking M_{target}:

$$\Phi_0 \coloneqq \bigwedge_{p \in P} M(p) \bowtie M_{\text{target}(p)} \quad \text{where } \bowtie = \begin{cases} = & \text{for reachability} \\ \geq & \text{for coverability} \end{cases}$$

The only behavioral difference between reachability and coverability is in the choice of $\bowtie$; all other aspects of the CEGAR engine are identical for both property types. Every refinement adds constraints which hold in every reachable marking (or every realizable firing sequence), so Φ over-approximates the set of reachable markings satisfying the target condition, and if Φ becomes unsatisfiable, then the target marking is unreachable/uncoverable.

Most refinements are implemented as a pair of functions: one to search for a reason why the solver's current solution is spurious, and one to encode that insight into Φ, eliminating the spurious solution (and hopefully many others) from the search space. Others are applied once, preemptively, without any spurious solution.

The engine runs in two phases, shown in Algorithm 9. In the *structural* phase, Φ only contains the place variables, so a candidate solution is just a marking M, and only structural refinements (traps and siphons, P-invariants) are applied. These are cheap, and often enough to prove unreachability. When no structural refinement can exclude the candidate marking, the engine enters the *behavioral* phase: it adds one integer variable $X(t) \geq 0$ per transition together with the marking equation, so a solution becomes a pair (M, X) of a marking and a Parikh vector. Besides the structural refinements, this phase applies refinements that reason about X, and finally tries to realize X as a concrete firing sequence in the original Petri net.

Algorithm 9: `CegarDecide`$\big(N, M_0, M_{\text{target}}, \bowtie\big)$

$\Phi \leftarrow \bigwedge_{p \in P} M(p) \bowtie M_{\text{target}(p)}$
$D \leftarrow \text{MaximalSiphonIn}(N, P \setminus \langle M_0 \rangle)$; $\Phi \leftarrow \Phi \wedge \bigwedge_{p \in D} M(p) = 0$
loop: *(structural phase)*
  if Φ is unsatisfiable: **return** unreachable
  if the solver gives up: **return** $\bot$
  $M \leftarrow$ the solver's model of the place variables
  if $\text{MarkedTrap}(M)$ or $\text{Invariant}(M)$ yields a constraint ψ:
    $\Phi \leftarrow \Phi \wedge \psi$; **continue**
  break
$\Phi \leftarrow \Phi \wedge \bigwedge_{t \in T} X(t) \geq 0 \wedge \text{MarkingEquation} \wedge \text{TransitionOrdering}$
loop: *(behavioral phase)*
  if Φ is unsatisfiable:
    if the unsat core contains an increment constraint: **return** $\bot$
    return unreachable
  if the solver gives up: **return** $\bot$
  $(M, X) \leftarrow$ the solver's model of the place and transition variables
  if $\text{MarkedTrap}(M)$, $\text{TrapBecomesMarked}(M, X)$ or $\text{Invariant}(M)$ yields a constraint ψ:
    $\Phi \leftarrow \Phi \wedge \psi$; **continue**
  if $\text{Realize}(X)$ finds a firing sequence σ: **return** reachable via σ
  $\Phi \leftarrow \Phi \wedge \text{Increments}(X)$

Algorithm 9: The two-phase CEGAR procedure for reachability and coverability.

The refinements are tried in the order listed, and only the first refinement which excludes the current candidate solution is applied before the next iteration of the loop. The candidate marking which ends the structural phase is discarded: the solver has to propose a new one after the marking equation and transition ordering constraints are added to begin the behavioral phase. When Φ becomes unsatisfiable, `petrivet` also returns the solver's UNSAT core, mapped back to the refinements it consists of, as a human-readable explanation of why the target marking is unreachable/uncoverable.

The following paragraphs explain the individual refinements in more detail.

Initially Unmarked Siphons. A siphon which is unmarked in the initial marking must remain unmarked in every reachable marking. `petrivet` computes the

maximal such siphon once at the beginning of the CEGAR procedure (Section 4.6.1, with $S = P \setminus \langle M_0 \rangle$), and sets every place variable it contains to zero:

$$\bigwedge_{p \in D} M(p) = 0.$$

This preemptive refinement prevents the solver from placing tokens where they can never be placed.

Initially Marked Traps. A trap which is marked in the initial marking must remain marked in every reachable marking, so a potential solution marking M which leaves such a trap unmarked is spurious [56]. `petrivet` computes the maximal trap Q among the places unmarked in M (Section 4.6.1, with $S = P \setminus \langle M \rangle$). Every trap unmarked in M is contained in Q, so if any initially marked trap is unmarked in M, then Q is initially marked as well. In that case the SMT problem is refined with the constraint $\sum_{p \in Q} M(p) \geq 1$.

This is the *maximal M-trap* strategy of [56, Sec. 6]. It is cheap to compute, but since every other such trap is contained in Q, it gives the weakest of the constraints that could be derived. [40] and [37] instead find such a trap with a separate SAT query, which can return a smaller trap and a stronger constraint, at the cost of a SAT call per refinement.

P-invariants. Given a potential solution marking M, we search for an invariant which separates M from the initial marking, using the encoding $\varphi_{\text{kind}} \wedge \varphi_{\text{separate}(M_0, M)}$ from Section 4.6.2. A solution proves that M is not reachable, so M is a spurious solution of the abstraction.

`petrivet` first searches for a P-invariant ($\bowtie = \{=\}$) following [37]. If none exists, `petrivet` also searches for either a P-subinvariant or P-surinvariant in a single query ($\bowtie = \{\leq, \geq\}$). Exact P-invariants are preferred because they give a stronger refinement, but any of the three types of invariants can be used to refine the abstraction. All three variants of φ_{kind} are encoded once, each guarded by a Boolean selector variable, in a separate solver instance from the main abstraction. For each candidate M, only the invariant type selector and $\varphi_{\text{separate}(M_0, M)}$ are asserted, and they are retracted again afterwards, so the same instance is reused for every candidate of a CEGAR run.

If an invariant y is found, the abstraction is refined with the constraint

$$\sum_{p \in \langle y \rangle} y(p) \cdot M(p) \bowtie y \cdot M_0$$

over the abstraction's place variables $M(p)$, with the same relation $\bowtie$ that was used to find y. The constraint holds in every reachable marking but not in the

candidate, so the candidate is excluded from all later iterations, together with every other spurious marking that violates the same invariant.

Marking Equation. On entering the behavioral phase, we introduce a non-negative integer variable $X(t)$ for each transition $t \in T$, and relate each place variable to the initial marking and the transition variables according to the marking equation:

$$\bigwedge_{p \in P} M(p) = M_0(p) + \sum_{t \in T} C(p,t) \cdot X(t)$$

From this point on, a potential solution from the SMT solver consists of a marking M and a Parikh vector X.

Transition Ordering. The marking equation has no notion of the order in which transitions fire. It therefore admits Parikh vectors containing cycles of transitions which "borrow" tokens that do not exist: a transition t consumes from a place p which is insufficiently marked initially, and is repaid by a transition which in turn depends on t. [37] introduces two refinements against this.
Let C^- and C^+ be the pre- and post-incidence matrices, so that $C = C^+ - C^-$, and let $F(p,t) \coloneqq \{t' \in {\bullet p} \setminus \{t\} : C(p,t') > 0\}$ be the transitions other than t which have a net additive effect on p. The *read arc constraint* applies to every transition t which reads from an insufficiently marked place, i.e. $C(p,t) = 0$ and $C^-(p,t) > M_0(p)$:

$$X(t) > 0 \Rightarrow \bigvee_{t' \in F(p,t)} X(t') > 0.$$

The *causality constraint* applies to every insufficiently marked input place p of t, i.e. $C^-(p,t) > M_0(p)$, and additionally requires one of the transitions in $F(p,t)$ to fire *before* t. For this, every transition gets an integer variable $o(t)$ which models its first occurrence in a firing sequence:

$$X(t) > 0 \Rightarrow \bigvee_{t' \in F(p,t)} (X(t') > 0 \wedge o(t') < o(t)).$$

If $F(p,t) = \emptyset$, the disjunction is empty and the constraint becomes $X(t) = 0$.

These two refinements are closely related. The premise of the causality constraint includes read arcs, and its conclusion implies the conclusion of the read arc constraint, so every read arc constraint is implied by a causality constraint. `petrivet` therefore unifies them into a single refinement, called *transition ordering*, which only uses the causality form, and is restricted to pairs (p,t) with $C^-(p,t) > M_0(p)$ for which there is a risk of spurious "borrowing" solutions.

To find these, `petrivet` computes the dependency graph over $P \cup T$ with an edge $p \to t$ for any $p \in P$ and $t \in T$ with $C^-(p,t) > M_0(p)$, i.e. wherever p is insufficiently marked for t in M_0, and all edges $t \to p$ from the original net. A borrowing cycle corresponds to a cycle in this dependency graph. In particular, a read arc appears as a cycle of length two: $p \to t \to p$. Causality constraints are added for exactly those pairs (p,t) where p and t are part of the same strongly connected component of the dependency graph, and ordering variables $o(t)$ are only added for exactly those involved transitions.

Lemma (Transition Ordering Correctness). Together with the marking equation, adding the causality constraints for the dependent pairs (p,t) does not change the set of solutions (M,X).

Proof. Let (p,t) be an edge whose endpoints lie in different strongly connected components of the dependency graph, and let (M,X) be a solution with $X(t) > 0$. Since t and p are not on a common cycle, t does not produce into p, so $C(p,t) = -C^-(p,t) < -M_0(p)$. The marking equation and $M(p) \geq 0$ give $\sum_{t' \neq t} C(p,t')X(t') > 0$, so some transition $t' \in F(p,t)$ has $X(t') > 0$. For the ordering, number the strongly connected components in topological order, and replace every $o(t)$ by its component's number, refined by the old $o(t)$ inside each component. This keeps all constraints within a component satisfied, and for every edge $t' \to p \to t$ between different components gives $o(t') < o(t)$. ■

Unlike in [37], where the constraints are added step by step after each satisfiable query, the transition ordering refinement is added once, together with the marking equation, since it does not depend on any candidate solution. As noted by [37], the result is still an over-approximation: the ordering variables only model the first firing of each transition, and subsequent firings remain subject to the same risk of spurious borrowing.

Trap Becomes Marked.

A trap which is unmarked in M but has some transition from $\langle X \rangle$ in its preset is a contradiction: firing the transitions in $\langle X \rangle$ would have to mark the trap, which then must stay marked. To search for one, `petrivet` computes the maximal trap Q among the places which are unmarked in M and adjacent to U (Section 4.6.1), and checks whether any transition in $\bullet Q$ is in U. If so, the SMT problem is refined with the constraint

$$\sum_{t \in \bullet Q} X(t) > 0 \Rightarrow \sum_{p \in Q} M(p) > 0.$$

This refinement is weaker than it could be. Q is computed as a trap of the whole net N, and the constraint does not depend on X; X only determines which trap is

checked. A stronger variant, suggested by Javier Esparza (personal communication), considers traps of the subnet N_U, which contains all places but only the transitions in U. A trap of N_U only needs to satisfy the trap property for the transitions that are actually fired, so it need not be a trap of N, and there are more candidates for a contradiction. Since Q may now lose tokens through the transitions $L = Q\bullet \setminus \bullet Q$ outside U, the constraint only holds for solutions which fire none of them:

$$\sum_{t\in L} X(t) = 0 \wedge \left(M_0(Q) > 0 \vee \sum_{t\in \bullet Q} X(t) > 0 \right) \Rightarrow \sum_{p\in Q} M(p) > 0.$$

For a trap of N, $L = \emptyset$, so the implemented refinement is the special case of this constraint (with the $M_0(Q) > 0$ case already covered by the Initially Marked Traps refinement). Implementing the subnet variant only requires restricting the trap computation to the transitions in U, and is left to future work.

Realizing the Parikh Vector. If no refinement excludes the candidate solution (M, X), `petrivet` attempts to find a firing sequence σ from M_0 which fires each transition t exactly $X(t)$ times, using a depth-first search over the markings and the remaining budget $X_{\text{budget}}(t) = X(t) - X_{\text{fired}}(t)$ for each transition t. The depth-first search is also reduced with stubborn sets [39]. If it succeeds, the sequence ends in M and M_{target} is reachable/coverable. Otherwise, the search records the dead ends where the least budget remained, and derives *increment constraints* from them following [53]. For each group of places S that blocks the remainder of X_{budget} from firing, it estimates the number of tokens n that the group additionally needs, and requires the transitions $t \in \bullet S \setminus \langle X_{\text{budget}} \rangle$ to fire often enough:

$$\sum_{t:\Delta_S(t)>0} \Delta_S(t) \cdot X(t) \geq n + (\text{tokens already produced along the dead end})$$

$$\text{where } \Delta_S(t) \coloneqq \sum_{p\in S} C(p,t).$$

Increment constraints are not facts about the net: they steer the solver towards a Parikh vector that can be realized, and are only justified for the minimal solutions of Φ, which `petrivet` currently does not enforce. A proof of unsatisfiability which relies on an increment constraint therefore proves nothing, and `petrivet` returns an inconclusive result instead (line 13 of Algorithm 9). This is an incomplete implementation of [53], which iterates minimal solutions and maintains a queue of increment constraints and other so-called *jump constraints* to be added to the SMT problem. However, even that more extensive implementation is not a full decision procedure.

Summary. `petrivet`'s CEGAR engine is a semi-decision procedure for reachability and coverability in both positive and negative cases, providing a witness firing sequence or a set of contradictory lemmas, respectively. Between these two outcomes, the solver may run until its deadline or become otherwise overwhelmed and return an inconclusive result. This is unavoidable to some extent, considering the enormous computational complexity of the problems at hand. But the current implementation also leaves room for improvement: the trap refinement in the behavioral phase does not yet use the subnet induced by the Parikh vector, increment constraints are not yet restricted to minimal solutions, and jump constraints are not yet implemented at all. The effectiveness of the engine is evaluated in Section 5.2.

4.8. Closing Remarks

The dispatch procedures implemented in `petrivet` form the analytical backbone of the library, and are constantly being improved, extended, and re-evaluated. Several known results and procedures are not yet implemented, but would likely improve its performance.

Structural reductions. Reduction rules shrink a Petri net while preserving the property of interest, so that every subsequent procedure operates on a smaller net [57]. [37, Sec. 5] gives reduction rules for deadlock and safety properties, and for free-choice nets, the reduction rules of [30] are complete for live and bounded systems. Applied as a preprocessing step, reductions would benefit every property at once, including the expensive fallback procedures.

Polynomial results for subclasses. For free-choice nets, liveness alone is co-NP-complete [14, Thm. 5.3.11], but simultaneous liveness and boundedness can be decided in polynomial time by the rank theorem [5, Lem. 6.17, Thm. 6.18], [14, Thm. 5.3.16], [28], [30, Cor. 2.8]. Whenever boundedness is already known or also requested, this would replace the Commoner/Hack criterion, whose high theoretical complexity often translates to poor performance in practice (Section 5.3). Reachability also has exact characterizations for subclasses which are largely unexplored in the current implementation: in live marked graphs, a marking is reachable iff it satisfies the marking equation [4, Thm. 19], [14, Thm. 5.2.7], and in live and bounded free-choice nets, iff the marking equation has a solution X such that the subnet N_U induced by $U = \langle X \rangle$ contains no trap unmarked in M_0 [14, Thm. 5.3.19], which is the same condition that underlies the subnet variant of the "Trap Becomes Marked" refinement in the CEGAR engine (Section 4.7.4).

Reusing the CEGAR engine. The CEGAR engine of Section 4.7.4 could answer more than reachability and coverability queries. A transition t can fire at least once iff the marking $M(p) = 1$ for $p \in {\bullet}\, t$ is coverable, so a coverability query

per transition would yield L_0 and L_1 verdicts even for unbounded nets, where `petrivet` currently has no procedure at all. Similarly, generalizing the target from a marking to a linear predicate would allow searching for deadlocks directly, using the target $\bigwedge_{t \in T} \bigvee_{p \in {}^\bullet t} M(p) < C^-(p,t)$, which would provide the safe procedure for deadlock-freedom of unbounded nets that is currently missing. The engine itself would also benefit from the extensions discussed at the end of Section 4.7.4: the subnet variant of the trap refinement, and minimal solutions with jump constraints [53].

State space exploration. `petrivet` currently does not cache state spaces. This particularly disastrous for Petri nets with less restricted structure and large state spaces, where the coverability graph fallback is activated for the majority of queries and has to be restarted from scratch each time. Furthermore, the set of reachable markings cannot be constructed without also constructing the graph structure, which is unnecessary for queries such as k-boundedness and deadlock-freedom. Beyond this, the exploration itself could be reduced: stubborn sets preserve all deadlocks of the full state space [39], and are already computed by the guided exploration of the CEGAR engine, but not used by the general state space explorer. For coverability, minimal coverability set algorithms [58], [59], [60] avoid much of the redundancy and ω-acceleration cost of the Karp-Miller graph, and backward search algorithms [61] or IC3 [62], [63] offer complete alternatives to it. For known bounded systems, the fallback procedure could construct the reachability graph instead of the coverability graph, which would avoid the overhead of ω-acceleration where there is no risk of non-termination. Finally, symbolic representations of the state space, such as decision diagrams [38], [64] or partial-order representations such as unfoldings [65] are established alternatives to explicit exploration for large bounded systems.

5. Evaluation

This chapter assesses the extent to which `petrivet` fulfills the objectives established in the introduction. The requirements derived from the chair's needs are listed in Section 1.1, and the survey in Section 2 applies them to existing tools. Here I ask whether the library gives correct verdicts, whether its central design decisions pay off, and how it compares with established tools. Parts of the evaluation are measured, and parts are assessed against the requirements by inspection and demonstration. The research questions that concern the artifact are the following (RQ1 is answered by the survey in Section 2):

- **RQ2.** Does the implementation give correct answers, and does it return "unknown" where it cannot decide? (Section 5.2)
- **RQ3.** How much does answering each question with the cheapest applicable procedure, instead of one general procedure, reduce the cost of analysis? (Section 5.3)
- **RQ4.** Can a caller effectively bound the cost of an analysis, either by restricting the effort or by setting a deadline? (Section 5.4)
- **RQ5.** How does `petrivet` compare with established tools in speed and memory? (Section 5.5)

A hypothesis for each question was established before the evaluation began except the comparison with established tools, which is descriptive. The requirements that cannot be reduced to a measurement, such as the availability of a programmatic interface or the support for standard model formats, are assessed in Section 5.6, using the same rubric as the survey. Section 5.7 collects the threats to validity of all parts.

5.1. Method

This section describes the methodology by which the evaluation was conducted.

5.1.1. Properties

The measured experiments draw their queries from a common set of nine Petri net properties. They all take a Petri net (N, M_0) as input, and all except state space size return a boolean verdict. Two (reachability and coverability) additionally take a target marking as input.

The properties are listed here:

- Deadlock-free: Is (N, M_0) deadlock-free?
- Bounded: Is (N, M_0) bounded?
- (1-)Safe: Is (N, M_0) 1-bounded?
- Live: Is (N, M_0) L_4-live?
- Quasi-live: Is (N, M_0) L_1-live?
- Reachable: Is M reachable in (N, M_0)?

- Coverable: Is M coverable in (N, M_0)?
- State space size: How many nodes and edges (markings and transition firings) are in the reachability graph of (N, M_0)?
- Stable place: Does (N, M_0) contain a place whose marking is constant in all reachable markings?

5.1.2. Net Collections

Four sources of Petri nets were used in the evaluation. Results are reported separately for each collection because they have different characteristics and are used for different purposes. Table 2 summarizes the four collections, their sizes, the source of the reference verdicts, and the purposes for which they are used.

Collection	**Size**	**Reference verdicts from**	**Used for**
Calibration nets	30	Worked out by hand	Validating the evaluation setup, correctness, cost control
Randomized nets	1,696	Agreement of two reference tools	Correctness
Parametrized nets	146 nets across 15 families	By construction	Correctness, dispatching, performance and memory comparison, cost control
2026 MCC[70] models	90 small, 20 large	Published verdicts[71]	Correctness, dispatching, cost control, performance and memory comparison

Table 2: The four collections of nets used in the evaluation.

Calibration nets. Thirty small Petri nets were constructed and analyzed by hand. They serve two purposes: to validate the measurement setup for the various tools,

[70] https://mcc.lip6.fr/

[71] https://yanntm.github.io/pnmcc-models-2026

and to provide verdicts which do not depend on any software. Figure 27 shows one of them.

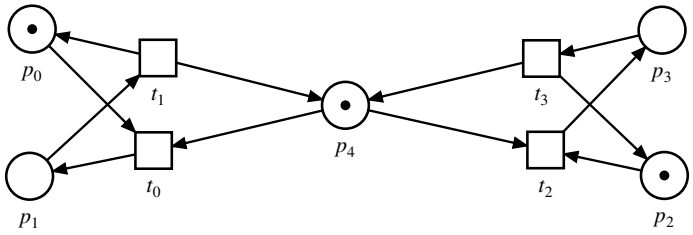


Figure 27: The calibration net `cal-22-ac-mutex`: two processes, each with an idle place (p_0, p_2) and a critical section (p_1, p_3), share the lock p_4. It has three reachable markings, and both processes are never in their critical section at the same time.

Randomized nets. A pseudorandom number generator was used to generate 100 nets each across six structural subclasses (circuits, state machines, marked graphs, extended free-choice nets, asymmetric-choice nets, and general nets) (Section 1.5.5.3), and three size ranges (2 to 5 places, 6 to 8 places, 9 to 12 places). This produced $100 \cdot 6 \cdot 3 = 1,800$ nets. Duplicates were identified by their content hash and pruned, and a few combinations of structural class and size range yielded fewer than 100 nets, leaving the final count at 1696. The generators for each structural class occasionally produced nets which also belonged to a more specific class, and in these cases the net was classified into the most specific class. The RNG was seeded, so the collection can be reproduced exactly, and the files are archived together with a checksum.

For each generated net, up to four target markings were additionally generated by a seeded random process for the reachability and coverability properties. Three strategies were used: the end markings of random firing sequences (expected reachable) those markings with a few tokens removed (expected coverable), or with one place raised above any value seen during the random walks (expected uncoverable), and random markings with at most two tokens more per place than the initial marking (expected unreachable). Figure 28 shows one of the generated nets.

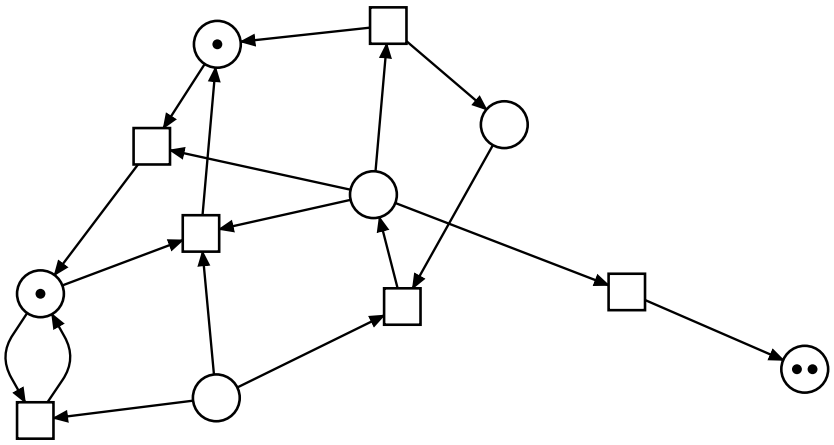
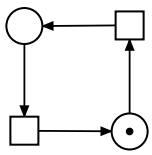
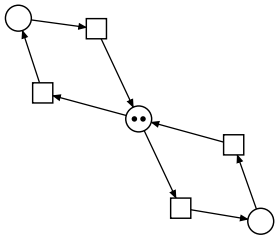
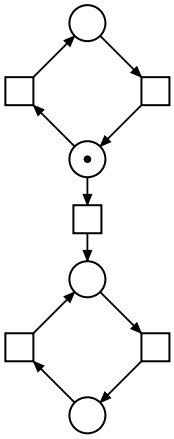
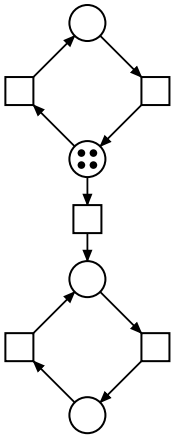
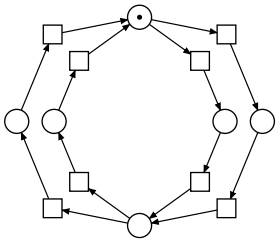
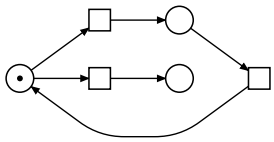
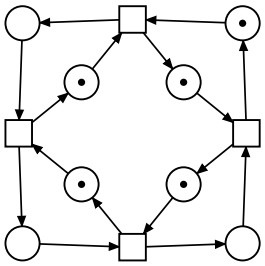
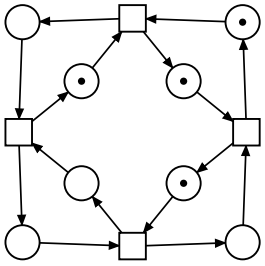
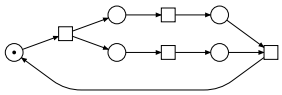
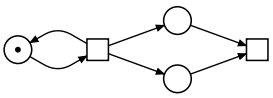
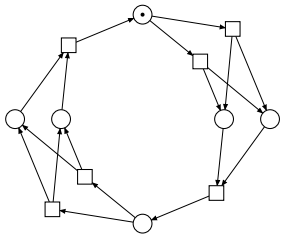


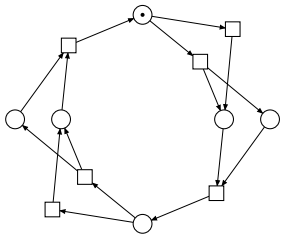
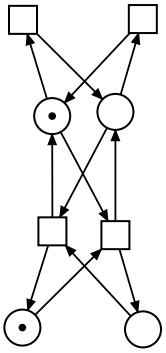
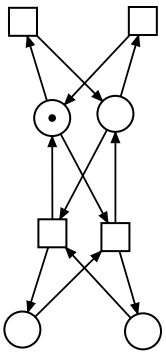
Figure 28: A randomized general net of medium size
(6 places, 6 transitions, 18 arcs, 4 tokens).

Parametrized nets. Fifteen families of parametrized nets were constructed, each following a specific pattern and parametrized by a size n. The properties boundedness, deadlock-freedom, liveness, and quasi-liveness are known for each family by construction. These families are used to evaluate how running time grows with the size of the net. Table 3 lists the families, the properties they satisfy, and a small example of each.

Family	**Description**	**DF**	**B**	**L**	**QL**	**Example**
circuit	A simple ring of n places and n transitions with one token.	yes	yes	yes	yes	
sm-hub-and-spokes	A state machine with one central place and n spokes, with n tokens.	yes	yes	yes	yes	

Family	Description	DF	B	L	QL	Example
sm-two-cycles	Two disjoint rings of $\frac{n}{2}$ places each, joined by a one-way transition, with one token.	yes	yes	no	yes	
sm-two-cycles-n-tokens	Like sm-two-cycles, but with n tokens in the first cycle.	yes	yes	no	yes	
sm-choice-ring	A ring of n sequential two-way decisions with a single token.	yes	yes	yes	yes	
sm-ring-with-sink	A ring of size n with $\frac{n}{2}$ tokens, plus a dead-end.	no	yes	no	yes	

Family	Description	DF	B	L	QL	Example
mg-buffer-ring	An outer counter-clockwise ring with one token and an inner clockwise ring using the same n transitions.	yes	yes	yes	yes	
mg-buffer-ring-unmarked-circuit	Like mg-buffer-ring, but containing an unmarked circuit.	no	yes	no	no	
mg-forkjoin-cycle	n concurrent branches of two places each.	yes	yes	yes	yes	
mg-forkjoin-unbounded	A transition with a self-loop on a marked place puts a token onto n places, and a sink transition consumes the tokens.	yes	no	yes	yes	
fc-forkjoin	A ring of n sequential two-way parallel decisions with $\frac{n}{2}$ tokens.	yes	yes	yes	yes	

Family	Description	DF	B	L	QL	Example
fc-forkjoin-leaky	Like fc-forkjoin, except that one of the choices does not produce enough tokens to continue.	no	yes	no	yes	
ac-cycle	*n* conjoined segments with a high place and a low place each. The high tokens can move independently; the low tokens can only move together with the high tokens. Contains one low and one high token.	yes	yes	yes	yes	
ac-cycle-no-low-token	Like ac-cycle, but without the low token.	yes	yes	no	no	

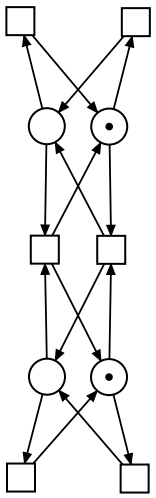
Family	Description	DF	B	L	QL	Example
general-cycle	Like ac-cycle, but the low tokens can also move independently.	yes	yes	yes	yes	

Table 3: The fifteen families of parametrized nets used in the evaluation, with their properties and a small example.
sm = state machine, mg = marked graph, fc = free-choice, ac = asym. choice.
DF = deadlock-free, B = bounded, L = live (L_4), QL = quasi-live (L_1).

The generated sizes are the powers of two from 2 to 1,024 (from 4 for `sm-two-cycles`, `sm-two-cycles-n-tokens`, `mg-buffer-ring` and `mg-buffer-ring-unmarked-circuit`). The largest parametrized net used in the evaluation (`sm-choice-ring` with $n = 1,024$) has 3,072 places and 4,096 transitions.

MCC models. From the model repository of the 2026 Model Checking Contest[72] I drew 90 “small” and 20 “large” non-colored Petri nets by a fixed rule (for the small models, the model nearest the median state-space size of its structural class, at most two per class; for the large models, five per examination, one per class), where a small net is one whose state space is known to be at most 10^6 markings, and a large net is one whose state space is larger than that or unknown.

MCC models which `petrivet` failed to parse, or which exceeded a file-size cap, were skipped: 22 small and 5 large non-connected nets, 22 small and 9 large nets which contained nonstandard ($\neq 1$) arc weights, and 15 large nets whose file size exceeded 5 MB.

Figure 29 and Figure 30 show examples of a small and a large MCC model, respectively.

[72] https://mcc.lip6.fr/models.php

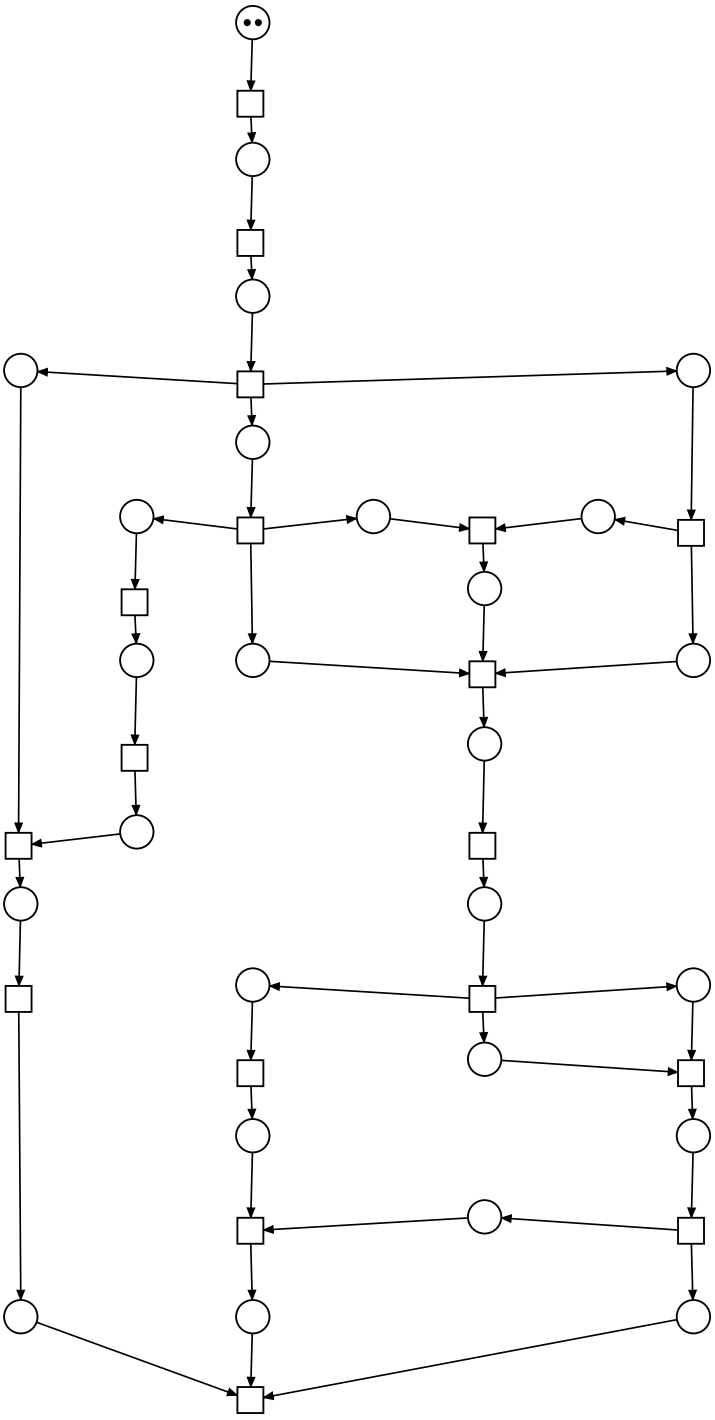


Figure 29: The small MCC model `HouseConstruction-PT-00002` (26 places, 18 transitions, 1,501 reachable markings).

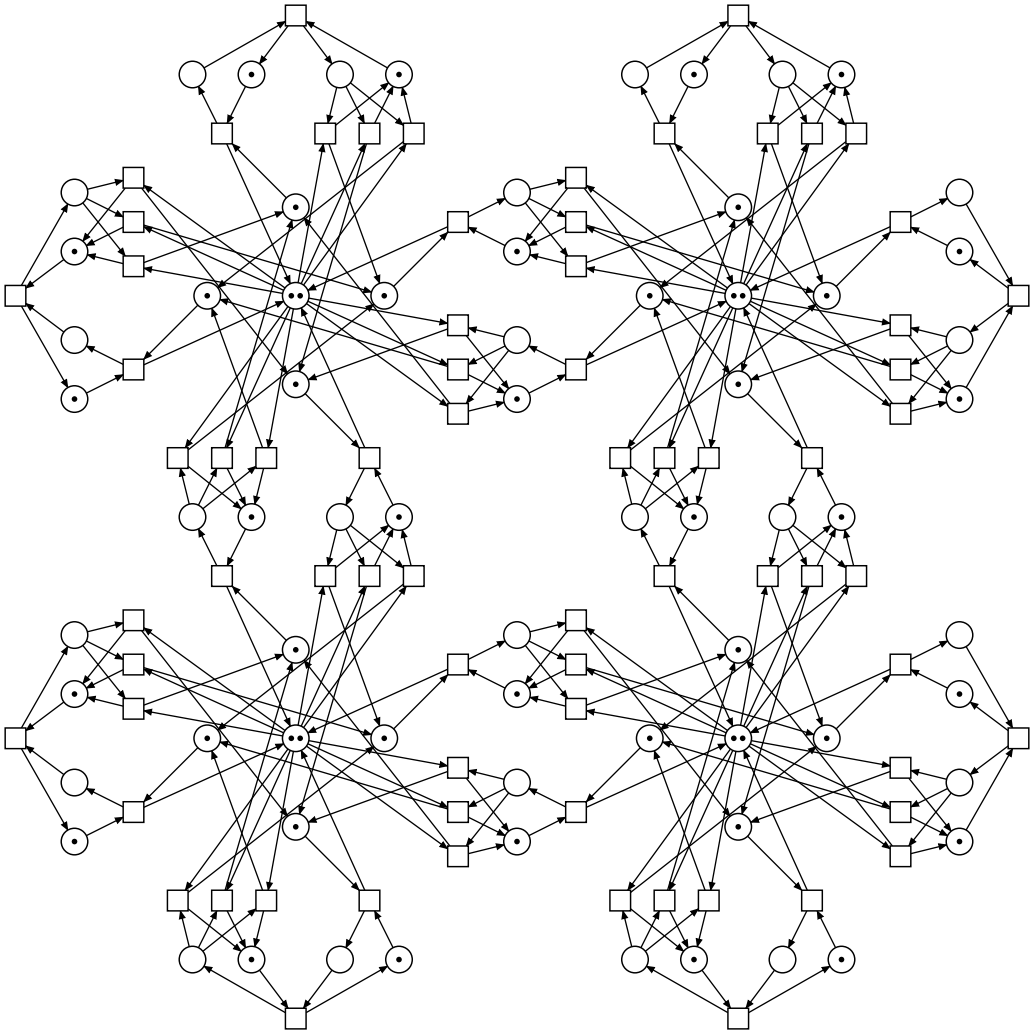


Figure 30: The large MCC model `SquareGrid-PT-020102` (68 places, 72 transitions, about $2.1 \cdot 10^{12}$ reachable markings).

The largest (by number of places and transitions) "small" MCC models used in the evaluation were `ServersAndClients-PT-200080` (16,881 places, 32,400 transitions, but only 16,401 reachable markings) and `RERS2020-PT-pb101` (906 places, 149,363 transitions, and 345,421 reachable markings).

The largest "large" MCC models used in the evaluation, `FamilyReunion-PT-L00050M0005C002P002G001` (12,194 places, 10,560 transitions) and `DLCflexbar-PT-6a` (2,069 places, 16,077 transitions), both have an unknown number of reachable markings: no participant tool in the 2026 MCC was able to compute the state space size of either model. They are suspected to be bounded. The large MCC model with the largest known state space used in the evaluation is `BusinessProcesses-PT-09` (403 places, 374 transitions, roughly $7.5 \cdot 10^{18}$

reachable markings). This shows that net size is a poor predictor of state space size, and that the evaluation includes a wide range of net sizes and state space sizes.

5.1.3. Tools under Comparison

`petrivet` enters the evaluation in three different configurations:

1. In the default **automatic dispatch** configuration, `petrivet` answers the query using its normal internal dispatching logic. This is what a user of the library would experience when using the library without utilizing any cost-control mechanism.
2. In the **efficient methods only** configuration, `petrivet` must answer the query using only polynomial-time procedures (if possible). This is equivalent to automatic dispatch with the efficiency cost-control mechanism enabled.
3. In the **brute-force** configuration, `petrivet` must answer the query via a breadth-first search of the reachability graph, which isolates the effect of the internal dispatching. If the property can be decided on a partially explored graph, then the search is terminated early; otherwise the search continues until the entire reachability graph is explored or a timeout occurs. For boundedness and coverability, the search runs on the Karp-Miller coverability graph instead, which is finite also for unbounded nets. As discussed in Section 1.5.4.2, the reachability graph is an infinitely large structure for unbounded Petri nets, which is why `petrivet` uses this method by default only where no other procedure applies: as the last-resort for deadlock-freedom, and for state space size and stable place.

A selection of three external tools from the survey in Section 2, LoLA 2.0[73] , TINA 4.0.0[74] , and ITS-Tools (commit 8a49519)[75] also enter the evaluation for comparison.

Every tool was wrapped with an adapter which translates the evaluation query into the tool's own input format, and translates the tool's output back. Every adapter was validated against the calibration nets to ensure correct translation and invocation of the tool. Where a tool does not directly support a property (for example, LoLA has no direct API for the liveness of a whole net), the adapter combines several smaller queries to produce the verdict. These are described in the appendix. TODO reference

All tools run as separate processes, under the same time and memory limits, on the same machine (see Section 5.1.5). Using a command-line interface for all tools places them on equal footing, including process start-up overhead. The

[73] https://theo.informatik.uni-rostock.de/theo-forschung/tools/lola/
[74] https://projects.laas.fr/tina/index.php
[75] https://lip6.github.io/ITSTools-web/

start-up time of each tool is measured on a trivial net and reported in the results, so that the overhead can be subtracted from the measured times.

5.1.4. Reference Verdicts

For the authoritative reference verdicts, I used a different strategy for each of the four collections. The calibration nets were analyzed by hand (and cross-checked by all tools during the evaluation), the randomized nets (and their associated target markings) were analyzed by LoLA and TINA for consensus, the parametrized nets were constructed with the verdicts known by design, and the MCC model verdicts were taken from the official oracle[76].

ITS-Tools is not used as a correctness reference for the randomized nets due to difficulties supporting all of the properties. It decides five of the six properties which overlap with the MCC's examinations (state space size, deadlock-freedom, quasi-liveness, 1-safety, and stable place); the others are either not supported or cannot be answered in time: for liveness it spends a fixed 30 seconds on an unknown internal procedure (which invariably consumes the entire evaluation time limit)[77]. For the state space size property it only reports the number of markings and not the number of transition firings. This second limitation is a fundamental design trade-off: while `petrivet`, LoLA and TINA are all explicit state model checkers which fire individual transitions and evaluate the resulting markings, ITS-Tools is a symbolic model checker which constructs the state space indirectly using hierarchical set decision diagrams (SDDs) and tailors the marking representation to exploit the structure of each individual net [38], [66], [67]. This gives it a flatter performance curve particularly for large nets, but it also means that it cannot know the number of individual transition firings, because it never fires them individually.

For the randomized nets, a disagreement or timeout from either reference tool was treated as a failure to produce an authoritative verdict, and the query was excluded from the correctness evaluation. These exclusions happened before any `petrivet` runs, so that `petrivet`'s verdicts could not influence which Petri nets entered the correctness evaluation.

For unbounded Petri nets, the reference tools gave particularly unreliable verdicts (or often no verdict at all) for the properties liveness, quasi-liveness, and deadlock-freedom. For this reason, the correctness evaluation is restricted to bounded nets for these three properties. The unbounded results are nonetheless

[76] https://yanntm.github.io/pnmcc-models-2026

[77] ITS-Tools shows the same fixed wait on deadlock-freedom: of the 57 deadlock-freedom queries that it did not answer within 30 seconds, 53 of them are answered between 30 and 35 seconds, and 55 are answered within 90 seconds (median 31.2 s). Its deadlock-freedom results are therefore limited by the evaluation's 30 s time limit, and not by the difficulty of the model.

reported where available for completeness, but they do not influence the outcome of the evaluation.

Furthermore, for the reachability and coverability properties, TINA timed out on every unbounded calibration net. For this reason, and because I saw no reason to doubt the correctness of LoLA's verdicts, I did not run TINA for these two properties on the unbounded randomized nets, and used LoLA's verdicts alone as the reference verdict for those nets. On bounded nets, a TINA timeout excluded the query as described above.

5.1.5. Measurement Setup

Measurements were conducted on a single desktop machine with a four-core, eight-thread Intel i7-4790K CPU @ 4.00GHz and 32 GiB of RAM. At most three runs were executed at a time; each timing and memory run was pinned to a unique physical CPU core with the sibling hyperthread left idle. The frequency governor was fixed at "performance" and turbo frequency boosting was switched off to avoid variability in clock speed. Graphical sessions, file synchronization and other background services were stopped for the measured runs.

Each run of `petrivet` and of the compared tools has a wall-clock time limit of 30 seconds, enforced outside the process, and a memory limit of 8 GiB. The labelling runs of LoLA and TINA for the reference verdicts of the randomized nets have a longer time limit of 60 seconds and no memory limit. A run that exceeds either limit is recorded as a timeout or memory-out, and the process is killed. A run that ends abnormally, through a crash, abort, or other error, is recorded under its own category. A timing query is run five times if its first run takes less than 1 second, three times if it takes less than 10 seconds, and once otherwise; a run that times out is not repeated. This rule was fixed in a short pilot study before the measured runs began. Settings, tool versions, and processor configuration are recorded with every run except the reference-labelling runs of LoLA and TINA.

Peak memory is measured in a second pass over the runs which produced a verdict, using a separate low-overhead wrapper that records the peak resident set size (RSS) of the process. The separate low-overhead wrapper served to lower the RSS measurement floor as far as possible, because the wrapper's own memory use is included in the peak measurement of the tool process. This measurement floor is about 7.4 MiB. For a tool which spawns child processes of its own, such as ITS-Tools, the measured value is that of the largest single process, and not the sum. For a run which times out, the second pass gives no peak memory value; the figures then show the peak memory recorded by the timing run at the moment it was stopped, which is a lower bound.

5.1.6. Analysis

Every table and figure below is generated from the raw measurement data by a Python script. Timings are summarized by medians and percentiles. For head-to-head comparisons of running times I use the one-sided paired Wilcoxon signed-rank test [68] at significance level 0.05, which makes no assumption about the distribution of times, with Holm's correction [69] across the structural classes. Timeouts are treated as censored data: their number is reported, and the comparison is repeated without them to see if the conclusion is robust to their presence. For the tool comparisons, I report the number of solved instances, the penalized average runtime where a failed run counts as twice the time limit, and cactus plots. Where no disagreement with the reference verdict is found, I provide the one-sided 95 percent upper bound on the true rate of disagreement (Clopper-Pearson interval [70]).

Terminology. The following sections use the following terminology:

- A **property** is a Petri net property as described in Section 5.1.1.
- A **collection** is a set of Petri nets as described in Section 5.1.2.
- A **query** is a combination of a Petri net and a property (and, for reachability and coverability, a target marking).
- A **run** is a single invocation of a tool on a query.
- A **verdict** is a definite answer (yes or no, or for state space size the two counts) produced by a run. A verdict may be correct or incorrect with respect to the reference verdict. A run may also produce no verdict, because it timed out, ran out of memory, crashed, or answered "unknown".

5.2. Correctness

RQ2 asks whether `petrivet` ever produces an incorrect verdict. The hypothesis is that it does not, on any model, for any property, from any procedure. "Unknown" is a legitimate response and counts as neither correct nor incorrect. All three operating modes of `petrivet` (per Section 5.1.3) are evaluated; disagreement between them is a correctness violation. The section first reports how many total cases could be compared and how many were excluded, then presents the agreement rate and the upper bound on the disagreement rate.

Table 4 shows, per net collection, how many verdicts of each of the three operating modes of `petrivet` could be compared with a reference verdict. Over all collections and modes, 40,200 runs could be compared, and 0 of them disagreed with the reference verdict. In addition, 793 queries on the randomized nets were excluded before any `petrivet` run, because the reference tools gave no authoritative verdict for them (see Section 5.1.4). The statistical strength of this result depends on the net collection. For `petrivet`'s automatic dispatch operating mode, the one-sided 95 percent upper bound on the rate of disagreement is

0.023% on the randomized nets (the two-tool group), 0.13% on the group confirmed by LoLA alone, 0.59% on the parametrized families and 0.91% on the small MCC models. On the large MCC models the automatic mode gave a verdict in only 16 of 80 runs, so the bound is 17% and the large models say little about correctness.

Collection	Automatic dispatch	Brute-force	Efficient methods only	Disagree-ments	Upper bound (dispatch)	No verdict (dispatch)
Randomized nets (LoLA/TINA consensus)	13,246	13,270	7,116	0	0.02%	0.2%
Randomized nets (LoLA verdict only)	2,237	1,840	134	0	0.13%	1.5%
Parametrized families	506	390	431	0	0.59%	9.8%
MCC models, small	327	532	37	0	0.91%	39.3%
MCC models, large	16	11	11	0	17.08%	80.0%
Calibration nets	32	32	32	0	8.94%	0.0%

Table 4: Verdicts of the three modes of `petrivet` that could be compared with a reference verdict, per collection, the disagreements, the one-sided 95 percent upper bound on the disagreement rate of the automatic mode, and the share of runs without a verdict.

For the automatic dispatch mode with the small MCC models, 180 of the runs concern the two properties for which it has no procedure other than brute-force exploration (state space size and stable place), and most of the rest are timeouts or memory-outs due to the library’s defensive construction of the coverability graph instead of the faster to construct, but potentially infinite, reachability graph.

For reachability and coverability on the randomized nets, Table 5 separates the cases by the verdict and by the kind of reference. The group confirmed by LoLA alone is skewed: for coverability, 1,081 of its verdicts are positive and only 166 negative, because a reachable or coverable target of an unbounded net is much easier to find than an unreachable one is to refute. No verdict of either group contradicts the reference.

Property	Reference	Verdict	Compared	Disagree-ments	Upper bound	No verdict
reachability	LoLA and TINA agree	reachable	1,599	0	0.19%	2
reachability	LoLA and TINA agree	not reachable	2,610	0	0.11%	2
reachability	LoLA alone	reachable	559	0	0.53%	8
reachability	LoLA alone	not reachable	431	0	0.69%	0
coverability	LoLA and TINA agree	coverable	1,689	0	0.18%	10
coverability	LoLA and TINA agree	not coverable	1,729	0	0.17%	10
coverability	LoLA alone	coverable	1,081	0	0.28%	26
coverability	LoLA alone	not coverable	166	0	1.79%	0

Table 5: Reachability and coverability on the randomized nets: verdicts of `petrivet` dispatch against the reference. “No verdict” counts the runs with a reference verdict for which `petrivet` returned no verdict.

ITS-Tools, which was not used as a reference, was checked against the same reference verdicts. In the collections where it ran, 670 of its verdicts could be compared, and 0 of them contradict the reference. On nets that are unbounded, which are outside the scope of the comparison, it decided deadlock-freedom for 10 nets, and every one of these verdicts contradicts the construction truth: it reports a reachable deadlock where there is none. That is an observation about the tool, not about the comparison.

The automatic dispatch mode of `petrivet` was also checked for cache-coherence. `petrivet` caches intermediate results about deadlock-freedom, boundedness, and liveness properties, which represents a potential source of error: if the cache is not coherent, the verdict to one property may vary depending on which properties were decided before. On 209 randomized nets, the properties were decided in several orders, and every verdict was compared with the verdict to the same property on a fresh `PetriNet` and, where the reachability graph is small, with an explicit-state computation. There were 0 differences (upper bound 1.4%).

No verdict of the three modes of `petrivet` contradicts a reference verdict, so the data support the hypothesis of RQ2 within the upper bounds given above.

Bugs discovered during the evaluation. Three bugs were discovered and fixed during the correctness evaluation, which is a good sign that the evaluation is meaningful.

In the first runs of the randomized nets, the automatic dispatch mode of `petrivet` answered nine reachability and coverability queries incorrectly: it falsely proved targets unreachable/uncoverable where they were in fact reachable/coverable. Eight of the nine runs had a reference verdict (five confirmed by both tools, three by LoLA alone) and one had none; all but one net belonged to the general class. The cause was a misinterpretation of “increment constraints” as described by

[53]: added to the SMT problem to steer the search for a realizable firing sequence, they are not a fact about the net but a navigation step that can be helpful in finding a solution. An SMT solver may return UNSAT in the presence of an increment constraint even if the target is reachable/coverable, but the library trusted that as a definite refutation. The library now reports “unknown” for such a refutation; the 5,315 negative verdicts of the automatic dispatch mode for these properties were removed and run again, and no disagreement remains: the nine problematic runs are now “unknown”, and the other 5,306 runs are unchanged.

The second discovered bug was a stack overflow in a dependency during reachability graph analysis. `petrivet` computes the strongly connected components of the reachability graph for liveness, using Tarjan’s algorithm implemented in the `petgraph` crate[78]. This implementation is recursive, and overflows the stack on reachability graphs of roughly 10^5 states or more. Switching to an alternative iterative algorithm (Kosaraju’s[79]) fixed the problem.

The third bug was a case in which the evaluation harness itself misled the evaluation: the adapter of one reference tool read output that had been truncated due to excessive length, and interpreted an intermediate message as the final verdict. Two disagreements with the reference verdicts revealed this defect; the adapter was corrected and the affected runs were repeated.

None of these three bugs is visible in the final data. They are listed here solely for transparency and to illustrate the value of the evaluation.

5.3. Benefit of Procedure Dispatch

RQ3 asks whether `petrivet`’s internal dispatch of procedures, instead of always using one general procedure, reduces the cost of analysis. The hypothesis is that, for nets of restricted structural class, `petrivet`’s automatic dispatch is cheaper than the brute-force configuration, with a positive paired effect after correction for the number of structural classes tested. The experiment uses the parametrized families of nets, which are constructed to be of a specific structural class and have a high complexity ceiling. The randomized nets are not used because they are too small for reliable timings. The MCC models are used as well, although on many of them dispatch cannot select a specialized procedure over the general approach. Two of the nine properties (state space size and stable place) are only implemented in `petrivet` via state space exploration, and are therefore excluded from this experiment.

Method. Every query is run twice: once with `petrivet`’s automatic dispatch and once with brute-force. The time taken by the two runs is compared. Runs which

[78] https://docs.rs/petgraph/0.8.3/petgraph/algo/scc/tarjan_scc/index.html
[79] https://docs.rs/petgraph/0.8.3/petgraph/algo/scc/kosaraju_scc/index.html

time out at 30 seconds or exhaust the memory limit are counted as 30 seconds, which is an underestimation of the true time. The comparisons are therefore additionally repeated excluding the runs that timed out or ran out of memory in either mode.

Results on the parametrized families. Table 6 shows a summary of which mode is faster for each structural class and the associated p-value after Holm-Bonferroni correction for the number of classes tested (one-sided Wilcoxon signed-rank test; a class counts as faster when the corrected p-value is below 0.05) The automatic dispatch is clearly faster than brute-force for circuit, state machine and marked graph nets, and this holds with and without the timed-out runs. For extended free-choice nets, the automatic dispatch is faster only when the timed-out runs are counted, and brute-force is generally faster on the queries that both modes finish. For asymmetric-choice and general nets the automatic dispatch is not faster.

Class	Paired runs	One or both runs timed out or ran out of memory	p-value	Faster	Faster without timeouts
circuit	40	0	< 0.001	yes	yes
state machine	188	80	< 0.001	yes	yes
marked graph	148	55	< 0.001	yes	yes
extended free-choice	80	54	0.002	yes	no
asymmetric-choice	80	19	1.000	no	no
general	40	12	1.000	no	no

Table 6: Performance summary of automatic dispatch against brute-force on the parametrized families, by structural class.

Performance by size. Figure 31 shows the analysis time taken by dispatch and brute-force for every property on each parametrized family as the net grows in size. Outlined symbols indicate that the run timed out at that time value, and blackened symbols indicate that the run ran out of memory. Queries where neither dispatch nor brute force returned a verdict say nothing about the difference and are left out of the comparison. The performance ratio is the time of the brute-force divided by the time of the automatic dispatch: greater than 1 means that the automatic dispatch is faster, and less than 1 means that it is slower. Four patterns show:

- For `circuit`, `sm-choice-ring`, `sm-two-cycles`, `mg-buffer-ring` and `mg-buffer-ring-unmarked-circuit`, the performance ratio is constant and modest, between about 1 and 2 at every size. The dispatch helps marginally, but the number of reachable markings is small enough that brute-force is not a problem.

- For families with exponentially growing state spaces (`sm-hub-and-spokes`, `sm-ring-with-sink`, `sm-two-cycles-n-tokens`, and `mg-forkjoin-cycle`), the performance ratio grows significantly with the size of the net, and from size 16 or 32, brute-force times out on most or all questions while the automatic dispatch still answers in milliseconds. The ratio is at least several hundred. `mg-forkjoin-unbounded` is a special case: brute-force hopelessly enumerates an infinite state space and never reaches a conclusion on deadlock-freedom, liveness or quasi-liveness (it answers boundedness within milliseconds).
- For extended free-choice nets, the dispatch is about even with brute-force at small sizes, until brute-force begins to time out, and then the dispatch is faster until it also begins to time out.
- For asymmetric-choice and general nets, the median performance ratio over the properties is less than one as the nets grow, down to a ratio of about 0.004 for `ac-cycle` at size 64 (the automatic dispatch takes about 280 times as long), and 0.33 for `general-cycle`. The slowdown is consistent with the nontrivial cost of the Commoner/Hack criterion and the Karp-Miller coverability graph, which are used by the automatic dispatch on these nets. Only on boundedness, from size 256 for `ac-cycle` and from size 128 for `general-cycle`, does the automatic dispatch return a verdict where brute force times out.

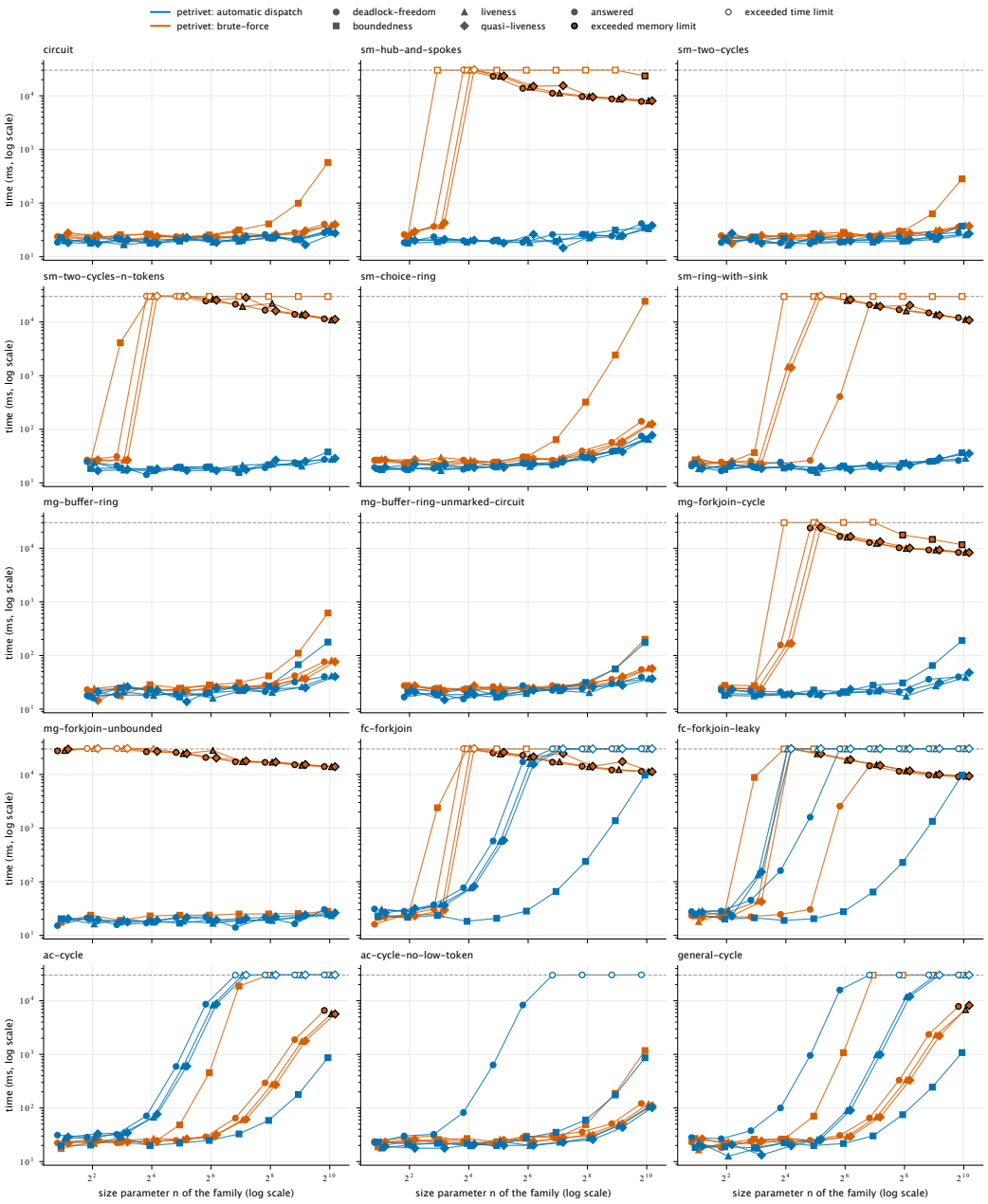


Figure 31: Analysis time of automatic dispatch (blue) against brute-force (orange) per parametrized family. Net size is on the x-axis, and time in milliseconds is on the y-axis (log scale, lower is better). The shape of the symbol indicates the property being checked, an outlined symbol means that the run timed out at that time value, and a blackened symbol means that the run ran out of memory. Symbols of different properties are offset slightly on the x-axis for visibility.

Figure 32 illustrates every query as a dot in a scatter plot, with the time taken by automatic dispatch on the x-axis and the time taken by brute-force on the y-axis.

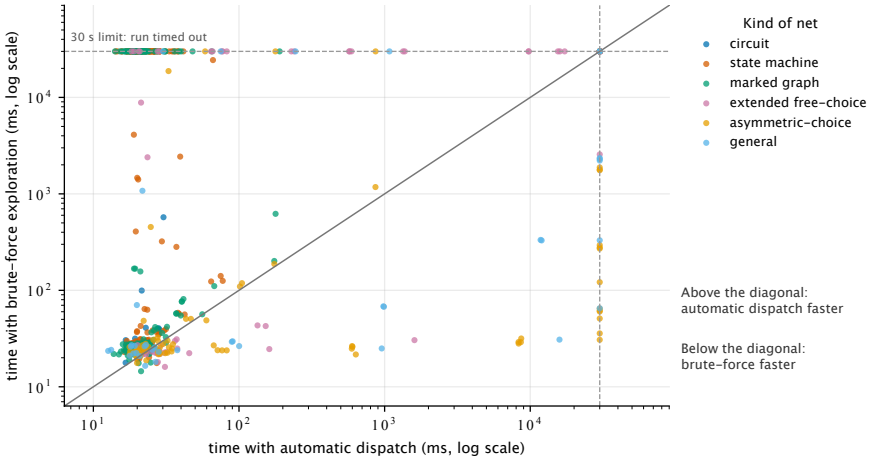


Figure 32: Every dot is a pair of runs: the time of the automatic dispatch (x-axis) against the time of brute-force (y-axis), colored by family.

The tail of queries under the diagonal, primarily of the asymmetric-choice and general classes, is where the automatic dispatch is currently slower than brute force: this is consistent with the overhead of the Commoner/Hack criterion and the Karp-Miller coverability graph.

How large a net can be handled. Table 7 shows the largest size (n) of each family that a mode answers every query for within 30 seconds. The automatic dispatch handles larger nets than brute-force in 6 of the 15 families, and brute-force handles larger nets in 2 families. In the remaining families both modes reach the same size, and in 5 of those it is the largest size that was tried, 1,024. Note that `mg-forkjoin-unbounded` is unbounded, so the brute-force was unable to finish at any size, while the automatic dispatch answered all queries at size 1,024.

Family	Largest size without timeouts for automatic dispatch	Largest size without timeouts for brute-force	Largest size without a timeout in either mode	Performance ratio at that size
`circuit`	1,024	1,024	1,024	1.44
`sm-hub-and-spokes`	1,024	4	4	1.38
`sm-two-cycles`	1,024	1,024	1,024	1.44
`sm-two-cycles-n-tokens`	1,024	8	8	1.51
`sm-choice-ring`	1,024	1,024	1,024	1.90
`sm-ring-with-sink`	1,024	8	8	1.21
`mg-buffer-ring`	1,024	1,024	1,024	1.92
`mg-buffer-ring-unmarked-circuit`	1,024	1,024	1,024	1.46
`mg-forkjoin-cycle`	1,024	8	8	1.25
`mg-forkjoin-unbounded`	1,024	–	–	–
`fc-forkjoin`	64	8	8	0.83
`fc-forkjoin-leaky`	8	8	8	0.41
`ac-cycle`	64	128	64	0.00
`ac-cycle-no-low-token`	64	1,024	64	1.12
`general-cycle`	64	64	64	0.33

Table 7: The largest size parameters that each mode answers every query for without timing out, and the performance ratio of the two modes at the highest size where neither mode times out. A dash means that there is no such size.

Memory usage. Figure 33 shows the peak memory of both modes for every family and size.

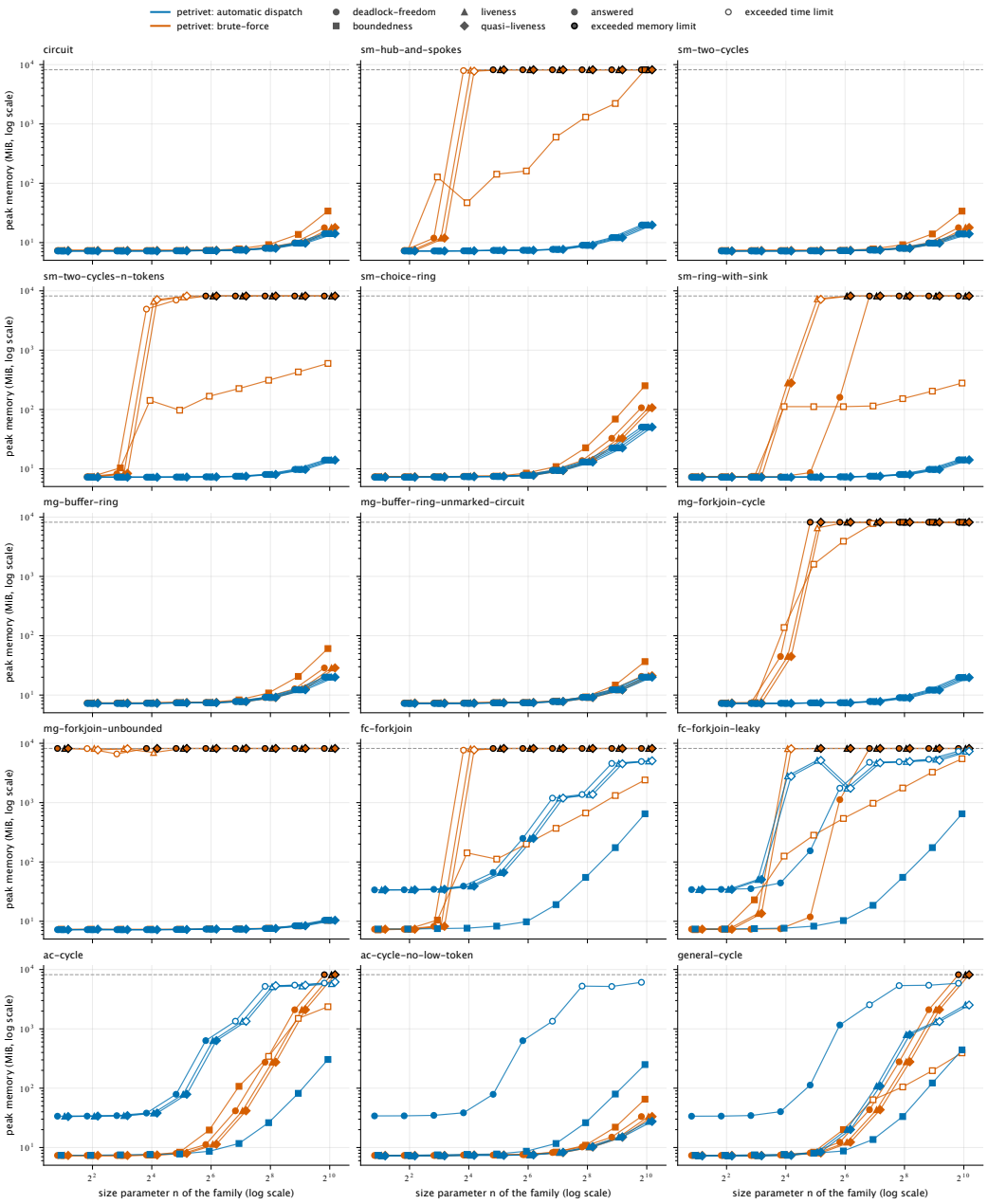


Figure 33: Peak memory of the automatic dispatch (blue) and brute-force exploration (orange) per family. The net size is on the x-axis, and the peak memory in MiB is on the y-axis (lower is better). The shape of the symbol indicates the property being checked, an outlined symbol means that the run timed out at that memory value, and a blackened symbol means that the run ran out of memory. Symbols of different properties are offset slightly on the x-axis for visibility.

For deadlock-freedom, liveness, and quasi-liveness queries in the free-choice and asymmetric-choice families, and for deadlock-freedom in the one general family, a significant increase in memory usage starting at the smallest size is immediately apparent. This is consistent with the overhead of the Commoner/Hack criterion, which is used in exactly these cases. The Commoner/Hack criterion is therefore a likely cause of the reduced performance and increased memory usage of the automatic dispatch on these families.

The competition models. Unlike the parametrized families, the MCC models are not designed to be of a specific structural class, and dispatch is less often able to select a specialized procedure over the general approach. Of the 90 small models, only 31 belong to a non-general class (22 asymmetric-choice, 6 extended free-choice, 2 marked graph and 1 state machine), and the other 59 are general nets. For 43 of the 90 models (48%) the automatic dispatch can do something other than explore, either because the net belongs to a restricted class or because the efficient methods alone answer at least one query (27 models). For all other models it can only run its structural checks and then explore, and any difference to the brute-force mode is then the overhead of these checks and, except for deadlock-freedom, of the Karp-Miller coverability graph vs. the reachability graph.

The first measurement on these models was bad for the automatic dispatch. On liveness it answered only 47 of 90 models, against 88 for brute-force, and where both answered, it took a median 5.0 times as long. I timed the individual stages of the library on single models to find out why, and found two causes. These timings were taken by hand and are not part of the figures and tables.

The first cause was in the implementation of the Karp-Miller coverability graph, which requires updating token counts of every discovered marking to ω where there exists a predecessor marking that it covers (so-called "ω-acceleration"). This necessarily brings increasing overhead as the graph grows in size, but the implementation was not optimized for the bounded case: it always checked all predecessors, even those which were added after the marking was first discovered. On a bounded net with 16,587 reachable states this took 51 seconds, whereas constructing the reachability graph took just 12 milliseconds. This was addressed by initially only comparing each new marking to the markings on the path by which it was first discovered, and only falling back to checking all predecessors if/when an ω is found. This produces identical results in my testing, and the construction time dropped to 130 milliseconds on the same net. I did not prove that the new approach produces identical results in general, but what it produces is still a sound coverability graph.

Figure 34 shows the two modes on every small model, before and after the optimization. Several runs which automatic dispatch previously timed out on now finish: after the optimization the automatic dispatch returns verdicts on 82 of 90 liveness cells (before: 47), and on the models that both modes answer, it is at least twice as slow on 46, about equal on 32 and at least twice as fast on 4. Deadlock-freedom queries are nearly unaffected, because the automatic dispatch explores the reachability graph, not the coverability graph, as its fallback for that property. The second cause, explained next, also applies to the deadlock-freedom queries.

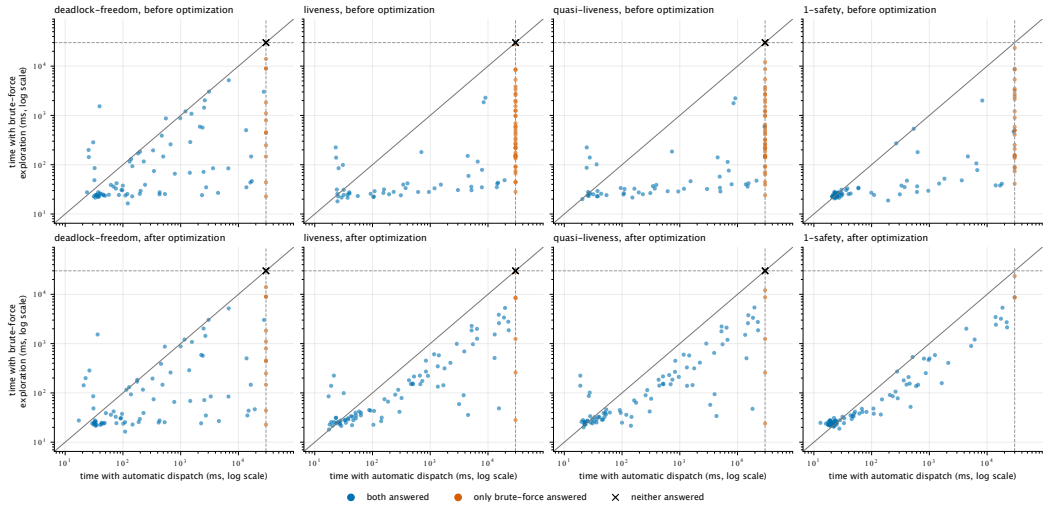


Figure 34: The automatic dispatch against brute-force on the 90 small MCC models, one panel per query, before and after the optimization of ω-acceleration described above.

The second cause remains in the codebase at the time of writing: for free-choice, asymmetric-choice, and general nets, the automatic dispatch attempts to use the Commoner/Hack criterion to decide the property where applicable (liveness of free-choice and asymmetric-choice nets, deadlock-freedom of all three), but this does not pay off for the majority of Petri nets. On one asymmetric-choice net with only 204 reachable states, the automatic dispatch took 32 seconds to return, while the reachability graph took 1.7 milliseconds. The Commoner/Hack criterion is the only other stage on that path, so it is the likely cause. Further investigation is needed to determine when the Commoner/Hack criterion is worth using, and the dispatch should be adjusted accordingly in a future version of the library.

Summary. Dispatching to specialized procedures by the structure of the net pays off where the structure allows a shortcut, and where the number of states explodes. On unstructured benchmark nets it costs time, because structural

checks and the conservative coverability-graph fallback add overhead to what is otherwise the same exploration.

5.4. Cost-Control

RQ4 asks whether a caller can effectively bound the cost of an analysis. Two mechanisms are examined.

The first mechanism is the ability to restrict `petrivet` to polynomial procedures only, and the hypothesis is that there are nets for which these procedures return a verdict within the time limit while the general brute-force procedure times out. The experiment reports the proportion of runs where the polynomial procedures are able to provide a verdict.

The second mechanism is the ability to set a deadline for the analysis, and the hypothesis is that `petrivet` returns "close" to the requested deadline. The definition of "close" was fixed as "within 200 ms" before the measured runs began, so that the data cannot influence the definition. The experiment measures how long beyond the requested deadline the library's automatic dispatch actually returns, for deadlines of 10, 100, and 1,000 milliseconds, in those runs the automatic dispatch without a deadline took more than 2 seconds or timed out. The distribution of the overshoot is reported, together with the proportion of runs which lie within the 200 ms threshold. The nets which trigger the largest overshoots are reported as well, because they indicate parts of the implementation which check the deadline too infrequently and could be improved.

The efficient-only mode. Table 8 shows how often the polynomial procedures are able to produce a verdict. On the parametrized families they give a verdict for 408 of 538 runs (76%) and decline in the rest. In 126 of the runs the polynomial methods answer where brute-force times out or runs out of memory, so the hypothesis is supported. The same happens for 8 runs of the large MCC models. On the small MCC models the efficient methods answer only 10% of the runs, and in none of them does brute-force time out. For MCC nets the restriction usually prevents a verdict from being returned at all.

Collection	Runs	Verdict	No verdict	Incorrect	Verdict where brute-force times out
Calibration nets	8	8	0	0	0
Parametrized families	538	408	130	0	126
MCC models, small	360	37	320	0	0
MCC models, large	80	11	68	0	8

Table 8: The number of runs where the efficient methods return a verdict, where they do not, where they return an incorrect verdict, and where they return a verdict while brute-force times out, for each collection of nets.

The deadline. 207 runs for which the automatic dispatch without a deadline took more than 2 seconds or timed out were repeated with deadlines of 10, 100 and 1,000 milliseconds. These counts and the statistics below leave out the runs that did not return before the 30-second limit (3 per deadline), because they report no overshoot. Table 9 and Figure 35 show how long after the deadline the library returned. The typical overshoot is small: the median is 3.5, 10.5 and 46.5 milliseconds, and 97.6%, 98.1% and 83.1% of the runs return within the target threshold of 200 milliseconds. The overshoot grows with the deadline. The hypothesis holds for the typical run, but it is not guaranteed: the distribution has a tail.

Deadline (ms)	Runs	Timed out	Median overshoot (ms)	90th percentile overshoot (ms)	95th percentile overshoot (ms)	Maximum overshoot (ms)	Proportion within threshold	Answered after deadline
10	207	3	3.5	7.9	37.9	9,579	97.6%	1.0%
100	207	3	10.5	44.8	49.4	9,630	98.1%	1.0%
1,000	207	3	46.5	342.6	389.5	8,577	83.1%	1.0%

Table 9: Statistics of the overshoot of `petrivet`'s return time beyond the requested deadline, for deadlines of 10, 100 and 1,000 milliseconds.

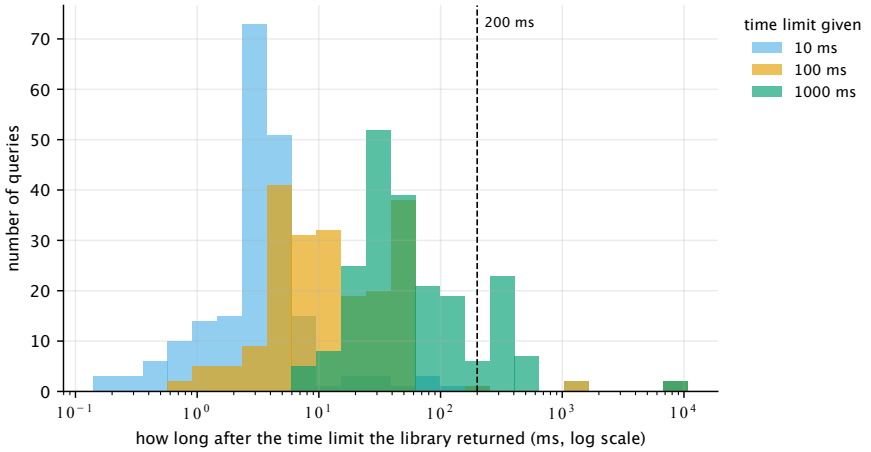


Figure 35: Distribution of the overshoot time between the deadline and return, for deadlines of 10, 100 and 1,000 ms. Bars further left mean that `petrivet` stopped closer to its deadline.

Investigation into the runs with the largest overshoot reveals a structural cause: the dispatch procedure for boundedness checks the deadline between its stages, but two of the stages can take a long time to complete and do not check the deadline internally. This is where the largest overshoots come from (Table 10): the boundedness question on the largest `fc-forkjoin` and `fc-forkjoin-leaky`

nets returns after about 9 seconds whatever the deadline, and then gives the correct verdict, so the deadline was ignored and not enforced. The liveness and quasi-liveness questions on one MCC model of the RERS2020 family, and the deadlock question on a ServersAndClients net, overshoot by about half a second to a second and then give no verdict. Worse than a late return is no return: for 3 runs at every deadline, on the 1-safety property for one large MCC model (FamilyReunion) and on two small ones (ServersAndClients), `petrivet` did not return at all, and timed out after 30 seconds. These are blind spots in the implementation which should be fixed. Thankfully, the fix is straightforward: the deadline needs to be checked more frequently in those procedures.

Collection	Net or family	Property	Deadline (ms)	Overshoot (ms)	Verdict
Parametrized families	`fc-forkjoin`	boundedness	100	9,629.7	TRUE
Parametrized families	`fc-forkjoin`	boundedness	10	9,578.9	TRUE
Parametrized families	`fc-forkjoin-leaky`	boundedness	10	9,526.2	TRUE
Parametrized families	`fc-forkjoin-leaky`	boundedness	100	9,391.9	TRUE
Parametrized families	`fc-forkjoin`	boundedness	1,000	8,577.1	TRUE
Parametrized families	`fc-forkjoin-leaky`	boundedness	1,000	8,438.4	TRUE
MCC models, small	RERS2020	liveness	10	1,159.0	none
MCC models, small	RERS2020	quasi-liveness	10	1,117.0	none
MCC models, small	RERS2020	liveness	100	1,072.5	none
MCC models, small	RERS2020	quasi-liveness	100	1,047.7	none

Table 10: The ten largest deadline overshoots.

Summary. The evidence generally supports the hypotheses of RQ4, with some caveats. The polynomial procedures are able to answer some queries where the general brute-force times out, but for many real-world Petri nets they are too limited to answer at all. The deadline mechanism is effective for the typical run, but there are blind spots in the implementation which allow `petrivet` to overshoot the deadline by a second or more, and even to not return at all. This can be fixed by checking the deadline more frequently in the affected procedures.

5.5. Comparison with Established Tools

RQ5 asks how `petrivet` compares with established tools in speed and memory. The comparison is performed with LoLA, TINA, and ITS-Tools, and is intended to place `petrivet` in the landscape of existing tools rather than to rank the tools against each other.

Queries answered. Table 11, Table 12 and Table 13 show how many queries each tool answers within the 30-second time limit, per property and collection. Under each count is the penalized average time (PAR-2) in seconds, in which a timeout, memory-out, or crash counts as twice the time limit; “unknown” results returned before the limit count as the time it took. A dash means that the tool has

no procedure for the query. No counted verdict of any tool contradicts the reference verdict; the exception is unbounded deadlock-freedom, explicitly out of scope of the comparison, where ITS-Tools gives incorrect verdicts (Section 5.2).

Tool	deadlock-freedom	boundedness	liveness	quasi-liveness
petrivet: automatic dispatch	123 / 144 9.1 s	144 / 144 0.2 s	127 / 144 7.4 s	127 / 144 7.4 s
petrivet: brute-force	97 / 144 19.7 s	94 / 144 21.3 s	92 / 144 21.7 s	92 / 144 21.7 s
petrivet: efficient methods only	94 / 144 0.0 s	144 / 144 0.2 s	104 / 144 0.0 s	104 / 144 0.0 s
LoLA	123 / 144 9.1 s	95 / 144 21.2 s	98 / 144 20.1 s	135 / 144 5.5 s
TINA	94 / 144 17.0 s	104 / 144 17.0 s	94 / 144 17.1 s	94 / 144 17.0 s
ITS-Tools	126 / 144 8.6 s	–	–	144 / 144 1.6 s

Table 11: Queries answered by each tool on the parametrized families (144 per property), and the penalized average time in seconds under each count.

On the parametrized families, `petrivet` dispatch answers all 144 boundedness queries (LoLA 95, TINA 104), 127 liveness queries (LoLA 98, TINA 94) and 123 deadlock-freedom queries, as many as LoLA and more than TINA (94). LoLA answers more quasi-liveness queries (135 against 127).

Tool	size of the state space	deadlock-freedom	liveness	quasi-liveness	1-safety	stable place
petrivet: automatic dispatch	–	76 / 90 11.1 s	82 / 90 7.8 s	82 / 90 7.9 s	87 / 90 3.6 s	–
petrivet: brute-force	89 / 90 1.7 s	88 / 90 2.0 s	88 / 90 2.3 s	88 / 90 2.3 s	90 / 90 0.8 s	89 / 90 1.7 s
petrivet: efficient methods only	–	3 / 90 0.1 s	4 / 90 0.1 s	4 / 90 0.1 s	26 / 90 2.0 s	–
LoLA	89 / 90 1.4 s	89 / 90 1.0 s	79 / 90 9.4 s	86 / 90 3.7 s	85 / 90 4.0 s	87 / 90 2.4 s
TINA	86 / 90 4.5 s	86 / 90 4.4 s	86 / 90 4.4 s	86 / 90 4.4 s	89 / 90 1.5 s	86 / 90 5.2 s
ITS-Tools	73 / 90 17.2 s	56 / 90 23.8 s	–	74 / 90 12.6 s	87 / 90 3.2 s	76 / 90 11.0 s

Table 12: Queries answered by each tool on the 90 small MCC models, and the penalized average time in seconds under each count.

On the small MCC models, LoLA and TINA answer more deadlock-freedom queries (89 and 86) than `petrivet` dispatch (76), and more quasi-liveness queries (86 and 86 against 82). For liveness, `petrivet` dispatch (82) is ahead of LoLA (79) and behind TINA (86), and for 1-safety all tools are within a few models of each other.

Tool	deadlock-freedom	liveness	quasi-liveness	1-safety
petrivet: automatic dispatch	4 / 20 50.5 s	4 / 20 48.0 s	4 / 20 48.0 s	4 / 20 48.0 s
petrivet: brute-force	6 / 20 43.7 s	0 / 20 60.0 s	0 / 20 60.0 s	5 / 20 46.0 s
petrivet: efficient methods only	0 / 20 0.1 s	4 / 20 0.1 s	4 / 20 0.1 s	3 / 20 3.4 s
LoLA	15 / 20 15.6 s	9 / 20 36.8 s	12 / 20 25.2 s	3 / 20 51.0 s
TINA	0 / 20 54.0 s	0 / 20 54.0 s	0 / 20 54.0 s	4 / 20 48.0 s
ITS-Tools	15 / 20 16.8 s	–	16 / 20 16.9 s	19 / 20 4.2 s

Table 13: Queries answered by each tool on the 20 large MCC models, and the penalized average time in seconds under each count.

On the large MCC models, `petrivet` dispatch answers 4 of 20 deadlock-freedom queries where LoLA and ITS-Tools answer 15, and TINA none; these are 20 models, so the comparison on them is a first impression and not a measurement of a distribution.

Time per property. The cactus plots in Figure 36 and Figure 37 show, for each property, the times of the answered queries sorted from the fastest to the slowest, one curve per tool. A curve ends early for a tool that answers fewer queries, and the left end of a curve is the cost of the smallest model, which is dominated by the tool's start-up overhead (Table 15). The high start-up overhead of ITS-Tools is explained by its JVM-based implementation. Figure 38 shows the same for the large MCC models.

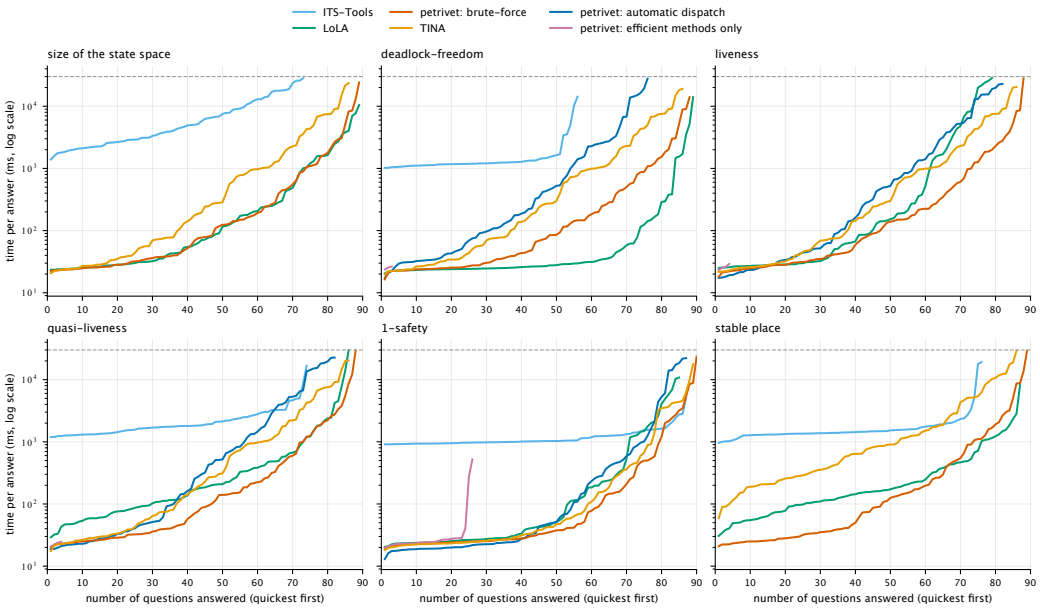


Figure 36: Cactus plots for the 90 small MCC models, one panel per property: the answered queries of each tool sorted ascending by time. The further a curve extends to the right, the more queries the tool answers, and the lower it runs, the faster.

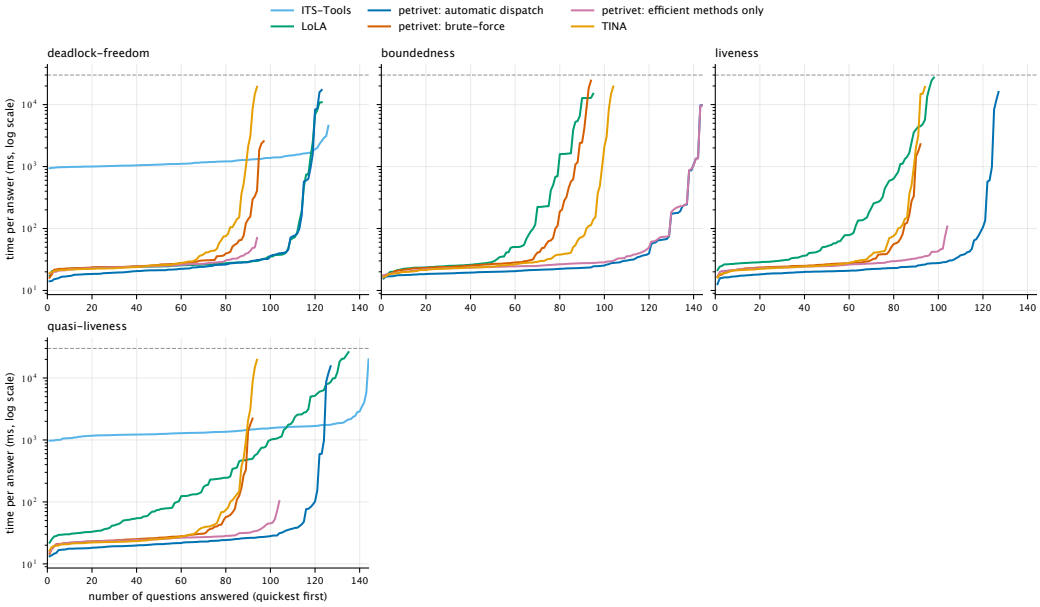


Figure 37: Cactus plots for the parametrized families, one panel per property, as in Figure 36.

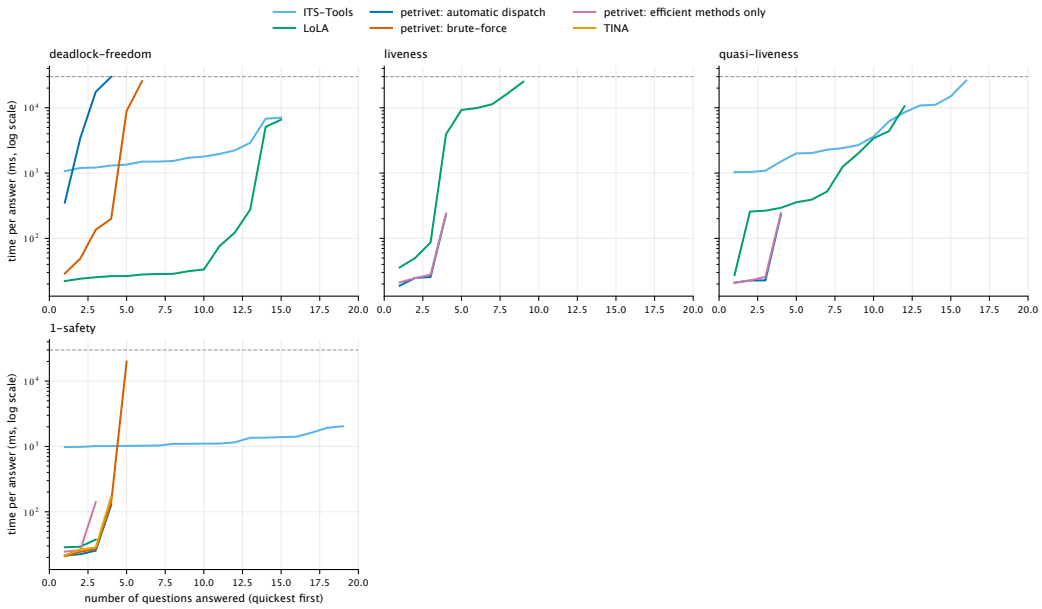


Figure 38: Cactus plots for the 20 large MCC models, one panel per property, as in Figure 36.

Head to head on the MCC models. A cactus plot compares the distributions of the query times, but hides how the tools compare on the same model. Figure 39 and Figure 40 therefore plot, for every model and property, the time of `petrivet` against the time of LoLA and against the time of TINA.

On the small models `petrivet` is at least twice as slow as LoLA on 53 of the 76 deadlock-freedom queries that both answer, and faster by at least a factor 2 on only 2. For liveness and quasi-liveness the models are split: against LoLA, `petrivet` is at least twice as fast on 23 and 30 models, within a factor 2 on 25 and 22, and at least twice as slow on 27 and 29, respectively.

The comparison with TINA is more even: on deadlock-freedom, `petrivet` is at least twice as fast on 25 models, within a factor 2 on 25 and at least twice as slow on 24. For liveness, quasi-liveness and 1-safety it is within a factor 2 on most models (54, 54 and 75).

On the models where only one of the two tools answers, the picture depends on the property: for deadlock-freedom LoLA answers 13 models that `petrivet` does not, and `petrivet` none that LoLA does not; for liveness `petrivet` answers 7 models that LoLA does not, and LoLA 4 models that `petrivet` does not. On the large models the two tools both answer at most 3 models of a question, so the figure is mostly a picture of where each tool fails: LoLA answers 12 deadlock-freedom models that `petrivet` does not, and `petrivet` 1.

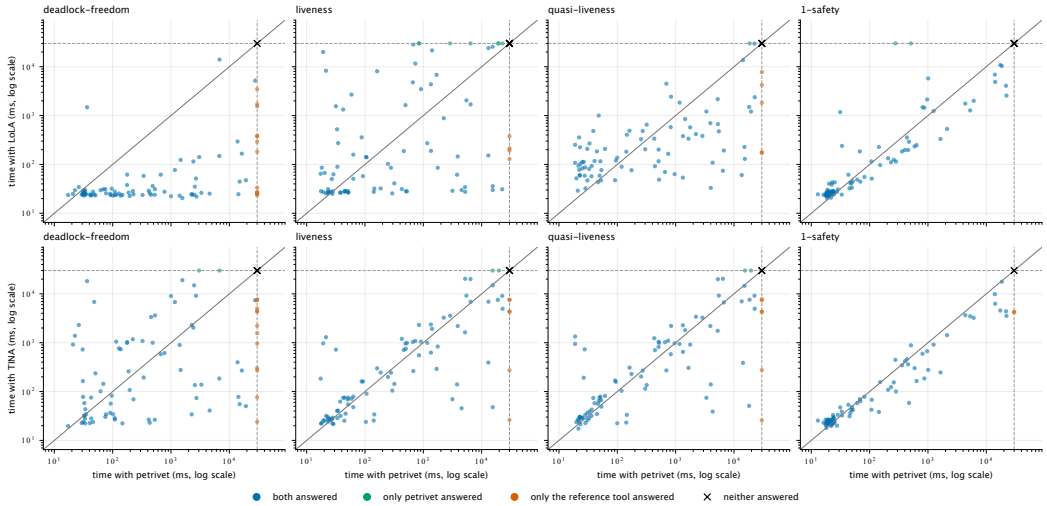


Figure 39: Time of `petrivet` (dispatch, x-axis) against the time of LoLA (top row) and TINA (bottom row), one dot per small MCC model and property. Dots above the diagonal are models on which `petrivet` was faster.

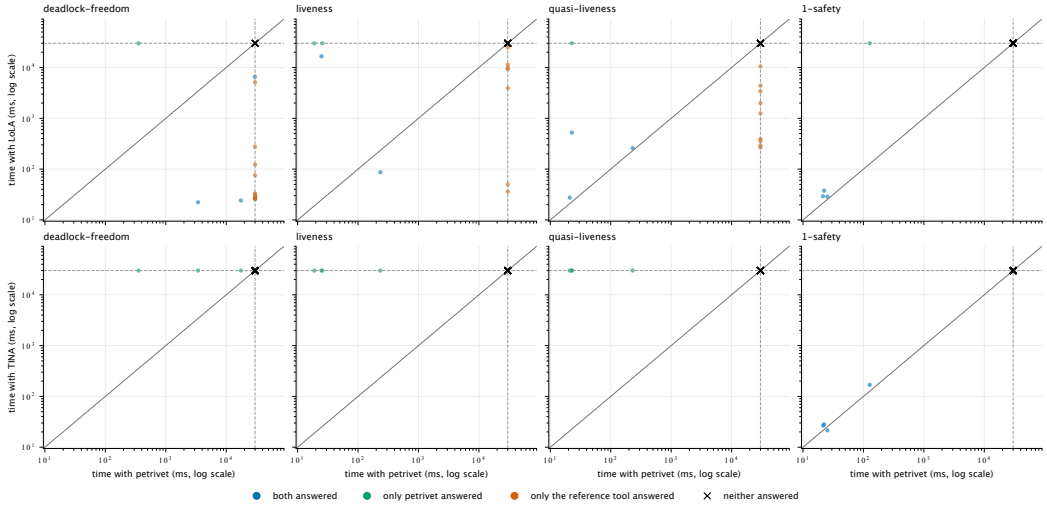


Figure 40: The same as Figure 39 for the 20 large MCC models.

Scaling on the parametrized families. While the MCC models vary in structure and size at once, the parametrized families elicit the same structure at differing sizes. Figure 41 shows, for each family, the time to decide four properties by net size, and Table 14 the largest net size at which every property is decided within the time limit.

`petrivet` dispatch decides all four properties up to size 1,024 on 10 of the 15 families, TINA on 8 and LoLA on 2. On the families whose state spaces grow exponentially with net size (`sm-hub-and-spokes`, `sm-two-cycles-n-tokens`, `sm-`

`ring-with-sink`, `mg-forkjoin-cycle`, `fc-forkjoin`), LoLA and TINA time out from size 8 to 16, whereas `petrivet` dispatch reaches size 1,024 (64 on `fc-forkjoin`), because it decides these properties from the structure and the initial marking of the net alone, without exploring the state space. `mg-forkjoin-unbounded` has no size at which the tools decide all four properties. The picture reverses on `ac-cycle`, `ac-cycle-no-low-token` and `general-cycle`, whose number of reachable markings grows only polynomially (about n^2 for `ac-cycle` and `general-cycle`): TINA reaches size 1,024 on all three, and `petrivet` dispatch stops at size 64, where its structural analysis becomes the bottleneck (the memory figure below shows it using 5 to 7 GiB). LoLA is in between: it reaches size 1,024 only on `circuit` and `sm-two-cycles`, and times out at size 64 on `ac-cycle` and `general-cycle`.

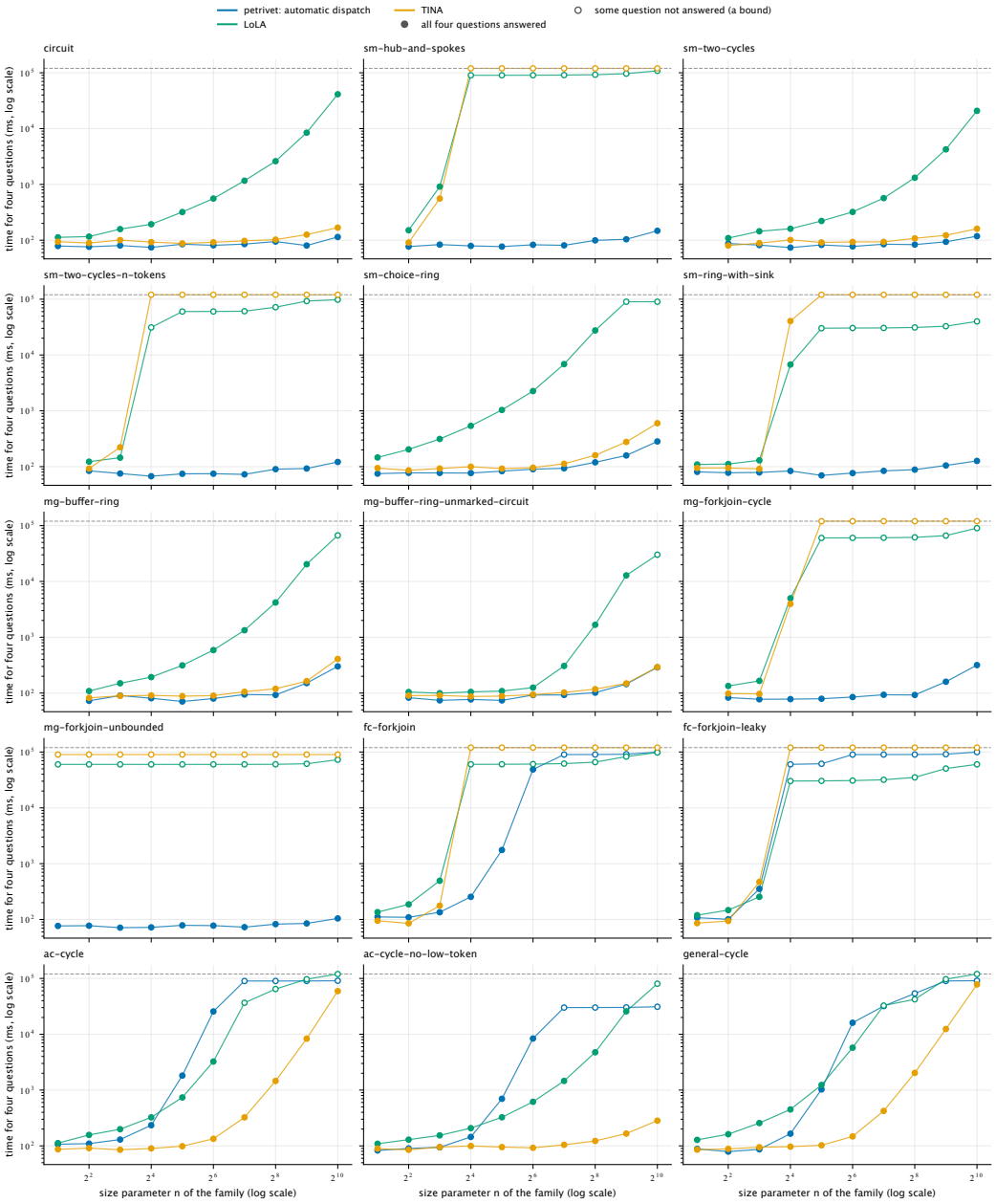


Figure 41: Time taken to decide four properties against the size of the net family, per tool. A hollow marker means that at least one run timed out, so the time is a lower bound.

Family	petrivet: automatic dispatch	LoLA	TINA
circuit	1,024	1,024	1,024
sm-hub-and-spokes	1,024	8	8
sm-two-cycles	1,024	1,024	1,024
sm-two-cycles-n-tokens	1,024	8	8
sm-choice-ring	1,024	256	1,024
sm-ring-with-sink	1,024	16	16
mg-buffer-ring	1,024	512	1,024
mg-buffer-ring-unmarked-circuit	1,024	512	1,024
mg-forkjoin-cycle	1,024	16	16
mg-forkjoin-unbounded	1,024	–	–
fc-forkjoin	64	8	8
fc-forkjoin-leaky	8	8	8
ac-cycle	64	64	1,024
ac-cycle-no-low-token	64	512	1,024
general-cycle	64	64	1,024

Table 14: The largest size of each family at which each tool decides all four properties within the limit (a dash: no such size). The sizes tried are the powers of two from 2 to 1,024.

Memory. Every tool has a start-up memory footprint below the measurement floor of the wrapper, 7.5 MiB, except ITS-Tools, which measured at 81 MiB because of the Java Virtual Machine (see Table 15). Figure 42 and Figure 43 plot the peak memory of `petrivet` against that of LoLA and TINA for each MCC model and property, in the same way as Figure 39 and Figure 40 for the time. `petrivet` dispatch uses at least twice as much memory as LoLA on 86 of the 90 small MCC models for deadlock-freedom, and on 65 and 67 models for liveness and quasi-liveness. Against TINA the models are split: for liveness `petrivet` uses at least twice as much on 30 models and at most half on 18.

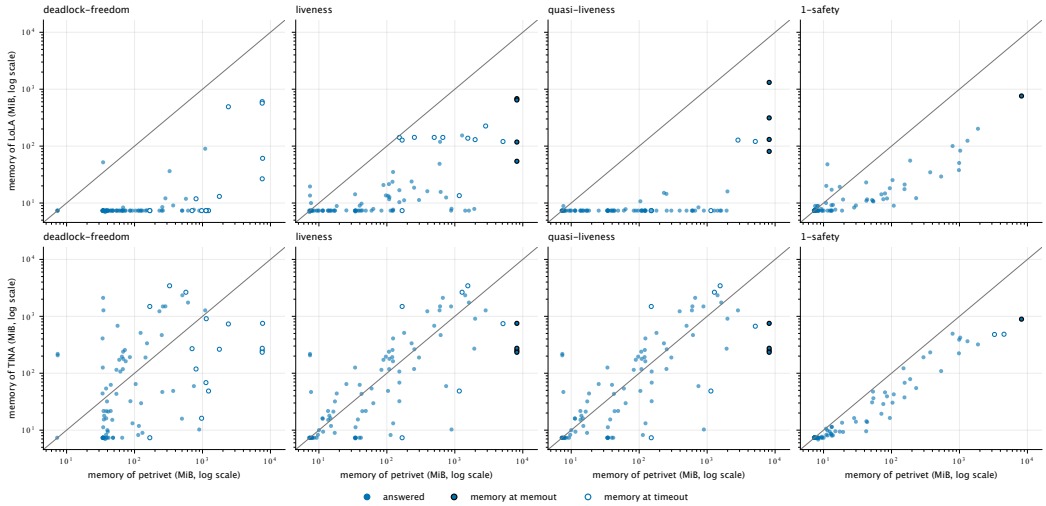


Figure 42: Peak memory of `petrivet` (dispatch, x-axis) against that of LoLA (top row) and TINA (bottom row) for each small MCC model and property. Dots above the diagonal are models on which `petrivet` used less memory. A dot with a dark outline includes a run that was ended by the memory limit, and a hollow dot includes a run which timed out (its value is the memory when it was stopped, a lower bound).

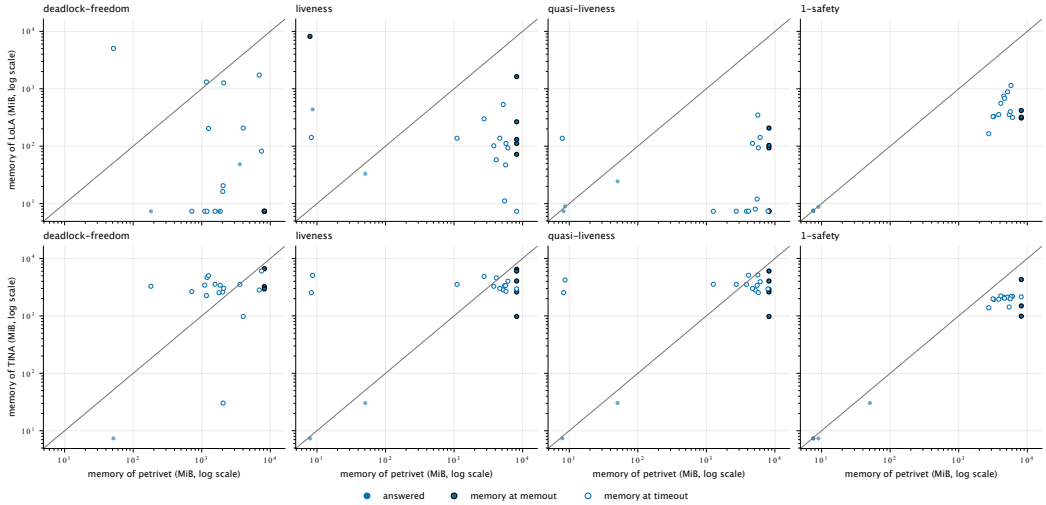


Figure 43: The same as Figure 42 for the 20 large MCC models.

On the parametrized families, Figure 44 shows the same effect as the time: on the families with an exponential number of markings LoLA and TINA run into the memory or the time limit with several GiB in use, whereas `petrivet` dispatch stays at a few tens of MiB. On the asymmetric-choice and general cycles the order is reversed, and `petrivet` is the tool with the largest memory.

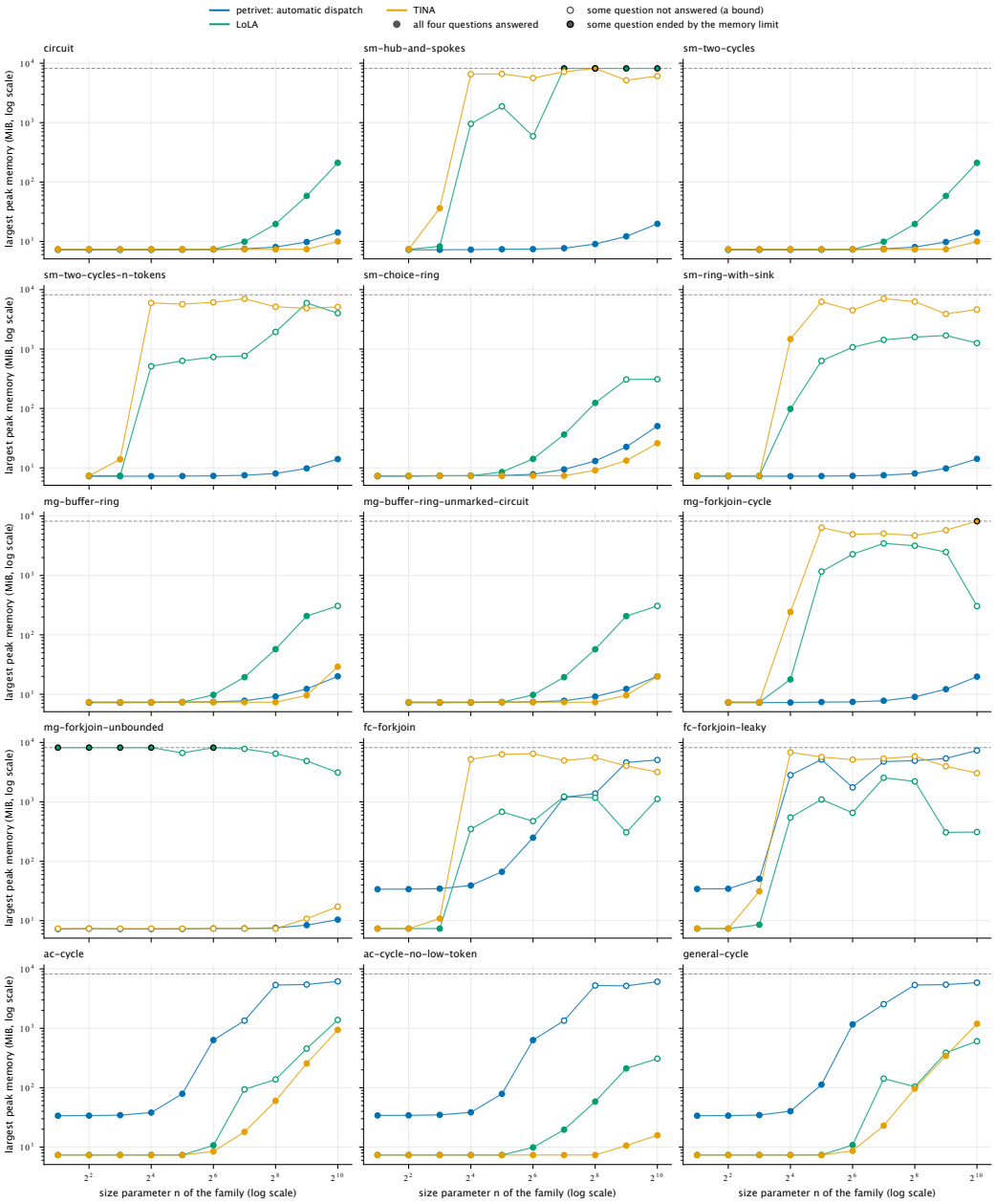


Figure 44: Peak memory against the size of the family, per tool: the largest peak of the four questions at each size (logarithmic axes, MiB; the dashed line is the memory limit of 8 GiB). A hollow marker means that some question was stopped by the time limit, so its value is the memory when it was stopped (a lower bound; it can be lower for the largest sizes, because the run got less far in 30 s), and a dark outline that some question was ended by the memory limit.

Figure 45 shows the memory footprint of the state space of the small MCC models, for those runs where the tools report the number of markings.

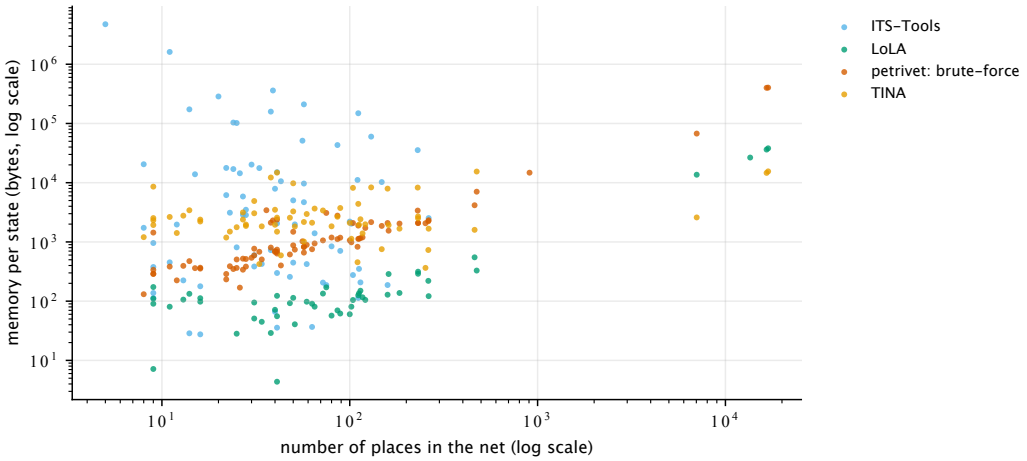


Figure 45: Memory footprint per reachable marking against the number of places, for each tool and each MCC model on which the tool reports a verdict on the number of markings. The start-up memory of the tool is subtracted.

The median bytes per marking is 45 for LoLA, 824 for `petrivet`, 1,940 for TINA and 2,209 for ITS-Tools.

A clear positive correlation is visible for `petrivet`, LoLA, and TINA, the explicit-state tools: the more places, the more memory per marking. For ITS-Tools, there is seemingly no correlation between the number of places and the memory per marking, which is explainable by its use of SDDs to store the state space: depending on how successful the SDD compression is, the memory per marking can be higher or lower than that of other tools.

It should be noted that `petrivet` is currently unable to generate the set of reachable markings (the nodes of the reachability graph) without also generating the edges, which is theoretically possible, and would reduce the memory footprint by a nontrivial factor.

Start-up. Table 15 shows the time and memory taken by each tool to decide deadlock-freedom for a trivial net as measured by the separate low-overhead wrapper described in Section 5.1. Unfortunately, the measurement floor of the wrapper is roughly 7.5 MiB, which obscures the true start-up memory of every tool except ITS-Tools.

Tool	Time (ms)	Memory (MiB)
ITS-Tools	989.9	80.6
LoLA	14.4	< 7.5
petrivet: automatic dispatch	18.4	< 7.5
petrivet: brute-force	8.2	< 7.5
petrivet: efficient methods only	7.9	< 7.5
TINA	7.2	< 7.5

Table 15: Start-up cost of each tool: wall time and peak memory on deadlock-freedom of a trivial net.

Limitations.

- Not every run was repeated, so the measurements are subject to noise.
- The time limit of 30 seconds and the memory limit of 8 GiB are arbitrary and obscure the true performance of the tools beyond those limits.
- The memory of a run stopped by the time limit is the memory at that moment and includes the memory footprint of the harness.
- The comparison omits the reachability and coverability properties due to insufficient time.

Conclusion. The comparison shows that `petrivet` is generally comparable with established tools in speed and memory, with some significant advantages on more restricted classes of nets, and some disadvantages on the less restricted classes.

5.6. Assessment against the Remaining Requirements

This section grades `petrivet` on all eight requirements R1 to R8 that were established in Section 1.1 with the rubric that was applied to the surveyed tools in Section 2, so that its rating can be placed in Table 1.

This grading cannot be independent: I wrote both the rubric and `petrivet`, and no second rater checked either (Section 2). To limit this bias, three rules were followed. First, every grade rests only on evidence that an independent reader can verify in `petrivet`'s repository. The deeper knowledge of the implementation was not used to justify a grade, and the evaluation in this chapter is only used to support R3 (cost control), which was evaluated in Section 5.4, and R5 (performance), which was evaluated in Section 5.5. Second, where the survey had already resolved a borderline case for another tool, the same resolution is applied to `petrivet`. Third, where a requirement is met only by some parts of the API, the gaps are named.

R1, programmatic access: ✓. `petrivet` is a library with a documented API: every public item carries documentation, rendered by `rustdoc`, and the crate documentation leads with a runnable example. Each analysis is a method call with a typed result rather than a string or integer code:

```
let mutual_exclusion_violation = petri_net
    .coverability()
    .target([(critical_section_a, 1), (critical_section_b, 1)])
    .call();
match mutual_exclusion_violation {
    Some(Coverability::Coverable { marking, firing_sequence }) => { ... }
    Some(Coverability::Uncoverable { contradiction }) => { ... }
    None => { ... }
}
```

Listing 13: Example of a programmatic query for coverability, with a typed result that carries different evidence depending on the answer.

The API is only available from Rust. Other languages need a wrapper: either a command-line interface with structured output, like the one built for the evaluation (`petrivet-cli`, which prints one JSON object per query), or bindings produced by a generator such as pyo3[80] for Python, cbindgen[81] for C/C++, ts-rs[82] for TypeScript, or wasm-bindgen[83] for WebAssembly, among others. `petrivet` itself currently ships no such bindings.

R2, safe portability: (✓). `petrivet` is written in Rust and forbids unsafe code in its own source (`#![forbid(unsafe_code)]` in `lib.rs`). Its default configuration, however, links the Z3 SMT solver, which is written in C++. By the rubric, this is a native-code dependency, and the survey graded the Z3-based checkers SMPT and Petrinizer "(✓)" for the same reason. Built with the pure-Rust solver OxiZ instead (`--no-default-features --features oxiz,pnml`), `petrivet` has no native dependency left and would meet the requirement fully.

R3, cost control: ✓. Every analysis method accepts a deadline, and the methods for boundedness, liveness, and deadlock-freedom also accept an effort restriction to polynomial procedures only:

```
petri_net
    .is_deadlock_free()
    .effort(Effort::Efficient)
    .timeout(Duration::from_millis(100))
    .call()
```

Listing 14: Example of a cost-controlled query for deadlock-freedom.

[80] https://github.com/pyo3/pyo3
[81] https://github.com/mozilla/cbindgen
[82] https://github.com/aleph-alpha/ts-rs
[83] https://github.com/wasm-bindgen/wasm-bindgen

A query that is not decided within these bounds returns None, which the type system keeps distinct from a negative verdict. Section 5.4 shows that the restriction to polynomial procedures returns verdicts where brute force times out, and that the deadline is kept within 200 ms in the large majority of runs; it also names the stages that do not check the deadline and cause the tail. There is no way to cancel a running query other than its deadline, which the rubric does not require.

R4, model interchange: ✓. petrivet reads and writes PNML for place/transition nets (`PnmlDocument::from_xml`, `PetriNet::to_pnml`), behind the default feature `pnml`. It only accepts ordinary nets that are connected, however. When the MCC models for the evaluation were drawn, 44 of the 134 drawn small models (33%) were rejected for this reason, 22 because of arc weights and 22 because they are not connected (corpus/mcc/selected.json). The rubric grades reading PNML at all, so the grade is unaffected, but in practice a third of the MCC models cannot be loaded.

R5, adequate performance and memory: ✓. `petrivet` participated in the Model Checking Contest 2026 [46], which, by the rubric, is independent evidence on non-trivial models; like for the other tools in the surveyed, the grade reflects that this evidence exists, not how good it is. The result was poor: `petrivet` placed last in every examination it entered. The submitted build dates from May 2026 and predates most of the procedures described in Section 4, so the contest measured an early version of the library and says little about the one evaluated in this thesis. The current version is evaluated against LoLA, TINA, and ITS-Tools in Section 5.5, but that evaluation was conducted by the author of the tool, and its models and limits were chosen by him. It shows `petrivet` competitive on liveness and on the parametrized families, but behind LoLA and TINA on deadlock-freedom and quasi-liveness on the small MCC models. Independent evidence for the current version would require participation in a later contest.

R6, evidence with answers: ✓ (method, witness, certificate, theorem, culprits). Reachability and coverability return a firing sequence as a witness for a positive answer, and for a negative answer the lemmas that contradict the target marking as a certificate. Each lemma can be checked against the net on its own, and together they form an infeasible linear system that can be re-checked by any solver, which is a certificate in the sense of the rubric. The deadlock iterator deadlocks() returns the reachable deadlock markings themselves, though without the firing sequence leading to them. The Commoner/Hack criterion returns the offending siphon and its unmarked maximal trap as culprits. The documentation names the theorem behind each procedure and links to a literature index (petrivet::literature). The evidence is uneven, however: `liveness()` and

`boundedness()` return only the verdict. The procedure that decided them is recorded internally, but not yet exposed.

R7, extensibility: ✓. The source is published under the LGPL 3.0, an OSI-approved license, and `petrivet` is usable as a library, which meets the rubric. Internally, procedures are written against generic interfaces where alternatives exist: the CEGAR engine against the `SmtSolver` trait, and the state space explorer against the `TokenOps` trait of its marking type. An extension concept for new model features, such as arc weights or time, does not exist yet and is listed in the roadmap of the README.

R8, approachability: (✓). Of the five repository indicators, `petrivet` meets four:

- a public GitHub repository with an issue tracker,
- a build from a clean checkout with the standard Rust toolchain (cargo build),
- automated tests (173 unit and integration tests, and a benchmark workflow in continuous integration),
- a `README.md` with a roadmap, and a `CONTRIBUTING.md` with contribution guidelines and a checklist for adding a new procedure,
- only one author and maintainer, so the rubric's requirement for a team is not met.

Like the other tools maintained by a single person, `petrivet` therefore partly meets the requirement.

Conclusion. `petrivet` fully meets six of the eight requirements (R1, R3, R4, R5, R6, R7), partly meets the other two (R2, R8), and fails none, which is more than any tool in the survey (at most five). The requirements that are not fully met are due to the C++ dependency of Z3 (R2) and the single maintainer (R8), and not to any deficiency in the design or implementation of `petrivet` itself.

5.7. Threats to Validity

This section lists the threats to validity of the evaluation.

- The reference verdicts for part of the randomized nets rest on one tool, LoLA.
- The randomized and parametrized nets are artificial and not necessarily representative of real-world models.
- The sample of MCC models is rather small.
- The results were measured on a single 10-year-old machine and may not be representative of other hardware.
- The library under test and the evaluation were both written by the same person.
- The memory measurements are not precise because of the overhead of the evaluation harness itself.

6. Discussion

This chapter interprets the results of the evaluation with respect to the research questions and requirements, summarizes the strengths and weaknesses of `petrivet`, and reflects on the lessons learned during its implementation.

6.1. Research Questions Revisited

RQ1: existing tools. The survey found no tool that meets all eight requirements (Section 2). The tools that are strongest in performance and evidence, the contest participants, are native command-line programs with proprietary input formats and no library interface, while the tools that are easiest to embed and extend decide few properties and offer no performance evidence. The gap that motivated `petrivet` is therefore not a missing algorithm, but a missing combination: the established procedures exist, but not in a form that a process-orchestration system can call, bound, and build on. `petrivet` was designed for this combination, and by the same rubric it fully meets six requirements, partly meets two, and fails none (Section 5.6), with the caveat that I rated it myself.

RQ2: correctness. No verdict of any of the three modes of `petrivet` contradicts a reference verdict (Section 5.2). This result should be read together with the three bugs found on the way to it. The most serious of them produced wrong verdicts on reachability and coverability, and it was only found because the evaluation compared every verdict with two independent tools. Unit tests did not find it, because they encoded the same misunderstanding as the code. The evaluation was therefore not only a measurement of correctness but a necessary part of achieving it, and the zero disagreements hold for the version after these fixes. The price of soundness is the share of queries answered with “unknown”: on the small MCC models the automatic dispatch gives no verdict for a large share of runs, mostly because it defensively builds the coverability graph where the reachability graph would have sufficed. That is the intended trade-off, but the trade-off is currently more conservative than it needs to be (Section 4.8).

RQ3: procedure dispatch. Dispatching to the cheapest applicable procedure pays off where the theory promises it: for circuits, state machines and marked graphs, the automatic dispatch is faster than brute force, and for families with exponentially growing state spaces it answers in milliseconds where brute force times out (Section 5.3). For extended free-choice nets the benefit is marginal, and for asymmetric-choice and general nets the dispatch is slower, because it pays for the Commoner/Hack criterion and the coverability graph without gaining a polynomial procedure in return. Two consequences follow. First, the benefit of dispatch depends on the distribution of structural classes in the models it is applied to. Most of the competition models are general nets, on which dispatch has little to offer; whether process models in practice fall into the restricted

classes more often is a question this evaluation does not answer. Second, dispatch is not free: its preliminary checks cost time on exactly the nets where they rarely succeed. A cheaper order of checks for general nets, or a polynomial procedure for free-choice nets such as the rank theorem, would address the two classes where dispatch currently loses. The evaluation also improved the library directly: investigating the poor first results on the MCC models led to the optimization of ω-acceleration, without which the coverability graph was orders of magnitude slower than the reachability graph on bounded nets.

RQ4: cost control. Both mechanisms work as intended for the typical query, with known limits (Section 5.4). The restriction to polynomial procedures answers queries on the parametrized families and the large MCC models where brute force times out, but rarely gives a verdict on the small MCC models, for the same reason that dispatch rarely helps there: few of them belong to a class with a polynomial procedure. The deadline is kept within a few milliseconds in the median, but the distribution has a tail caused by stages that do not check the deadline. For the chair's use case this means that a caller can rely on the deadline in practice, but not yet as a hard guarantee; the causes are named, and closing them is a matter of passing the deadline into the remaining stages.

RQ5: comparison with established tools. `petrivet` is competitive with LoLA and TINA on liveness and on the parametrized families, and behind them on deadlock-freedom and quasi-liveness on the small MCC models (Section 5.5). Given that LoLA and TINA have been developed for decades and optimized for exactly these benchmarks, being in the same range on some properties is a positive result, but `petrivet` is not a replacement for a dedicated model checker on hard instances. The gap on deadlock-freedom is explained by the fallback procedures: `petrivet` attempts the Commoner/Hack criterion first and then explores the full state space, whereas the established tools use reductions such as stubborn sets and structural reductions, which are known but not yet implemented (Section 4.8). `petrivet`'s low start-up overhead compared with JVM-based tools matters for the intended use, in which many small queries are issued from within another program.

6.2. Strengths and Weaknesses

The strengths of `petrivet` lie in the combination that the survey found missing: a memory-safe library with a typed API, cost control on every query, evidence with its answers, and verdicts that agree with the reference tools. Its procedure dispatch makes restricted classes of nets cheap to analyze, and its procedures are documented with their theoretical foundation.

Its weaknesses are, first, performance on general nets, where it falls back to state space exploration without the reductions that make the established tools fast;

second, its restriction to ordinary, connected nets, which excluded a third of the MCC models drawn for the evaluation; third, the uneven evidence returned by its API, where some properties return only a verdict; and fourth, its development by a single author, which shows in the partial rating for approachability.

6.3. Implementation Discussion

Separating the API from the core. Early versions of `petrivet` made no distinction between the user-facing API and the internal core logic: the user interacted directly with internal data structures and called internal procedures. This was simple, but the data the analysis algorithms needed was not always in the form the user wanted to see, and the performance of an analysis depended on the exact sequence of operations the user performed to construct the net. The current design separates the two: the API presents a friendly builder interface which prevents common mistakes at compile time, while the core works on a dense, canonical index-based representation. The trade-off is the need for an intermediate translation layer, which adds design complexity and marginal overhead to every query, and care must be taken not to let this translation leak into the hot path of the core logic. The rule that has slowly emerged, though not yet throughout the codebase, is that the core never depends on the API, and the API only calls into the core and maps its results back. It has proven a successful way to contain the translation, at the cost of some boilerplate.

Sharing code between procedures. Several procedures that the literature presents separately turned out to share most of their implementation. The reachability and coverability graphs are explored by the same algorithm, generic over the marking type, and differ only in the ω-acceleration step (Algorithm 1). The read arc and causality constraints of [37] became a single refinement, and the maximal siphon and trap computations are one algorithm and its dual. Sharing paid off in correctness as much as in code size: a fix to the explorer fixes both graphs, and the reachability graph explorer doubles as the reference against which the subclass procedures are tested.

Semi-decision procedures and soundness. The most serious bug found during the evaluation was not a programming error but a misreading of the literature. The increment constraints of [53] steer the search for a realizable firing sequence; they are not facts about the net, but I had treated an unsatisfiable problem that contained them as a proof of unreachability, and `petrivet` returned nine wrong verdicts (Section 5.2). The lesson is that in a semi-decision procedure the distinction between *what is known* and *what is assumed* must be maintained explicitly, and is easy to lose. Recording why a problem is unsatisfiable, initially added to explain negative answers to the user, turned out to be what made this bug diagnosable and fixable at all.

From theorem to code. Translating results from the literature into code was the most error-prone part of the work. Besides the increment constraints, I misunderstood a suggestion to search for traps in the subnet induced by a Parikh vector, and implemented a weaker variant that searches the whole net (Section 4.7.4); I initially implemented asymmetric choice with the wrong one of two definitions in common use; and I more than once misunderstood the conditions under which a theorem applies as broader than they actually are, for example a sufficient condition for deadlock-freedom that was used as if it were also necessary. None of these were caught by unit tests, because the tests encoded the same misreading as the implementation. They were caught by comparing against independent sources: the reachability graph on small nets, third-party tools on the evaluation corpus, and conversations with domain experts. Writing this thesis was itself such a check: formalizing the encodings and writing pseudocode for them exposed two further discrepancies between intent and implementation.

Cost control must be designed in from the start. Deadlines were added to `petrivet` after most procedures existed, by threading an optional deadline through each of them. The evaluation shows the result (Section 5.4): the typical query returns close to its deadline, but the stages that do not check the deadline run to completion and cause the tail of the distribution. A deadline only bounds the cost of a query if every loop whose running time depends on the size of the input checks it. This is easy to forget in each individual procedure, and hard to retrofit, so it belongs in the design of the procedure interface from the start. The same holds for the distinction between “no” and “unknown”: because every analysis returns an `Option` from the beginning, the type system enforces that every caller handles the unknown case, and the efficient-only mode could be added later without changing any signatures.

Dependencies are part of the correctness surface. Choosing a dependency means taking on its limits. The recursive strongly-connected-component algorithm of `petgraph` overflows the stack on large reachability graphs. Bugs in the early-stage pure-Rust OxiZ solver (not evaluated) directly impact the correctness of `petrivet`. Z3 itself brings native C++ code into an otherwise memory-safe library (Section 5.6). When correctness is a priority, the choice of dependencies is just as important as the correctness of the code itself.

Restricting the scope. `petrivet` supports only ordinary Petri nets, without arc weights. This kept the structural theory and its implementation simple, since many structural results, such as the Commoner/Hack criterion, are stated for ordinary nets. But the choice is reflected in the data layout of the core: presets and postsets are stored as adjacency lists of indices, not as maps from indices to

weights, which is why adding arc weights is not a small change, and it excluded a third of the MCC models drawn for the evaluation (Section 5.6). In hindsight, deciding early which model extensions a library will eventually support determines how much of it has to be rewritten later.

7. Conclusion

This thesis presented `petrivet`[2], a Rust library for analyzing place/transition Petri nets designed to meet the requirements of a process-orchestration setting. It is called from code, supports model interchange via PNML, allows bounding the cost of every query, returns evidence with its answers, and returns “unknown” where it has no answer. It has procedures for k-boundedness, liveness, deadlock-freedom, reachability, and coverability, dispatching each query to the most efficient known procedure that can answer it.

The research questions were answered as follows.

- **RQ1**: “To what extent do existing Petri net analysis tools meet the requirements R1 to R8?”
  No surveyed tool meets all eight requirements (Section 2).
- **RQ2**: “Does the implementation give correct answers, and does it return "unknown" where it cannot decide?”
  No verdict of `petrivet` contradicts a reference verdict across all evaluated queries (Section 5.2), after fixing the three bugs found during the evaluation.
- **RQ3**: “How much does answering each question with the cheapest applicable procedure, instead of one general procedure, reduce the cost of analysis?”
  Dispatch makes the more restricted net classes easier to analyze, often by orders of magnitude, but costs time on the more general nets (Section 5.3).
- **RQ4**: “Can a caller effectively bound the cost of an analysis, either by restricting the effort or by setting a deadline?”
  The restriction to polynomial procedures can return verdicts where brute-force times out, and deadlines are kept within 200 ms for the typical query, though some stages which do not check the deadline internally can cause overshoot (Section 5.4).
- **RQ5**: “How does `petrivet` compare with established tools in speed and memory?”
  `petrivet` is competitive with LoLA, TINA, and ITS-Tools for liveness and on the parametrized families, but behind them on deadlock-freedom and quasi-liveness on MCC models (Section 5.5).

The interpretation of these results, and lessons learned during implementation, are discussed in Section 6.

7.1. Future Work

The most direct improvements to `petrivet` concern performance on nets of less restricted structure. Known procedures that are not yet implemented, among them structural reductions, the rank theorem for free-choice nets, partial-order reductions, and minimal coverability set algorithms, are listed in Section 4.8.

Property-directed reachability (IC3) [62], [71] would add a complementary approach to the existing CEGAR engine.

The scope of `petrivet` can be extended beyond just ordinary Petri nets. Arc weights and place capacities are common extensions with the same analytical expressiveness as ordinary nets, but enhanced succinctness, and would allow for a more natural representation of certain systems. Colored Petri nets [72] are particularly useful in the context of business process orchestration, and would allow for a more direct representation of the systems that motivated this work. Temporal-logic model checking (LTL and CTL) would cover a wider range of queries than the current set of behavioral properties can express. Symbolic representations of the state space, such as decision diagrams [38], would allow for more efficient exploration of large bounded systems.

Finally, `petrivet` should be validated again outside this thesis by participating in upcoming editions of the Model Checking Contest [46], by language bindings that allow it to be used from other programming languages, and by adoption in real-world applications.

Bibliography

[1] C. A. Petri, *Kommunikation mit Automaten*, no. 2. in Schriften des IIM. Bonn: Institut für Instrumentelle Mathematik, 1962. Dissertation, Technische Hochschule Darmstadt. Accessed: Oct. 13, 2026. [Online]. Available: https://edoc.sub.uni-hamburg.de/informatik/volltexte/2011/160/

[2] W. M. P. van der Aalst, "Putting high-level Petri nets to work in industry," *Computers in Industry*, vol. 25, no. 1, pp. 45–54, Nov. 1994, doi: 10.1016/0166-3615(94)90031-0.

[3] W. M. P. van der Aalst, "Workflow Verification: Finding Control-Flow Errors Using Petri-Net-Based Techniques," in *Business Process Management: Models, Techniques, and Empirical Studies*, W. van der Aalst, J. Desel, and A. Oberweis, Eds., Berlin, Heidelberg: Springer, 2000, pp. 161–183. doi: 10.1007/3-540-45594-9_11.

[4] T. Murata, "Petri nets: Properties, analysis and applications," *Proceedings of the IEEE*, vol. 77, no. 4, pp. 541–580, Apr. 1989, doi: 10.1109/5.24143.

[5] E. Best and R. Devillers, *Petri Net Primer: A Compendium on the Core Model, Analysis, and Synthesis*. in Computer Science Foundations and Applied Logic. Cham: Springer Nature Switzerland, 2024. doi: 10.1007/978-3-031-48278-6.

[6] K. Schmidt, "LoLA A Low Level Analyser," in *Application and Theory of Petri Nets 2000*, M. Nielsen and D. Simpson, Eds., Berlin, Heidelberg: Springer, 2000, pp. 465–474. doi: 10.1007/3-540-44988-4_27.

[7] R. M. Dijkman, M. Dumas, and C. Ouyang, "Semantics and analysis of business process models in BPMN," *Information and Software Technology*, vol. 50, no. 12, pp. 1281–1294, Nov. 2008, doi: 10.1016/j.infsof.2008.02.006.

[8] J. Mangler and S. Rinderle-Ma, "CPEE - Cloud Process Execution Engine," in *Proceedings of the BPM Demo Sessions 2014*, L. Limonad and B. Weber, Eds., in CEUR Workshop Proceedings, vol. 1295. Eindhoven, Sep. 2014, pp. 51–55. Accessed: Oct. 19, 2026. [Online]. Available: http://eprints.cs.univie.ac.at/4136/

[9] J. Mangler and S. Rinderle-Ma, "Cloud Process Execution Engine: Architecture and Interfaces," Aug. 2022. Accessed: Oct. 19, 2026. [Online]. Available: https://arxiv.org/abs/2208.12214v2

[10] M. Chinosi and A. Trombetta, "BPMN: An introduction to the standard," *Computer Standards & Interfaces*, vol. 34, no. 1, pp. 124–134, Jan. 2012, doi: 10.1016/j.csi.2011.06.002.

[11] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2007, doi: 10.2753/MIS0742-1222240302.

[12] A. R. Hevner, “A Three Cycle View of Design Science Research,” *Scandinavian Journal of Information Systems*, vol. 19, no. 2, pp. 87–92, 2007, [Online]. Available: https://aisel.aisnet.org/sjis/vol19/iss2/4

[13] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–106, Mar. 2004, doi: 10.2307/25148625.

[14] J. Esparza, “Petri Nets Lecture Notes,” Munich, Apr. 2026. Lecture notes, Technical University of Munich, IN2052, Summer 2026. [Online]. Available: https://www.moodle.tum.de/course/view.php?id=117156

[15] T. Murata, “State equation, controllability, and maximal matchings of petri nets,” *IEEE Transactions on Automatic Control*, vol. 22, no. 3, pp. 412–416, Jun. 1977, doi: 10.1109/TAC.1977.1101509.

[16] E. W. Mayr, “An algorithm for the general Petri net reachability problem,” in *Proceedings of the thirteenth annual ACM symposium on Theory of computing*, in STOC '81. New York, NY, USA: Association for Computing Machinery, May 1981, pp. 238–246. doi: 10.1145/800076.802477.

[17] S. R. Kosaraju, “Decidability of reachability in vector addition systems (Preliminary Version),” in *Proceedings of the fourteenth annual ACM symposium on Theory of computing*, in STOC '82. New York, NY, USA: Association for Computing Machinery, May 1982, pp. 267–281. doi: 10.1145/800070.802201.

[18] W. Czerwiński, S. Lasota, R. Lazić, J. Leroux, and F. Mazowiecki, “The Reachability Problem for Petri Nets Is Not Elementary,” *Journal of the ACM (JACM)*, vol. 68, no. 1, p. 7:1–7:28, Dec. 2020, doi: 10.1145/3422822.

[19] W. Czerwiński and Ł. Orlikowski, “Reachability in Vector Addition Systems is Ackermann-complete,” arXiv, Oct. 2022. arXiv:2104.13866 [cs.FL]. Accessed: Oct. 17, 2026. [Online]. Available: http://arxiv.org/abs/2104.13866

[20] R. M. Karp and R. E. Miller, “Parallel program schemata,” *Journal of Computer and System Sciences*, vol. 3, no. 2, pp. 147–195, May 1969, doi: 10.1016/S0022-0000(69)80011-5.

[21] R. J. Lipton, “The Reachability Problem Requires Exponential Space,” Yale University, Department of Computer Science, New Haven, CT, Research Report 62, Jan. 1976. Accessed: Oct. 07, 2026. [Online]. Available: https://cir.nii.ac.jp/crid/1573387449044993408

[22] C. Rackoff, “The covering and boundedness problems for vector addition systems,” *Theoretical Computer Science*, vol. 6, no. 2, pp. 223–231, Jan. 1978, doi: 10.1016/0304-3975(78)90036-1.

[23] M. Hack, “The recursive equivalence of the reachability problem and the liveness problem for Petri nets and vector addition systems,” in *15th Annual Symposium on Switching and Automata Theory (swat 1974)*, Oct. 1974, pp. 156–164. doi: 10.1109/SWAT.1974.28.

[24] M. H. T. Hack, “Decidability questions for Petri Nets,” Dissertation, Massachusetts Institute of Technology, Cambridge, MA, 1976.

[25] N. D. Jones, L. H. Landweber, and Y. Edmund Lien, “Complexity of some problems in Petri nets,” *Theoretical Computer Science*, vol. 4, no. 3, pp. 277–299, Jun. 1977, doi: 10.1016/0304-3975(77)90014-7.

[26] M. H. T. Hack, “Analysis of Production Schemata by Petri Nets,” Massachusetts Institute of Technology, Cambridge, MA, technical report MAC-TR-94, Feb. 1972. Accessed: Oct. 07, 2026. [Online]. Available: https://hdl.handle.net/1721.1/149406

[27] J. Esparza, “Synthesis rules for Petri nets, and how they lead to new results,” in *CONCUR '90 Theories of Concurrency: Unification and Extension*, J. C. M. Baeten and J. W. Klop, Eds., Berlin, Heidelberg: Springer, 1990, pp. 182–198. doi: 10.1007/BFb0039060.

[28] P. Kemper and F. Bause, “An efficient polynomial-time algorithm to decide liveness and boundedness of free-choice nets,” in *Application and Theory of Petri Nets 1992*, K. Jensen, Ed., Berlin, Heidelberg: Springer, 1992, pp. 263–278. doi: 10.1007/3-540-55676-1_15.

[29] E. Best, “Structure Theory of Petri Nets: the Free Choice Hiatus,” in *Petri Nets: Central Models and Their Properties*, vol. 254, W. Brauer, W. Reisig, and G. Rozenberg, Eds., in Lecture Notes in Computer Science, vol. 254., Berlin, Heidelberg: Springer, 1987, pp. 168–205. doi: 10.1007/978-3-540-47919-2_8.

[30] J. Esparza and M. Silva, “On the analysis and synthesis of free choice systems,” in *Advances in Petri Nets 1990*, G. Rozenberg, Ed., Berlin, Heidelberg: Springer, 1991, pp. 243–286. doi: 10.1007/3-540-53863-1_28.

[31] J. Desel and J. Esparza, *Free Choice Petri Nets*. Cambridge University Press, 1995.

[32] B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, "Systematic literature reviews in software engineering – A systematic literature review," *Information and Software Technology*, vol. 51, no. 1, pp. 7–15, Jan. 2009, doi: 10.1016/j.infsof.2008.09.009.

[33] J. F. Jensen, T. Nielsen, L. K. Oestergaard, and J. Srba, "TAPAAL and Reachability Analysis of P/T Nets," in *Transactions on Petri Nets and Other Models of Concurrency XI*, M. Koutny, J. Desel, and J. Kleijn, Eds., Berlin, Heidelberg: Springer, 2016, pp. 307–318. doi: 10.1007/978-3-662-53401-4_16.

[34] B. Berthomieu, P.-O. Ribet, and F. Vernadat, "The tool TINA – Construction of abstract state spaces for petri nets and time petri nets," *International Journal of Production Research*, vol. 42, no. 14, pp. 2741–2756, Jul. 2004, doi: 10.1080/00207540412331312688.

[35] B. Berthomieu and F. Vernadat, "Time Petri Nets Analysis with TINA," in *Third International Conference on the Quantitative Evaluation of Systems - (QEST'06)*, Riverside, CA, USA: IEEE, Sep. 2006, pp. 123–124. doi: 10.1109/QEST.2006.56.

[36] Y. Thierry-Mieg, "Symbolic Model-Checking Using ITS-Tools," in *Tools and Algorithms for the Construction and Analysis of Systems*, C. Baier and C. Tinelli, Eds., Berlin, Heidelberg: Springer, 2015, pp. 231–237. doi: 10.1007/978-3-662-46681-0_20.

[37] Y. Thierry-Mieg, "Structural Reductions Revisited," in *Application and Theory of Petri Nets and Concurrency*, R. Janicki, N. Sidorova, and T. Chatain, Eds., Cham: Springer International Publishing, 2020, pp. 303–323. doi: 10.1007/978-3-030-51831-8_15.

[38] J.-M. Couvreur and Y. Thierry-Mieg, "Hierarchical Decision Diagrams to Exploit Model Structure," in *Formal Techniques for Networked and Distributed Systems - FORTE 2005*, F. Wang, Ed., Berlin, Heidelberg: Springer, 2005, pp. 443–457. doi: 10.1007/11562436_32.

[39] A. Valmari, "Stubborn sets for reduced state space generation," in *Advances in Petri Nets 1990*, G. Rozenberg, Ed., Berlin, Heidelberg: Springer, 1991, pp. 491–515. doi: 10.1007/3-540-53863-1_36.

[40] J. Esparza, R. Ledesma-Garza, R. Majumdar, P. Meyer, and F. Niksic, "An SMT-Based Approach to Coverability Analysis," in *Computer Aided*

Verification, A. Biere and R. Bloem, Eds., Cham: Springer International Publishing, 2014, pp. 603–619. doi: 10.1007/978-3-319-08867-9_40.

[41] W. M. P. van der Aalst, “Verification of workflow nets,” in *Application and Theory of Petri Nets 1997*, P. Azéma and G. Balbo, Eds., Berlin, Heidelberg: Springer, 1997, pp. 407–426. doi: 10.1007/3-540-63139-9_48.

[42] T. Freytag, P. Allgaier, A. Burattin, and A. Danek-Bulius, “WoPeD - A "Proof-of-Concept" Platform for Experimental BPM Research Projects,” in *Proceedings of the BPM Demo Track and BPM Dissertation Award*, in CEUR Workshop Proceedings, vol. 1920. Barcelona: CEUR-WS.org, 2017. Conference code: 15. [Online]. Available: https://ceur-ws.org/Vol-1920/BPM_2017_paper_190.pdf

[43] M. Heiner, M. Schwarick, and J.-T. Wegener, “Charlie – An Extensible Petri Net Analysis Tool,” in *Application and Theory of Petri Nets and Concurrency*, R. Devillers and A. Valmari, Eds., Cham: Springer International Publishing, 2015, pp. 200–211. doi: 10.1007/978-3-319-19488-2_10.

[44] D. Winterer, C. Zhang, and Z. Su, “Validating SMT solvers via semantic fusion,” in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, in PLDI 2020. New York, NY, USA: Association for Computing Machinery, Jun. 2020, pp. 718–730. doi: 10.1145/3385412.3385985.

[45] J. Park, D. Winterer, C. Zhang, and Z. Su, “Generative type-aware mutation for testing SMT solvers,” *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, p. 152:1–152:19, Oct. 2021, doi: 10.1145/3485529.

[46] F. Kordon *et al.*, “Complete Results for the 2026 Edition of the Model Checking Contest,” Jun. 2026. [Online]. Available: https://mcc.lip6.fr/2026/results.php

[47] S. Klabnik and C. Nichols, *The Rust Programming Language*, 2nd ed. San Francisco: No Starch Press, 2023.

[48] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, “RustBelt: securing the foundations of the Rust programming language,” *Proceedings of the ACM on Programming Languages*, vol. 2, no. POPL, p. 66:1–66:34, Dec. 2017, doi: 10.1145/3158154.

[49] O. Oanea, H. Wimmel, and K. Wolf, “New Algorithms for Deciding the Siphon-Trap Property,” in *Applications and Theory of Petri Nets*, J. Lilius and W. Penczek, Eds., Berlin, Heidelberg: Springer, 2010, pp. 267–286. doi: 10.1007/978-3-642-13675-7_16.

[50] M. Sharir, “A strong-connectivity algorithm and its applications in data flow analysis,” *Computers & Mathematics with Applications*, vol. 7, no. 1, pp. 67–72, Jan. 1981, doi: 10.1016/0898-1221(81)90008-0.

[51] R. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, Jun. 1972, doi: 10.1137/0201010.

[52] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, “Counterexample-Guided Abstraction Refinement,” in *Computer Aided Verification*, E. A. Emerson and A. P. Sistla, Eds., Berlin, Heidelberg: Springer, 2000, pp. 154–169. doi: 10.1007/10722167_15.

[53] H. Wimmel and K. Wolf, “Applying CEGAR to the Petri Net State Equation,” in *Tools and Algorithms for the Construction and Analysis of Systems*, P. A. Abdulla and K. R. M. Leino, Eds., Berlin, Heidelberg: Springer, 2011, pp. 224–238. doi: 10.1007/978-3-642-19835-9_19.

[54] Á. Hajdu, A. Vörös, and T. Bartha, “New Search Strategies for the Petri Net CEGAR Approach,” in *Application and Theory of Petri Nets and Concurrency*, R. Devillers and A. Valmari, Eds., Cham: Springer International Publishing, 2015, pp. 309–328. doi: 10.1007/978-3-319-19488-2_16.

[55] L. de Moura and N. Bjørner, “Z3: An Efficient SMT Solver,” in *Tools and Algorithms for the Construction and Analysis of Systems*, vol. 4963, C. R. Ramakrishnan and J. Rehof, Eds., in Lecture Notes in Computer Science, vol. 4963., Berlin, Heidelberg: Springer, 2008, pp. 337–340. doi: 10.1007/978-3-540-78800-3_24.

[56] J. Esparza and S. Melzer, “Verification of Safety Properties Using Integer Programming: Beyond the State Equation,” *Formal Methods in System Design*, vol. 16, no. 2, pp. 159–189, Mar. 2000, doi: 10.1023/A:1008743212620.

[57] G. Berthelot and G. Roucairol, “Reduction of Petri-Nets,” in *Mathematical Foundations of Computer Science 1976*, vol. 45, A. Mazurkiewicz, Ed., in Lecture Notes in Computer Science, vol. 45., Berlin, Heidelberg: Springer, 1976, pp. 202–209. doi: 10.1007/3-540-07854-1_175.

[58] A. Finkel, “The minimal coverability graph for Petri nets,” in *Advances in Petri Nets 1993*, G. Rozenberg, Ed., Berlin, Heidelberg: Springer, 1993, pp. 210–243. doi: 10.1007/3-540-56689-9_45.

[59] P.-A. Reynier and F. Servais, “Minimal Coverability Set for Petri Nets: Karp and Miller Algorithm with Pruning,” in *Applications and Theory of*

Petri Nets, L. M. Kristensen and L. Petrucci, Eds., Berlin, Heidelberg: Springer, 2011, pp. 69–88. doi: 10.1007/978-3-642-21834-7_5.

[60] A. Valmari and H. Hansen, "Old and New Algorithms for Minimal Coverability Sets," in *Application and Theory of Petri Nets*, S. Haddad and L. Pomello, Eds., Berlin, Heidelberg: Springer, 2012, pp. 208–227. doi: 10.1007/978-3-642-31131-4_12.

[61] P. A. Abdulla, S. P. Iyer, and A. Nylén, "SAT-Solving the Coverability Problem for Petri Nets," *Formal Methods in System Design*, vol. 24, no. 1, pp. 25–43, Jan. 2004, doi: 10.1023/B:FORM.0000004786.30007.f8.

[62] J. Kloos, R. Majumdar, F. Niksic, and R. Piskac, "Incremental, Inductive Coverability," in *Computer Aided Verification*, N. Sharygina and H. Veith, Eds., in Lecture Notes in Computer Science, vol. 8044. Berlin, Heidelberg: Springer, Jul. 2013, pp. 158–173. doi: 10.1007/978-3-642-39799-8_10.

[63] J. Kang, Y. Bai, and L. Jiao, "Abstraction-Based Incremental Inductive Coverability for Petri Nets," in *Application and Theory of Petri Nets and Concurrency*, D. Buchs and J. Carmona, Eds., Cham: Springer International Publishing, 2021, pp. 379–398. doi: 10.1007/978-3-030-76983-3_19.

[64] J.-M. Couvreur, E. Encrenaz, E. Paviot-Adet, D. Poitrenaud, and P.-A. Wacrenier, "Data Decision Diagrams for Petri Net Analysis," in *Application and Theory of Petri Nets 2002*, J. Esparza and C. Lakos, Eds., Berlin, Heidelberg: Springer, 2002, pp. 101–120. doi: 10.1007/3-540-48068-4_8.

[65] J. Esparza and C. Schröter, "Unfolding Based Algorithms for the Reachability Problem," *Fundamenta Informaticae*, vol. 47, no. 3–4, pp. 231–245, Oct. 2001.

[66] A. Hamez, Y. Thierry-Mieg, and F. Kordon, "Hierarchical Set Decision Diagrams and Automatic Saturation," in *Applications and Theory of Petri Nets*, K. M. van Hee and R. Valk, Eds., Berlin, Heidelberg: Springer, 2008, pp. 211–230. doi: 10.1007/978-3-540-68746-7_16.

[67] A. Hamez, Y. Thierry-Mieg, and F. Kordon, "Building Efficient Model Checkers using Hierarchical Set Decision Diagrams and Automatic Saturation," *Fundamenta Informaticae*, vol. 94, no. 3–4, pp. 413–437, Jul. 2009, doi: 10.3233/FI-2009-137.

[68] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945, doi: 10.2307/3001968.

[69] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979, Accessed: Oct. 31, 2026. [Online]. Available: https://www.jstor.org/stable/4615733

[70] C. J. Clopper and E. S. Pearson, “The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial,” *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934, doi: 10.2307/2331986.

[71] Z. Hassan, A. R. Bradley, and F. Somenzi, “Better generalization in IC3,” in *2013 Formal Methods in Computer-Aided Design*, IEEE, Oct. 2013, pp. 157–164. doi: 10.1109/FMCAD.2013.6679405.

[72] W. M. P. van der Aalst, C. Stahl, and M. Westergaard, “Strategies for Modeling Complex Processes Using Colored Petri Nets,” in *Transactions on Petri Nets and Other Models of Concurrency VII*, K. Jensen, W. M. P. van der Aalst, G. Balbo, M. Koutny, and K. Wolf, Eds., Berlin, Heidelberg: Springer, 2013, pp. 6–55. doi: 10.1007/978-3-642-38143-0_2.

8. Appendix

8.1. Related Work Protocol Amendments

8.2. Proof of Correctness of Algorithm 5

The proof uses the following known facts.

1. *Token movement* [14, Thm. 5.1.3]: if q is marked and $q \rightsquigarrow r$, the transitions on the path can be fired in order, which moves one token from q to r and leaves the other tokens in place.
2. *Strongly connected components* can be computed in linear time, and $\mathscr{C}$ is acyclic [51]. Hence every node of N reaches some terminal SCC.

Let $t \in T$ and $\{p_t\} \coloneqq {\bullet}t$ (the unique input place of t).

Let $\langle M_0 \rangle \coloneqq \{p \in P : M_0(p) > 0\}$ be the support of the initial marking.

Let $K(t)$ be the SCC of N containing t.

Lemma (Classification).
1. *Dead.* t is L_0 iff no place of $\langle M_0 \rangle$ reaches p_t.
2. *Strictly* L_1. If some place of $\langle M_0 \rangle$ reaches p_t and $K(t)$ is trivial, then t is L_1-live and not L_2-live.
3. L_3. If some place of $\langle M_0 \rangle$ reaches p_t and $K(t)$ is nontrivial, then t is L_3-live.
4. L_4. t is live iff there is a $q \in \langle M_0 \rangle$ with $\mathrm{Post}^*(q) \subseteq \mathrm{Pre}^*(p_t)$.

Proof.

(1) If $q \in \langle M_0 \rangle$ reaches p_t, move a token to p_t (Fact 1) and fire t. Otherwise, every marking M reachable from M_0 and every $r \in P$ satisfies $M(r) > 0 \Rightarrow \exists q \in \langle M_0 \rangle : q \rightsquigarrow r$, because a firing moves a token only to a successor of its place. So p_t is never marked.

(2) By (1), t is L_1-live. A token's trajectory is a walk in N. If it fired t twice, the walk between the two firings would close a circuit through t, contradicting that $K(t)$ is trivial. Hence every token fires t at most once, and so t fires at most $\sum_{p \in \mathrm{Pre}^*(p_t)} M_0(p)$ times in any firing sequence (hence not L_2-live).

(3) Move a token to p_t. As $K(t)$ is strongly connected and nontrivial, it contains a circuit through t, and the token can run around this circuit forever. This is an infinite firing sequence containing t infinitely often.

(4) ($\Leftarrow$) Let $B \coloneqq \mathrm{Post}^*(q)$ for some $q \in \langle M_0 \rangle$ with $B \subseteq \mathrm{Pre}^*(p_t)$. B is closed under successors, so no firing removes the last token from B: a firing either does not touch B or moves a token inside B. Hence $M(B) \geq M_0(q) \geq 1$ in every

reachable M. Every place of $B \subseteq \mathrm{Pre}^*(p_t)$ reaches p_t, so by Fact 1 t can fire from M.

($\Rightarrow$) Suppose no such q exists, so for every $q \in \langle M_0 \rangle$ there is $r_q \in \mathrm{Post}^*(q) \setminus \mathrm{Pre}^*(p_t)$. By Fact 1 we move every token of every marked place q to r_q. The resulting marking M has no token in $\mathrm{Pre}^*(p_t)$. $P \setminus \mathrm{Pre}^*(p_t)$ is closed under successors, since a place with a successor in $\mathrm{Pre}^*(p_t)$ would itself lie in $\mathrm{Pre}^*(p_t)$. So p_t is never marked from M on, and t is dead at M, hence not live.

The last case is the generalization of the argument for ($\Longrightarrow$) in [14, Thm. 5.1.3]. It remains to translate "$\mathrm{Post}^*(q) \subseteq \mathrm{Pre}^*(p_t)$" into a property of $\mathscr{C}$.

Lemma (Live SCCs).

Let $t \in T$ with $K := K(t)$ nontrivial. For every $q \in \langle M_0 \rangle$: $\mathrm{Post}^*(q) \subseteq \mathrm{Pre}^*(p_t)$ iff K is terminal and K is the only terminal SCC reachable from q.

Proof.

($\Rightarrow$) Since $q \in \mathrm{Post}^*(q) \subseteq \mathrm{Pre}^*(p_t)$, q reaches $p_t \in K$. If an arc leaves K to a node r, then r is reachable from q (through K), so $r \in \mathrm{Post}^*(q) \subseteq \mathrm{Pre}^*(p_t)$ reaches p_t, and $r \in K$, a contradiction. Hence K is terminal. Let K' be a terminal SCC reachable from q. Its places lie in $\mathrm{Post}^*(q) \subseteq \mathrm{Pre}^*(p_t)$, so they reach p_t; as no arc leaves K', $p_t \in K'$ and $K' = K$.

($\Leftarrow$) Let $r \in \mathrm{Post}^*(q)$. By Fact 2, r reaches a terminal SCC, which is reachable from q and hence equals K. K is strongly connected, so r reaches p_t, i.e. $r \in \mathrm{Pre}^*(p_t)$.

Lemma (The algorithm).

After termination, for every SCC K:

$\ell[K] = \mathrm{L}_0$ if no node of K is reachable from $\langle M_0 \rangle$;

$\ell[K] = \mathrm{L}_1$ if K is trivial and reachable from $\langle M_0 \rangle$;

$\ell[K] = \mathrm{L}_4$ if K is nontrivial, terminal, and the unique terminal SCC reachable from some marked place; and

$\ell[K] = \mathrm{L}_3$ otherwise.

The algorithm runs in time $O(|P| + |T| + |F|)$.

Proof.

Reachability. We show by induction on the position of K in π that $r[K] = \top$ at the end iff some SCC containing a marked place reaches K in $\mathscr{C}$. Initially $r = m$. Every predecessor of K precedes K in π, so by the induction hypothesis its flag

is final when it is visited, and a reachable predecessor sets $r[K]$. Conversely only a reachable predecessor or $m[K]$ can set $r[K]$. Since the nodes reachable from a marked place are those of the SCCs reachable from its SCC, r identifies exactly the SCCs reachable from $\langle M_0 \rangle$.

Terminal SCCs. Let $\mathrm{Term}(K)$ be the set of terminal SCCs reachable from K; it is nonempty by Fact 2. We show by induction over $\mathrm{rev}(\pi)$ that $\sigma[K] = \mathrm{One}(K')$ if $\mathrm{Term}(K) = \{K'\}$ and $\sigma[K] = \mathrm{Multiple}$ otherwise. If K is terminal, $\mathrm{Term}(K) = \{K\}$. Otherwise every path from K leaves through a successor, so $\mathrm{Term}(K) = \bigcup_{K' \in \mathrm{succ}(K)} \mathrm{Term}(K')$. All successors follow K in π, so their σ values are final, and the join of singletons $\{a\}$ and $\{b\}$ describes a singleton iff $a = b$, while Multiple is absorbing. So the union is a singleton iff all successors agree on the same single terminal SCC.

Levels. Marked SCCs and marked places have the same reachable set, so $K' \in \Sigma$ iff K' is the unique terminal SCC reachable from some marked place. Such a K' is reachable. The `match` assigns L_0 to unreachable SCCs, L_1 to reachable trivial SCCs, L_4 to reachable nontrivial SCCs in Σ, and L_3 to the remaining reachable nontrivial SCCs.

Complexity. The condensation and a topological order are computed in time linear in the size of N [51]. $\mathscr{C}$ has at most $|P| + |T|$ nodes and $|F|$ arcs. Each of the two passes visits every node and every arc once, and the computation of Σ, the level assignment and the final `flat_map` visit every node once.

Correctness. For every transition t, the algorithm returns $\ell[K(t)]$ with:

L_0 iff t is dead, L_1 iff t is strictly L_1-live, L_3 iff t is strictly L_3-live, and L_4 iff t is L_4-live. *Proof.* The four cases of the Algorithm lemma are exhaustive and mutually exclusive. The case L_0 is part (1) of the Classification lemma: since p_t is the only predecessor of t, a marked place reaches t iff it reaches p_t, i.e. iff $K(t)$ is reachable from $\langle M_0 \rangle$. The case L_1 is part (2), L_4 is part (4) combined with the Live SCCs lemma, and for L_3 we use part (3) and the fact that t is not live by part (4).

Corollary. No transition of a state machine is strictly L_2-live. For a strongly connected state machine with $M_0(P) > 0$, the algorithm returns L_4 for every transition, which agrees with [4, Thm. 4] and [14, Thm. 5.1.3].

Proof. A transition that is not dead is either L_1 and not L_2, or L_3 and hence L_2. A strongly connected net has one SCC, which is terminal and is the only terminal SCC reachable from any marked place.

8.3. Proof of Correctness of Algorithm 6

The proof reduces to four known facts about marked graphs (T-systems).

1. *Token invariance* [4, Thm. 6], [14, Prop. 5.2.2]: $M(\gamma) = M_0(\gamma)$ for every circuit γ and every reachable marking M. Hence an unmarked circuit stays unmarked, so $Z := \{t \in T : \exists \gamma : t \in \gamma \wedge M_0(\gamma) = 0\}$ does not depend on the marking, and no transition of Z is ever enabled (this is the ($\Longrightarrow$) direction of [14, Thm. 5.2.3]).
2. *Necessity of firing* ([4, §VII-D]): if N has a token-free directed path from i to j at M_0, then i must fire before j can first become enabled.
3. *Local liveness* (the ($\Longleftarrow$) direction of [14, Thm. 5.2.3]): Let M be a marking and t a transition such that no circuit unmarked at M has a token-free path to t. Then some marking reachable from M enables t. The proof of the cited theorem uses its hypothesis "every circuit is marked at M" only to guarantee that a maximal token-free path Π ending in t has a transition as first element whose input places are all marked. If a place were unmarked, extending Π would either yield a longer token-free path or close a circuit inside Π that is unmarked and has a token-free path to t, which our hypothesis excludes. The induction measure $|S_M|$ and its decrease under firing $M \xrightarrow{u} L$ ($S_L \subset S_M$) do not use the hypothesis again, and the hypothesis is preserved by firing by token invariance.
4. *Graph algorithms*: the strongly connected components of a digraph are computed in linear time [51], and a node lies on a directed cycle iff its component has at least two nodes.

Lemma (Unmarked circuits). Let N_0 be the subgraph of N induced by the unmarked places at M_0 and all transitions. Z is the set of transitions in nontrivial strongly connected components of N_0.

Proof. N_0 contains exactly the unmarked places, so a directed cycle of N_0 is an unmarked circuit and vice versa. The claim is Fact 4.

Lemma (Classification).

Let $D_0 := \{t \in T : \exists z \in Z : z \rightsquigarrow_0 t\}$ be the set of transitions reachable from an unmarked circuit via a token-free path at M_0.

Let $D_{\geq} := \{t \in T : \exists z \in Z : z \rightsquigarrow t\}$ be the set of transitions reachable from an unmarked circuit via any path at M_0.

Let $t \in T$.

1. If $t \in D_0$, then t is dead.
2. If $t \notin D_0$, then t is potentially firable (L_1-live).

3. If $t \in D_{\geq}$, then t fires boundedly often in every firing sequence, hence is not L_2-live.
4. If $t \notin D_{\geq}$, then t is L_4-live.

Proof.

(1) By Fact 1 every $z \in Z$ is dead. If $z \rightsquigarrow_0 t$, Fact 2 shows that z would have to fire before t can become enabled, so t is dead as well.

(2) Apply Fact 3 to $M = M_0$: the hypothesis is exactly $t \notin D_0$.

(3) Let $z = t_0, p_1, t_1, \ldots, p_k, t_k = t$ be a path with $z \in Z$, let σ be a firing sequence leading from M_0 to M, and let $\vec{\sigma}$ be the Parikh vector of σ. Since t_{i-1} and t_i are the only producer and consumer of p_i, the marking equation gives $0 \leq M(p_i) = M_0(p_i) + \vec{\sigma}(t_{i-1}) - \vec{\sigma}(t_i)$. As z is dead by (1), $\vec{\sigma}(t_0) = 0$, and induction yields $\vec{\sigma}(t) \leq \sum_i M_0(p_i)$.

(4) Let M be reachable. If a circuit were unmarked at M and had a path to t, it would be unmarked at M_0 by Fact 1, so $t \in D_{\geq}$. Hence the hypothesis of Fact 3 holds at every reachable M, so t can always be enabled again.

Lemma (Depth-first search). After the algorithm terminates, $\ell(t) = \mathrm{L}_0$ if $t \in D_0$, L_1 if $t \in D_{\geq} \setminus D_0$, and L_4 otherwise. It runs in time $O(|P| + |T| + |F|)$.

Proof.

A call $\mathrm{Dfs}(t, b)$ passes the `match` without returning only if $\ell(t) = \mathrm{L}_4$, or $\ell(t) = \mathrm{L}_1$ and $b = \bot$; in both cases the assignment strictly lowers $\ell(t)$. So every transition is expanded at most twice, which bounds the work by $O(|F|)$, and together with Fact 4 the total is linear. The search explores the product graph on states (t, b) with $b \in \{\bot, \top\}$, where passing a marked place sets b to $\top$. Arriving at t through unmarked places only ($\bot$) is at least as strong as arriving through a path that crossed a marked place ($\top$), because the same continuation from $(t, \bot)$ reaches every transition that $(t, \top)$ reaches, with a flag that is never higher than the flag of the same continuation from $(t, \top)$. The pruning in the `match` discards a state (t, b) exactly when $(t, \bot)$ or (t, b) was already visited. Therefore the algorithm computes reachability in the product graph from the states $\{(z, \bot) : z \in Z\}$ [51], and $\ell(t)$ is L_0 iff some $(t, \bot)$ is reachable (i.e. $t \in D_0$), L_1 iff only $(t, \top)$ is reachable, and L_4 iff no state of t is reachable.

Correctness. The algorithm returns ℓ with $\ell(t) = \mathrm{L}_0$ iff t is dead, $\ell(t) = \mathrm{L}_1$ iff t is strictly L_1-live, and $\ell(t) = \mathrm{L}_4$ iff t is live.

Proof. By the Depth-first search lemma, ℓ is determined by the sets $D_0 \subseteq D_{\geq}$. By the Classification lemma, the transitions in D_0 are dead, those in $D_{\geq} \setminus D_0$ are L_1-

live but not L_2-live, and those outside $D_{\geq}$ are live. These three cases are exhaustive and mutually exclusive, which gives the equivalences.

Corollary. In a marked graph, no transition is strictly L_2-live or strictly L_3-live.

8.4. Proof of Correctness of Algorithm 7

The proof uses the following known facts.

1. *Token movement* [14, Thm. 5.1.3]: if q is marked and $q \rightsquigarrow r$, the transitions on the path can be fired in order, which moves one token from q to r and leaves the other tokens in place.
2. *Enabledness in state machines*: a transition t is enabled at M iff $M(p_t) > 0$, where $\{p_t\} := {\bullet t}$ (the unique input place of t).

Let $\langle M_0 \rangle = \{p \in P : M_0(p) > 0\}$ be the support of the initial marking M_0.

Let $\Sigma = \{p \in P : p{\bullet} = \emptyset\}$ be the set of sink places.

Lemma (Deadlock markings).

A marking M is a deadlock iff every marked place of M has an empty postset.

Proof. By Fact 2, M enables no transition iff $M({\bullet t}) = 0$ for every $t \in T$. The places of ${\bullet T}$ are exactly the places with a nonempty postset, so this holds iff all marked places have an empty postset.

Lemma (Reachable deadlocks).

A deadlock is reachable from M_0 iff $\langle M_0 \rangle \subseteq \mathrm{Pre}^*(\Sigma)$.

Proof. ($\Leftarrow$) For every $q \in \langle M_0 \rangle$ pick $r_q \in \mathrm{Post}^*(q) \cap \Sigma$. By Fact 1 we move every token of every marked place q to r_q. The resulting marking has tokens only on sink places and is a deadlock by the previous lemma. If $\langle M_0 \rangle = \emptyset$, then M_0 itself is a deadlock. ($\Rightarrow$) Let M be a reachable deadlock and let $q \in \langle M_0 \rangle$. The set $\mathrm{Post}^*(q)$ is closed under successors, so a firing either does not touch $\mathrm{Post}^*(q)$, moves a token into it, or moves a token within it. Hence the number of tokens on $\mathrm{Post}^*(q)$ never decreases, and $M(\mathrm{Post}^*(q)) \geq M_0(q) \geq 1$. So some place $r \in \mathrm{Post}^*(q)$ is marked at M, and by the previous lemma r is a sink place. Thus q reaches a sink place, i.e. $q \in \mathrm{Pre}^*(\Sigma)$.

Lemma (Backward search). $\mathrm{PreStar}(X)$ returns $\mathrm{Pre}^*(X)$ and runs in time $O(|P| + |T| + |F|)$.

Proof.

Soundness. Every place added to X is a predecessor q of some transition $t \in \bullet p$, where p was already in the set. So $q \rightsquigarrow t \rightsquigarrow p$, and inductively q reaches a place of the initial X.

Completeness. Let $q \rightsquigarrow r$ with $r \in X$, via a path $q = p_0, t_1, p_1, \ldots, t_k, p_k = r$. Every place in X is pushed onto W exactly once, either initially or when it is added, so every place of the final set is popped. By induction from the end of the path, p_i is in the set: when p_{i+1} is popped, $t_{i+1} \in \bullet p_{i+1}$ and $p_i \in \bullet t_{i+1}$ are scanned, and p_i is added unless it is already present.

Complexity. Each place is pushed and popped at most once. Popping p scans $\bullet p$ and, for each $t \in \bullet p$, the single place p_t. Since every transition has exactly one output place, each transition t is scanned exactly once over the whole run, so the total work is $O(|P| + |T| + |F|)$.

Correctness. The algorithm returns true iff (N, M_0) is deadlock-free. It runs in time $O(|P| + |T| + |F|)$.

Proof. By the Reachable deadlocks lemma, (N, M_0) is deadlock-free iff $\langle M_0 \rangle \not\subseteq \mathrm{Pre}^*(\Sigma)$, i.e. iff some marked place cannot reach a sink place. By the Backward search lemma, $B = \mathrm{Pre}^*(\Sigma)$. The algorithm returns exactly $\langle M_0 \rangle \not\subseteq B$. The computation of Σ and the subset test are linear in $|P|$.

Corollary. A state machine whose initial marking has no token is never deadlock-free. A state machine without sink places is deadlock-free iff $M_0(P) > 0$. *Proof.* If $\langle M_0 \rangle = \emptyset$, M_0 is a deadlock. If $\Sigma = \emptyset$, then $\mathrm{Pre}^*(\Sigma) = \emptyset$, and $\langle M_0 \rangle \not\subseteq \emptyset$ iff $\langle M_0 \rangle \neq \emptyset$.

8.5. Proof of Correctness of Algorithm 8

The proof reuses two results of the liveness section for marked graphs, stated there as parts (3) and (4) of the Classification lemma:

1. If $t \in D_{\geq}$, there is a bound B_t such that t occurs at most B_t times in every firing sequence.
2. If $t \notin D_{\geq}$, then t is live.

Lemma (Deadlock-freedom and liveness). (N, M_0) is deadlock-free iff some transition is live.

Proof. ($\Leftarrow$) Let t be live and suppose a deadlock M were reachable. Then M enables no transition, so $[M\rangle = \{M\}$ and t is dead at M, contradicting liveness. ($\Rightarrow$) Suppose no transition is live. By Fact 2 every transition lies in $D_{\geq}$, so by Fact 1 every firing sequence has length at most $\sum_t B_t$. Take a firing sequence of

maximal length. If its final marking enabled a transition, the sequence could be extended, so the final marking is a reachable deadlock.

Lemma (Downstream search). After the loop, $d[t] = \top$ iff $t \in D_{\geq}$. The loop runs in time $O(|P| + |T|)$.

Proof.

Soundness. Every transition put on W is in Z or is the consumer v_p of a place p produced by a transition already marked in d. So $d[t] = \top$ implies $z \rightsquigarrow t$ for some $z \in Z$.

Completeness. Let $z \rightsquigarrow t$ via $z = t_0, p_1, t_1, \ldots, p_k, t_k = t$ with $z \in Z$. Every element of Z is on W initially, and when a transition is marked in d all consumers of its output places are pushed. Inductively t_i is pushed, and popped, hence $d[t_i] = \top$, for all i.

Complexity. A transition is expanded only when it is popped with $d[t] = \bot$, so at most once, and its expansion pushes $|p \bullet| = 1$ entry per output place. So at most $|T| + |F|$ entries are ever pushed and popped, and since every place of a marked graph has exactly one input arc and one output arc, $|F| = 2|P|$.

Correctness. The algorithm returns true iff (N, M_0) is deadlock-free. It runs in time $O(|P| + |T|)$.

Proof. By the Deadlock-freedom and liveness lemma, (N, M_0) is deadlock-free iff some transition is live. By Facts 1 and 2 a transition is live iff $t \notin D_{\geq}$: if $t \in D_{\geq}$, it fires at most B_t times and is not live, and otherwise it is live. By the Downstream search lemma Line 9 returns true iff $D_{\geq} \neq T$. The set Z is computed in linear time by the Unmarked circuits lemma of the liveness section and Tarjan's algorithm [51].

Corollary. If N is strongly connected, the algorithm returns true iff N has no unmarked circuit. If N has a source transition (a transition with an empty preset), the algorithm returns true.

Proof. In a strongly connected net every transition is reachable from every circuit, so $D_{\geq} = T$ iff $Z \neq \emptyset$. A source transition has no predecessors, so it is not in $D_{\geq}$.

Declaration of Academic Integrity

I hereby declare that the thesis submitted is my own unaided work. All direct or indirect sources used are acknowledged as references.

I am aware that the thesis in digital form can be examined for the use of unauthorized aid and in order to determine whether the thesis as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future theses submitted. Further rights of reproduction and usage, however, are not granted here.

This thesis was not previously presented to another examination board and has not been published.

Garching, 02.11.2026 Michael Dyer